CS 591 Lab Notebook

Lab 5

Name: Jarren Calizo
OdinID: calizo
Date: May 17, 2025

Table of Contents

CS 591 Lab Notebook **0**
Lab 5 0
Table of Contents 1
Lab 5.1 Network forensics 3
Step 1–2: Wireshark Setup and ICMP Echo Request Filtering 3
Step 3: ARP and MAC Addresses 5
Step 4: Examining Payloads (DHCP, DNS, NBNS) 8
Step 5: Filtering Out Connections 9
Step 6: Reconstructing Connections 11
Step 7: Discovering Account Information 15
Step 8: Reconstructing Files 18
Step 9: Post-Infection Activity 21
Lab 5.2 Scanning 23
Step 1: CIDR and Address Ranges 23
Step 2: Subnetworks 25
Step 3: Enterprise Server Setup 26
Step 4: Kali VM and Default Gateway Recon 26
Step 5: Service Version Fingerprinting and TCP/UDP Enumeration with Nmap 28
Step 6: iptables Host-Based Filtering 31
Step 7: iptables Port-Based Filtering 33
Step 8: iptables Outgoing Traffic Filters 36
Step 9–10: Reducing the Attack Surface by Removing TCP Services 38
Step 11: Disabling Remaining TCP Services (LDAP, IPP, MySQL) 42
Step 12: Removing Services (UDP) 45
Step 13: Validation 48
Step 14: Hiding Scanning 49
Lab 5.3 Virtual Private Networks 52
Step 1: VPNs Overview 52
Step 2: Launch VPN Access Server Deployment 53
Step 3: Configure VPN Server 54
Step 4: Configure VPN Settings 54
Step 5: Download and Install VPN Client 55
Step 6: Connect Using the VPN 56
Step 7: Clean up 57
Lab 5.4 Network OSINT 58
Step 1: Understanding OSINT in Network Reconnaissance 58

Step 2: IP Address Activity and OSINT Reconnaissance 58
Step 3: Using ipinfo.io for Aggregated OSINT Analysis 60
Step 4: getipintel API - Querying IP Intelligence Programmatically 62
Step 5: getipintel API - Using Output Programmatically 63
Step 6: DNSBL and Threat Intelligence Platform Lookups 64
Step 7: Shodan and Censys Reconnaissance 67
Steps 8 & 9: Wigle Geolocation Intelligence 69
Lab 5.5 DNS and user OSINT 71
Steps 1–3: Shodan Setup & recon-ng Domain Recon 71
Step 4: Shodan Host Enumeration via recon-ng 73
Step 5: Subdomain Enumeration with Knockpy 75
Step 6: Analysis with jq 76
Step 7: DNSDumpster Enumeration of pdx.edu 78
Step 8: Analysis of tryhackme.com 80
Step 9: Historical Certificate Analysis with crt.sh 82
Step 10: ViewDNS & WHOIS Historical Reconnaissance 86
Step 11 & 12: User and Social Media Intelligence 87
Step 12: Social Media & Source Code Mining 88
Step 13: crosslinked Enumeration 89
Step 14: recon-ng Whois PoC Mining 91

Lab 5.1 Network forensics

Step 1-2: Wireshark Setup and ICMP Echo Request Filtering

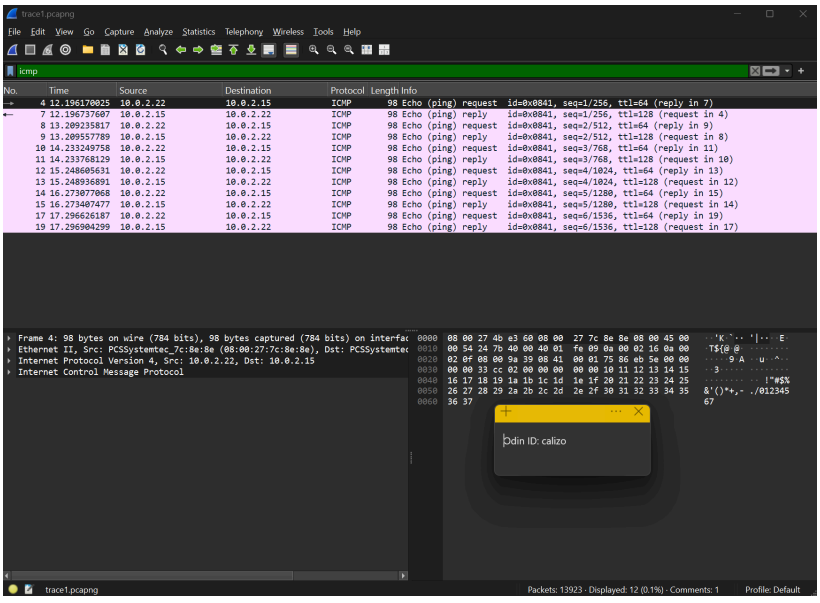
To kick off this network forensics lab, I began by setting up Wireshark, a packet analysis tool widely used in industry for traffic inspection and protocol-level investigation. After installing the latest version from wireshark.org and downloading the instructor-provided trace file (trace1.pcapng), I opened the file to begin exploring the traffic capture.

The trace file contains roughly 14,000 packets, making manual inspection inefficient. To isolate relevant communication patterns, I applied display filters within the Wireshark GUI.

Step 2.1 - Display All ICMP Packets
I first applied the display filter:

- icmp

This revealed a subset of the total packet stream, specifically ICMP traffic consisting of both Echo Requests and Echo Replies. ICMP (Internet Control Message Protocol) is often used for diagnostic purposes such as ping, which tests host reachability and round-trip time.



In my filtered results (see Screenshot 1), I observed alternating ICMP messages:

- Echo (ping) request packets from 10.0.2.22 to 10.0.2.15
- Corresponding Echo (ping) reply packets from 10.0.2.15 to 10.0.2.22

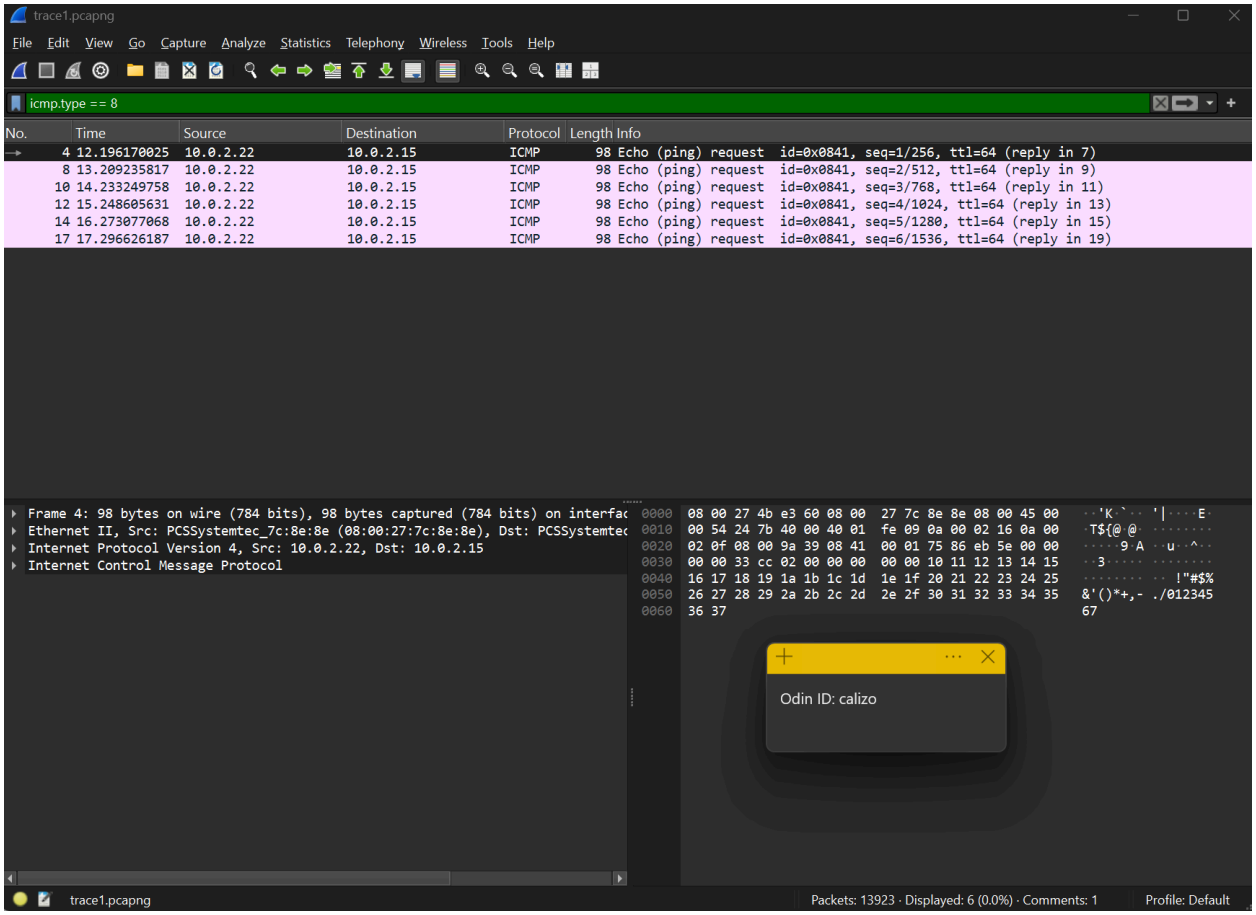
The packet summary showed details like id, sequence number, and ttl (time to live), which were helpful for identifying request-reply pairs and analyzing timing behavior.

At the bottom of Wireshark, the packet counter confirmed the trace contained 13,923 total packets, of which 12 ICMP packets (≈0.1%) matched the display filter.

Step 2.2 - Filter for Only ICMP Echo Requests
To further refine my analysis, I applied a protocol-specific filter to isolate just Echo Request packets:

- icmp.type == 8

This filter targets ICMP Type 8 messages, which represent Echo Requests.



Using Wireshark's GUI, I selected an Echo Request packet and right-clicked the Type field under the Internet Control Message Protocol section. I then chose "Apply as Filter → Selected" to auto-populate the correct syntax. The filtered view returned 6 ICMP Echo Request packets, as shown in Screenshot 2.

This narrowed display helps focus on initial probes sent out during the ping process, which is essential for identifying the origin of reconnaissance or monitoring behavior during intrusion analysis.

Answers to Numbered Instructions from Step 2

1. What display filter did you use to show all ICMP traffic?
   - icmp

2. What display filter did you use to show only ICMP Echo requests?
   - icmp.type == 8

3. What is the total number of packets in the trace?
   - 13,923 packets

4. How many packets are shown after applying the icmp filter?
   - 12 packets

5. How many packets are shown after applying the icmp.type == 8 filter?
   - 6 packets

Step 3: ARP and MAC Addresses

To identify active hosts in the network and associate them with hardware (MAC) addresses, I analyzed ARP reply traffic within the packet capture. ARP (Address Resolution Protocol) facilitates IP-to-MAC resolution, and ARP replies provide a direct mapping between IP addresses and their corresponding MAC addresses on the local network segment.

Filtering ARP Packets

I began by applying the display filter:

- arp

This isolated all ARP packets in the capture. To focus only on replies—which contain the sender's IP–MAC mapping—I right-clicked on an ARP reply's Opcode: reply (2) field and selected "Apply as Filter → Selected," which produced the refined filter:

- arp.opcode == 2

This filter narrowed the results to 62 ARP replies (see Screenshot 1). These replies were sufficient to extract unique device identities across the network.

Extracting Unique Host Mappings

From the ARP replies, I identified four unique hosts based on their IP addresses and MAC identifiers. Each reply displayed the following relevant fields in the middle pane:

- Sender MAC address
- Sender IP address

After stepping through a few representative replies (Screenshots 2–5), I recorded the following mappings:

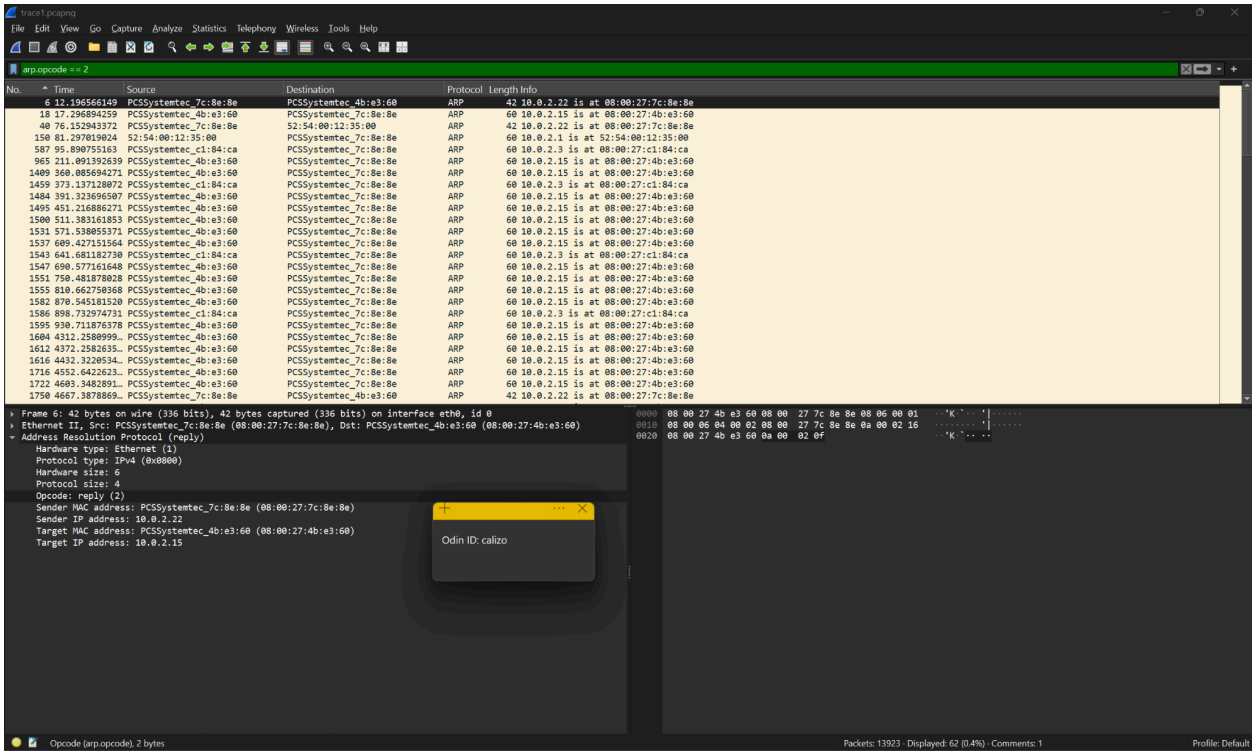
Answers to Step 3 Questions

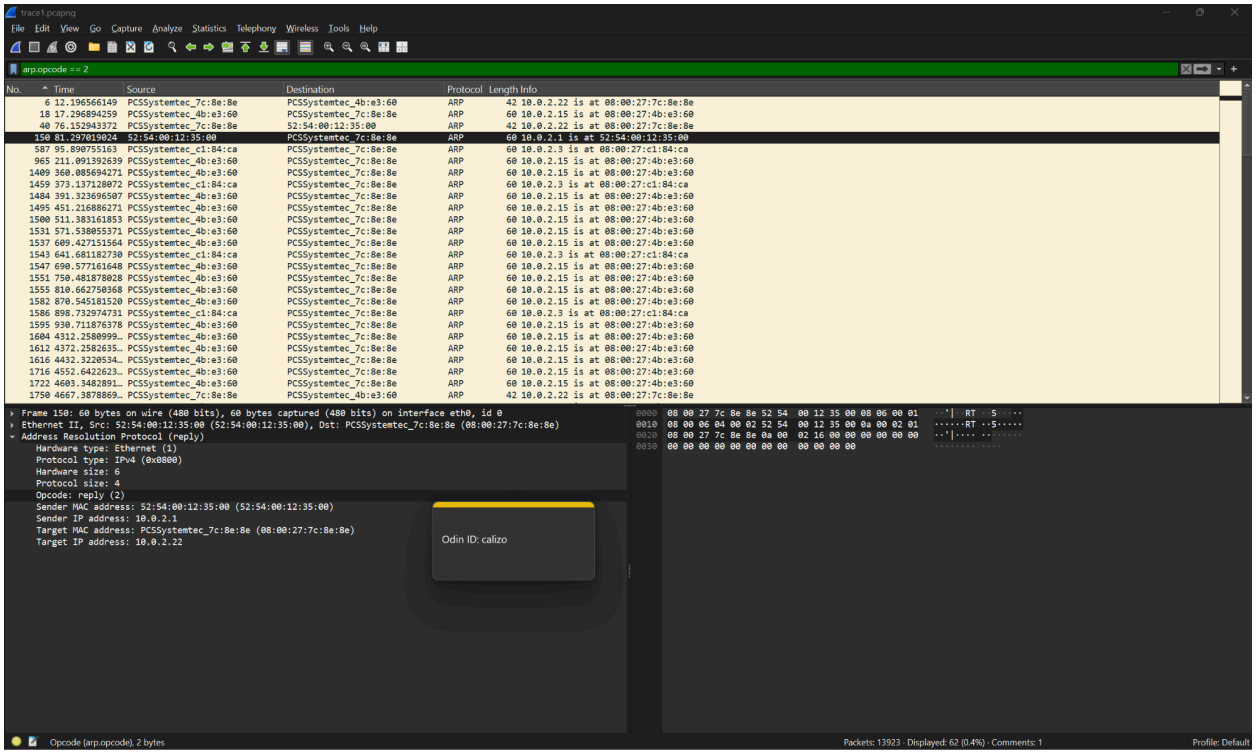
1. What is the display filter that has been applied?

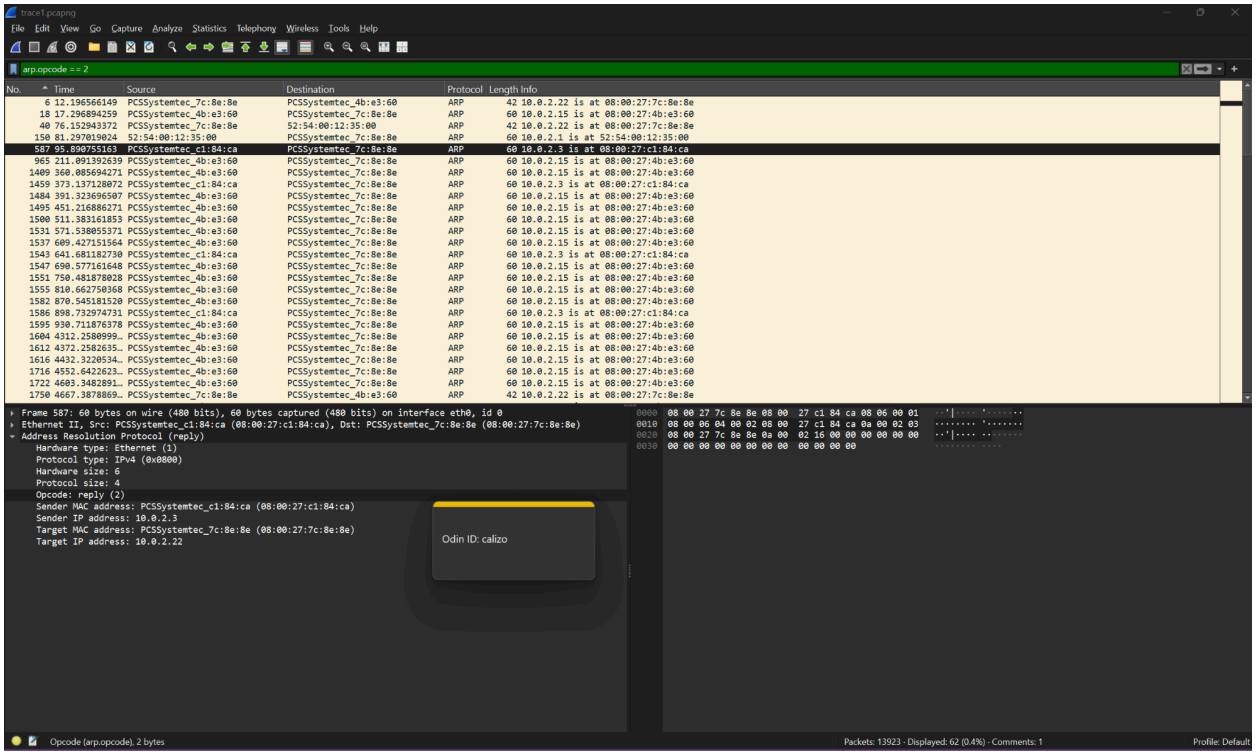
- arp.opcode == 2

2. List the IP addresses of each host and its corresponding MAC address:

IP Address	**MAC Address**
10.0.2.1	52:54:00:12:35:00
10.0.2.3	08:00:27:c1:84:ca
10.0.2.15	08:00:27:4b:e3:60
10.0.2.22	08:00:27:7c:8e:8e







These IP↔MAC mappings confirm the presence of four distinct networked devices captured in the trace. The recurring ARP replies validate their activity, enabling further host-based correlation in upcoming analysis steps.

Step 4: Examining Payloads (DHCP, DNS, NBNS)

In this step, I inspected several plaintext protocols that reveal system-level metadata through unencrypted packet payloads. Even though encryption like TLS protects much of modern web traffic, underlying protocols such as DHCP, DNS, and NBNS remain exposed, providing valuable insights for network defenders and attackers alike.

DHCP Traffic (Display filter: dhcp)
Upon filtering with dhcp, I found a DHCP Request packet from host 10.0.2.22. Expanding the Dynamic Host Configuration Protocol section in Wireshark revealed the following:

- Client MAC: 08:00:27:7c:8e:8e
- Requested IP: 10.0.2.3
- Hostname: kali (under DHCP Option 12: Host Name)

This hostname implies a Kali Linux machine, which can often indicate a penetration testing or adversarial toolset in network forensics.

DNS Queries (Display filter: dns)
With the filter dns, I observed several Standard Query packets.

One such query was for the following:

- Record Type: PTR (Pointer Record)
- Domain Name Queried: 15.2.0.10.in-addr.arpa

This is a reverse DNS lookup targeting the IP 10.0.2.15.

- DNS Server Queried: 103.86.96.100

This address is external to the 10.x.x.x private address space and could suggest upstream DNS resolution or tunneling behavior.

NBNS (NetBIOS Name Service) – SMB Discovery (Display filter: nbns)
SMB-based file sharing on Windows systems advertises hosts via NetBIOS broadcasts.

Under the nbns filter, I discovered the following broadcast query:

- Queried Host Name: WORKGROUP<1b>
- Query Type: NB (NetBIOS)
- Source: 10.0.2.15

The <1b> suffix indicates a query to the Domain Master Browser, a typical behavior of Windows SMB hosts announcing themselves or resolving other machines in the workgroup.

Answers to Step 4 Questions

What is the IP address of the DHCP server?

Not explicitly listed in the DHCP Request packet. However, the DHCP ACK/Offer response later in the trace likely shows it as 10.0.2.3.

What is the IP address assigned and the hostname of the machine making the DHCP request?

- IP Address Assigned: 10.0.2.3
- Hostname: kali

What DNS record type and DNS name are being requested?

- Record Type: PTR
- DNS Name: 15.2.0.10.in-addr.arpa

What is the IP address of the local DNS server the request is being made to?

- 103.86.96.100

What is the name of the host being broadcast?

- WORKGROUP<1b>

Step 5: Filtering Out Connections

To distinguish encrypted from analyzable traffic, I focused on TCP-based connections within the trace and refined my understanding of what data is accessible in plaintext. I used Wireshark's filtering and stream indexing to examine specific traffic flows, beginning with TLS-based sessions and filtering out encrypted services to isolate actionable content.

Identifying the First TLS Session (Stream Index 0)

After applying the basic TCP filter:

- tcp

I navigated to the very first TCP packet in the trace. Stream Index 0 (seen in packet #26) initiates a connection between 10.0.2.22 and 10.0.2.15 on destination port 3389. Upon looking this port up, I confirmed that:

- TCP Port 3389 is used by Microsoft Remote Desktop Protocol (RDP).

Following the TCP stream, the packet sequence quickly escalates into a TLSv1 handshake, as shown by the appearance of Client Hello, Server Hello, and Encrypted Handshake Message packets. This confirms that RDP traffic in this trace is TLS-encrypted, making payload inspection impractical beyond session setup.

Quantifying RDP (TCP/3389) Traffic
I then applied:

- tcp.port == 3389

Wireshark returned 12,922 matching packets out of 13,923 total, indicating that RDP traffic dominates the trace.

12922 / 13923 * 100 ≈ 92.8%

- 92.8% of all packets in this trace are encrypted RDP traffic over port 3389.

Filtering Out Encrypted RDP Traffic
To exclude TLS-based RDP traffic and reveal other potentially plaintext TCP streams, I applied:

- tcp and not tcp.port == 3389

This reduced the displayed packet count to 258 TCP packets — a much more manageable number for deep packet inspection.

Interestingly, these remaining TCP streams included:

- Communication between 10.0.2.15 and 10.0.2.22 over port 8888
- Payloads containing readable ASCII strings like ESTABLISHED, SESSION, and COMMAND, suggesting that this traffic is:
    - Unencrypted, and
    - Likely related to backdoor or remote shell activity.

This highlights how filtering encrypted ports can help spotlight suspicious plaintext activity that might otherwise be drowned out by noise.

<u>Answers to Step 5 Questions</u>
1. What kind of session does this connection carry?

- It is a Remote Desktop Protocol (RDP) session secured over TLSv1 (port 3389).

2. What percentage of packets in this trace carry traffic of this kind?

- 92.8% of packets are RDP traffic over TCP port 3389.

3. How many TCP packets remain after filtering out 3389? Do any of them carry encrypted TLS traffic?

- 258 TCP packets remain, and none appear to carry encrypted TLS traffic. The remaining packets show readable ASCII payloads, suggesting plaintext communication, possibly from a backdoor or netcat-based session.

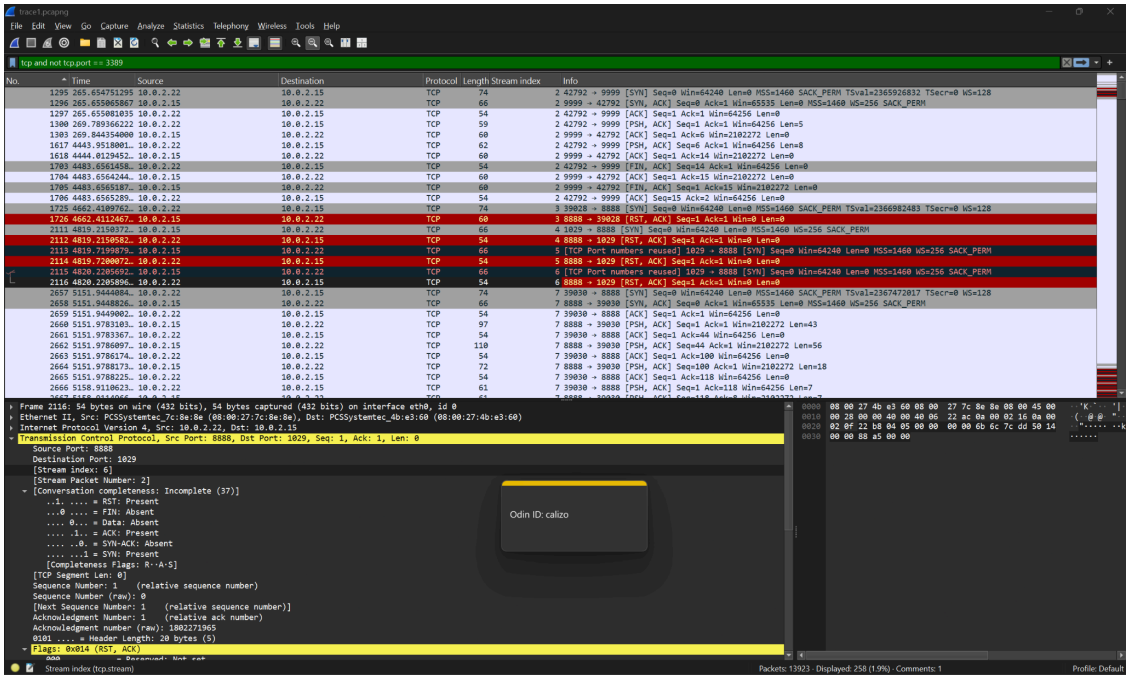
Step 6: Reconstructing Connections

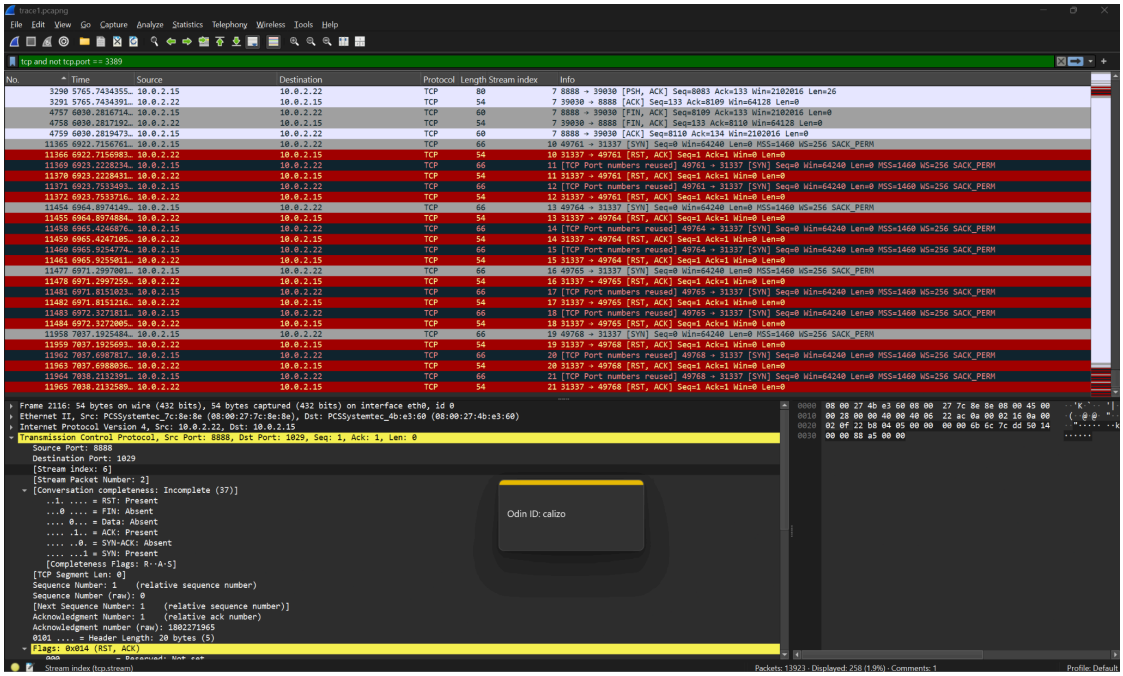
To trace adversary activity and uncover potential data exfiltration or system reconnaissance, I leveraged TCP stream analysis and Wireshark’s stream indexing. By isolating specific sessions and following their payloads, I was able to observe unencrypted shell activity and sensitive file listings—providing strong evidence of compromise.

Stream Index Column Setup
After applying the display filter:

- tcp and not tcp.port == 3389

I clicked on a TCP packet and right-clicked the Stream index field under the Transmission Control Protocol section, selecting “Apply as Column”. This added a "Stream" column to the top view (Screenshot 1), helping me easily group packets by session. This step was crucial for tracking conversations such as stream index 2 (testing connection) and 7 (the backdoor shell).





Reconstructing TCP Stream #2

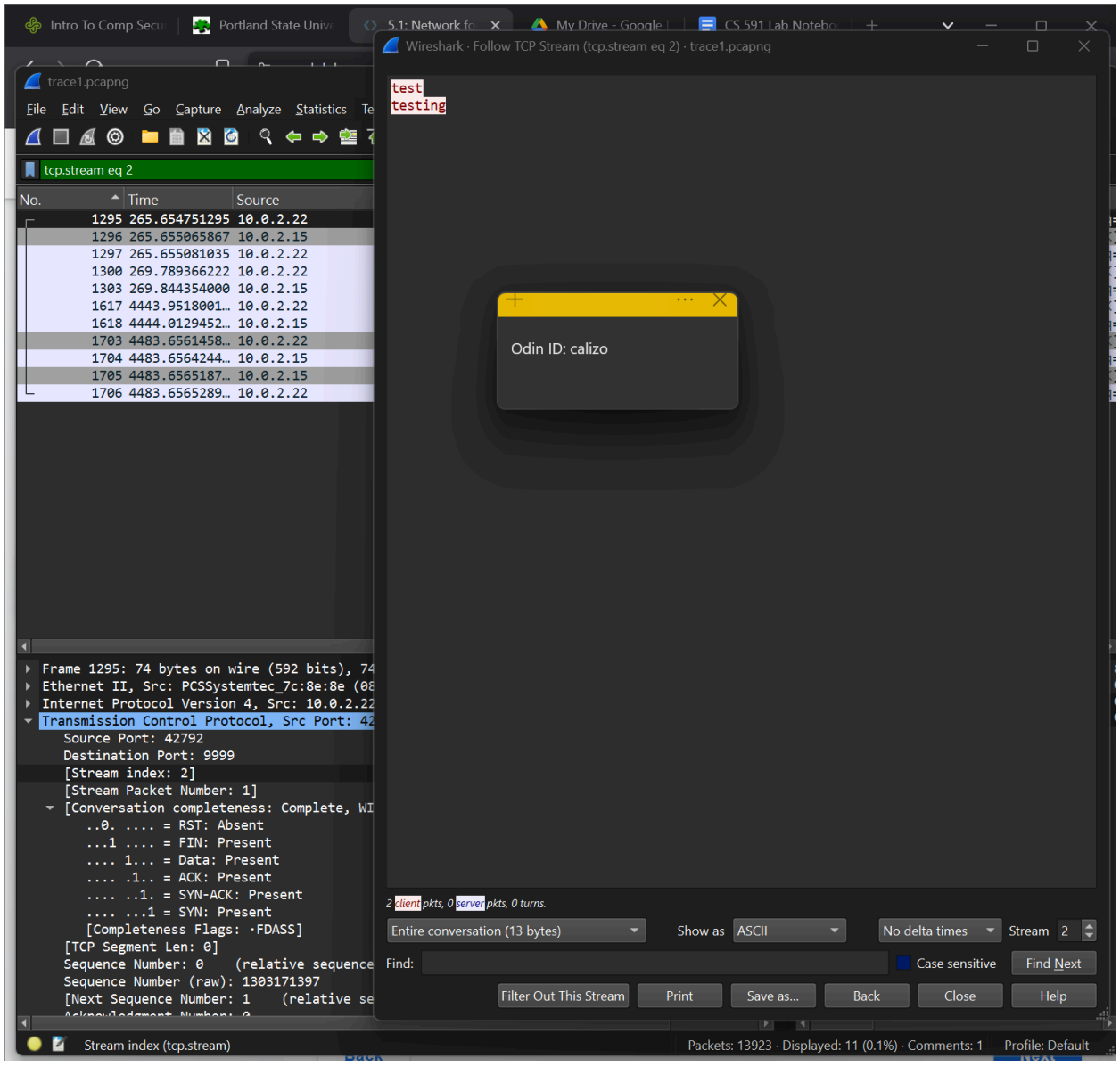
I right-clicked a packet with Stream index 2 and selected:

- Follow → TCP Stream

The reconstructed payload revealed a benign test exchange between 10.0.2.22 and 10.0.2.15, with simple plaintext strings like:

test

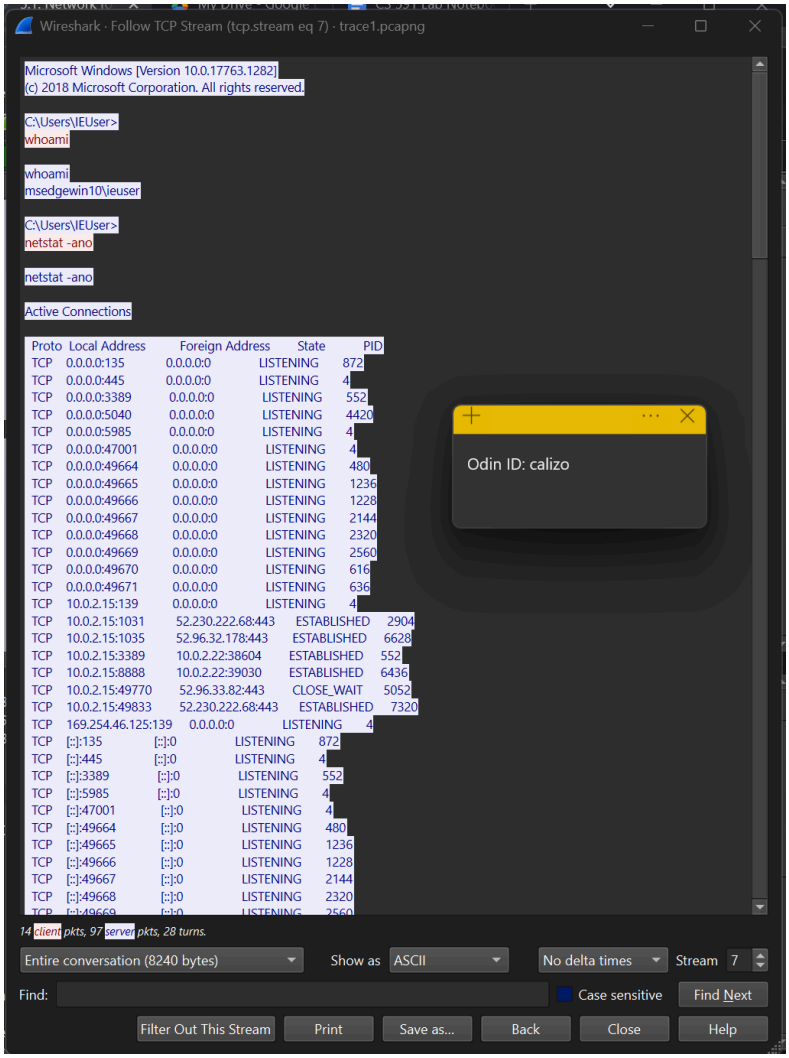
testing



Reconstructing TCP Stream #7 - Backdoor Shell

Next, I followed Stream index 7, which revealed an unencrypted Windows command shell. The adversary had remote access and issued several commands:

- whoami: returned msedgewin10\euser
- netstat -ano: revealed active listening ports and established connections
- cd Documents, dir: enumerated files and folders



From the netstat output, I identified services running on various ports and several established connections with external IPs.

Answers to Step 6 Questions

1. From the netstat output, what services are running on the first two TCP ports in the LISTENING state?

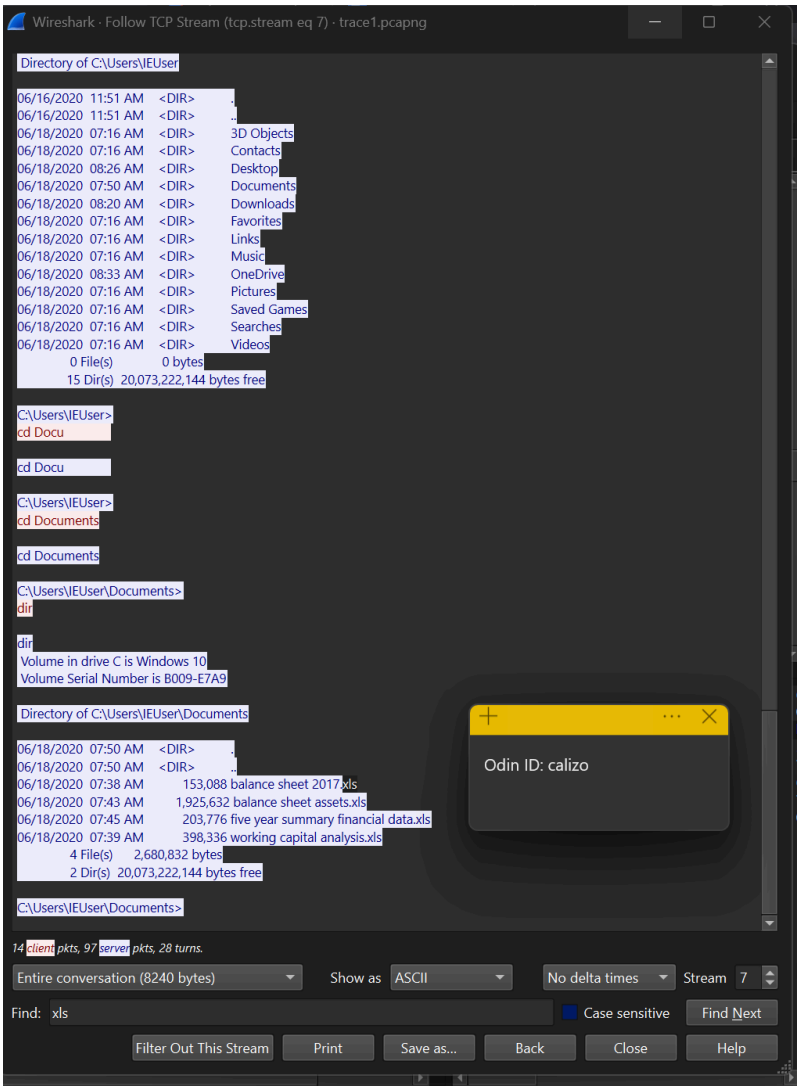
Port	Service
135	RPC Endpoint Mapper
445	Microsoft-DS (SMB over TCP)

These are standard services in a Windows environment and commonly targeted for lateral movement or exploitation.

2. Take a screenshot of the Excel (.xls) files that have been discovered.

From Stream 7’s directory listing under C:\Users\EUser\Documents, the adversary issued a dir command that returned:

- balance sheet 2017.xls
- balance sheet assets.xls
- five year summary financial data.xls
- working capital analysis.xls

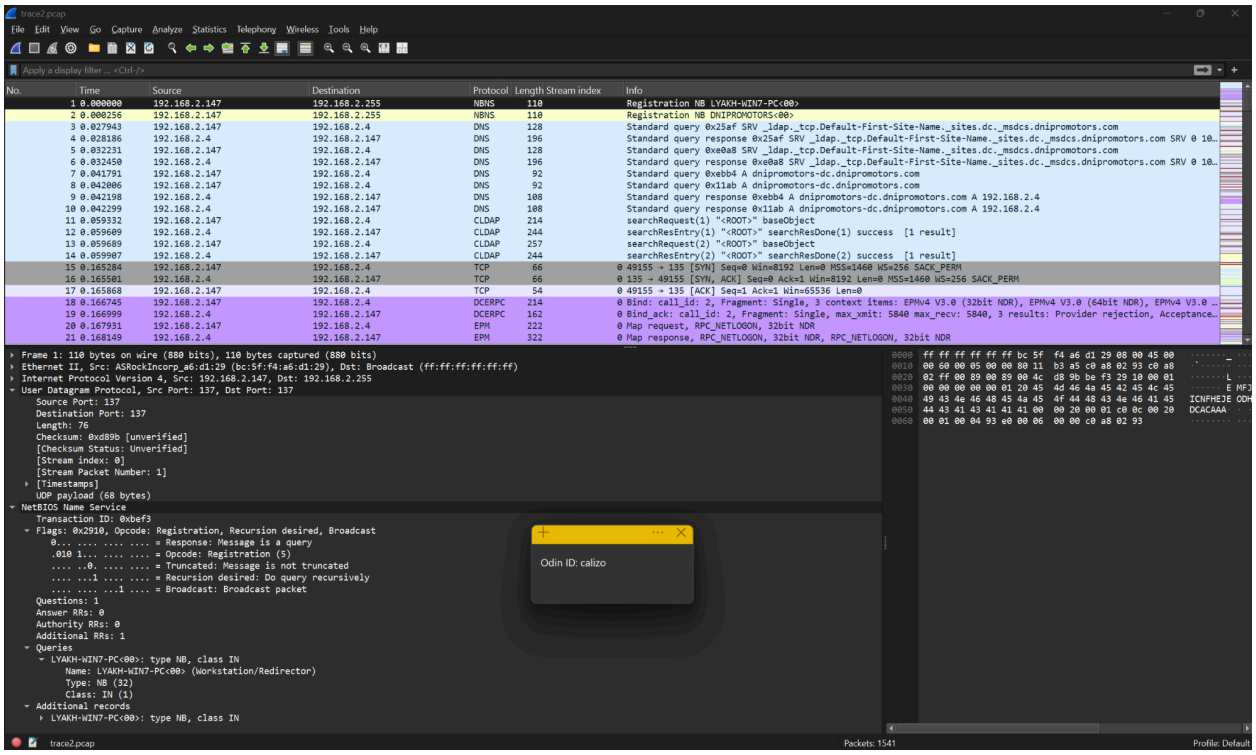


Step 7: Discovering Account Information

In this portion of the lab, we shifted over to trace2.pcap to inspect system identity broadcasting and early-stage authentication traffic. The focus was on correlating NetBIOS and DNS lookups with domain controller behavior to identify key hosts and credentials used in a Kerberos-based login attempt.

1. What is the IP address and name of the Windows Desktop machine on this network? What Windows domain is it attempting to register on?

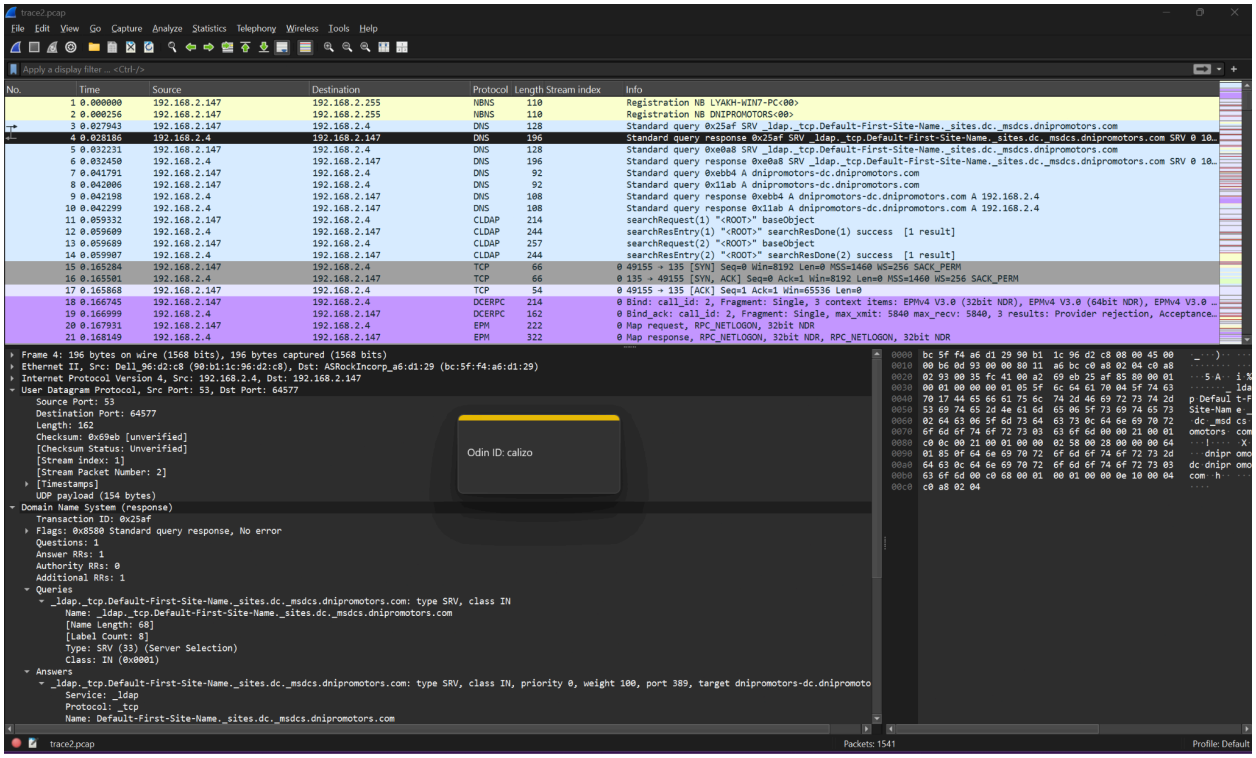
- The Windows Desktop machine has an IP address of 192.168.2.147.
- The hostname is LYAKH-WIN7-PC, as seen in the NBNS registration packets at the top of the trace.
- It attempts to register on the DNIPROMOTORS domain, which is clearly reflected in its broadcast NBNS registration message (Registration NB DNIPROMOTORS<00>).



Method: Manual recon using NBNS packet inspection in the first 10 frames of trace2.pcap. Stream indices 0 and 1 (broadcasts) contain this information.

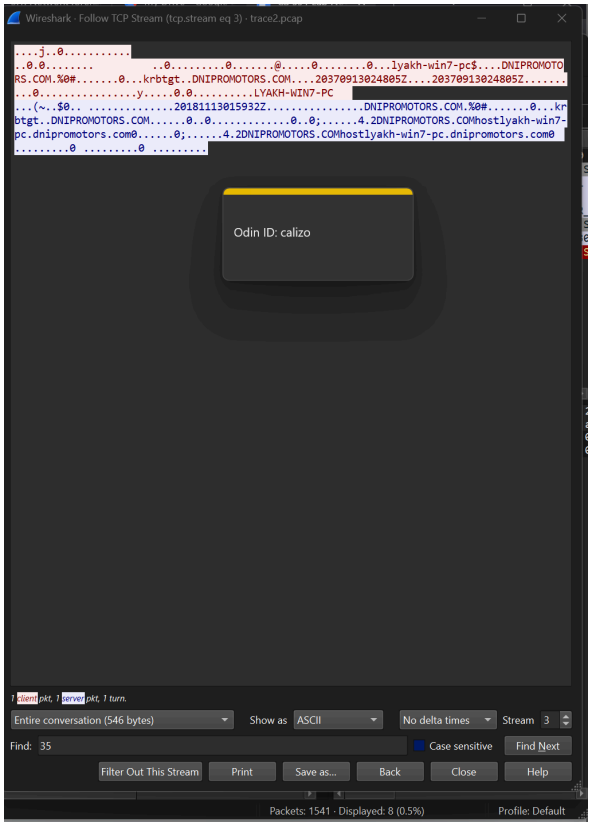
2. What is the IP address of the local DNS server and domain controller?

- The local DNS server and domain controller is 192.168.2.4, as revealed in the DNS query response (Transaction ID 0xeab8) resolving dnipromotors-dc.dnipromotors.com to this IP address.
- Both domain controller and DNS server roles are inferred from the SRV record for _ldap._tcp.Default-First-Site-Name._sites.dc._msdcs.dnipromotors.com, which resolves to the same hostname.



Method: Manual recon through DNS packet filtering (dns) and expanding the response payload for SRV/A records.

3. What username is attempting to authenticate?
    - The Kerberos authentication packet from Stream 3 reveals that the user attempting authentication is lyakh-win7-pc$@DNIPROMOTORS.COM, which is a machine account credential used during domain join or logon initialization.
    - This is found inside the ASCII-decoded payload of the Kerberos ticket-granting exchange (krbtgt@DNIPROMOTORS.COM), which occurs in TCP streams 35 through 37.



Method: Right-click > Follow TCP Stream on streams 35–37. View as ASCII. The credentials and ticketing data are visible directly in the payload.

This step demonstrated how even encrypted protocols can reveal user and domain metadata through unencrypted wrappers like NBNS, DNS, and initial Kerberos ticket headers. The combination of early-stage packet inspection and stream reconstruction in Wireshark was key for passive reconnaissance on the network’s trust relationships.

Step 8: Reconstructing Files

In this step, the objective was to identify and reconstruct a suspicious executable file downloaded via an unencrypted HTTP session, and then analyze its potential maliciousness using VirusTotal.

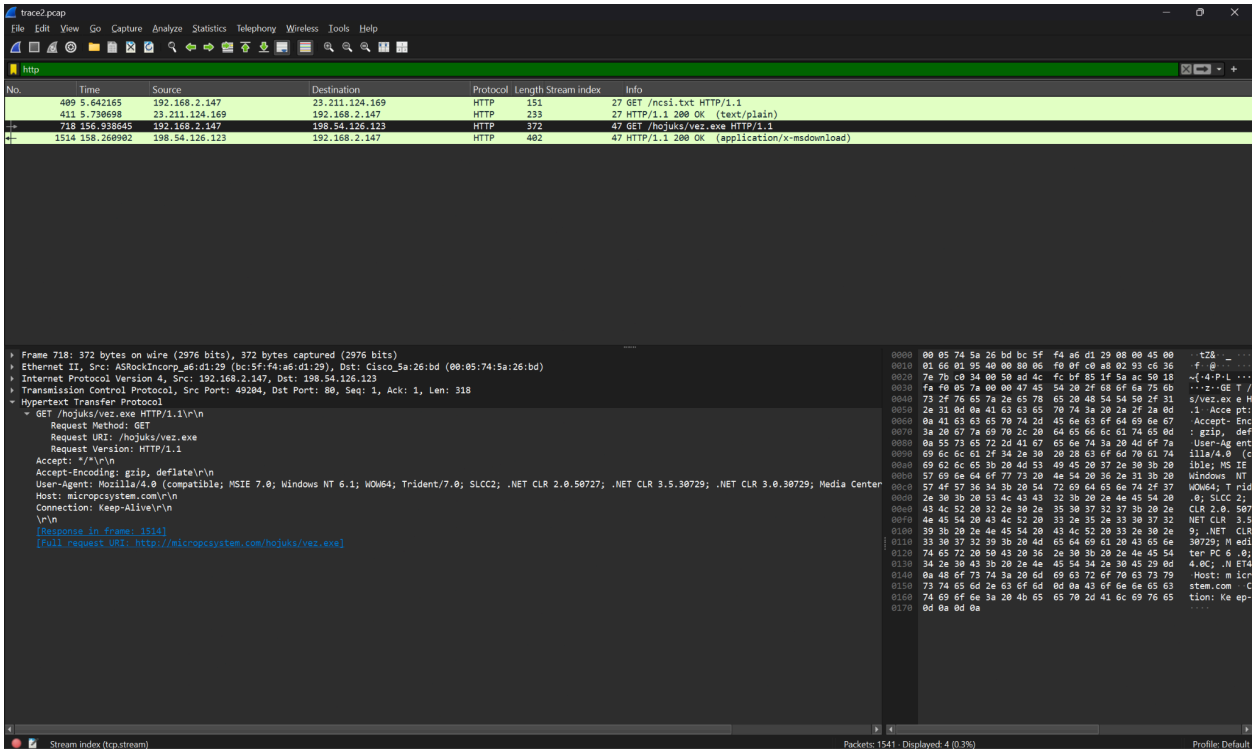
Manual Recon Breakdown:

- Initial Filter: I applied http as a Wireshark display filter to isolate HTTP traffic.
- I located the suspicious download in Stream index 47, identified by the URI /hojuks/vez.exe.

- Right-clicked one of the packets, selected Follow → TCP Stream, and examined the content to confirm that it was a Windows executable by spotting the classic MZ and PE header bytes within the ASCII view.
- I then used the "Save as…" option in the stream window to save the full response as vez.exe.
- Finally, I uploaded the file to VirusTotal and reviewed the scan results.

Questions & Answers:

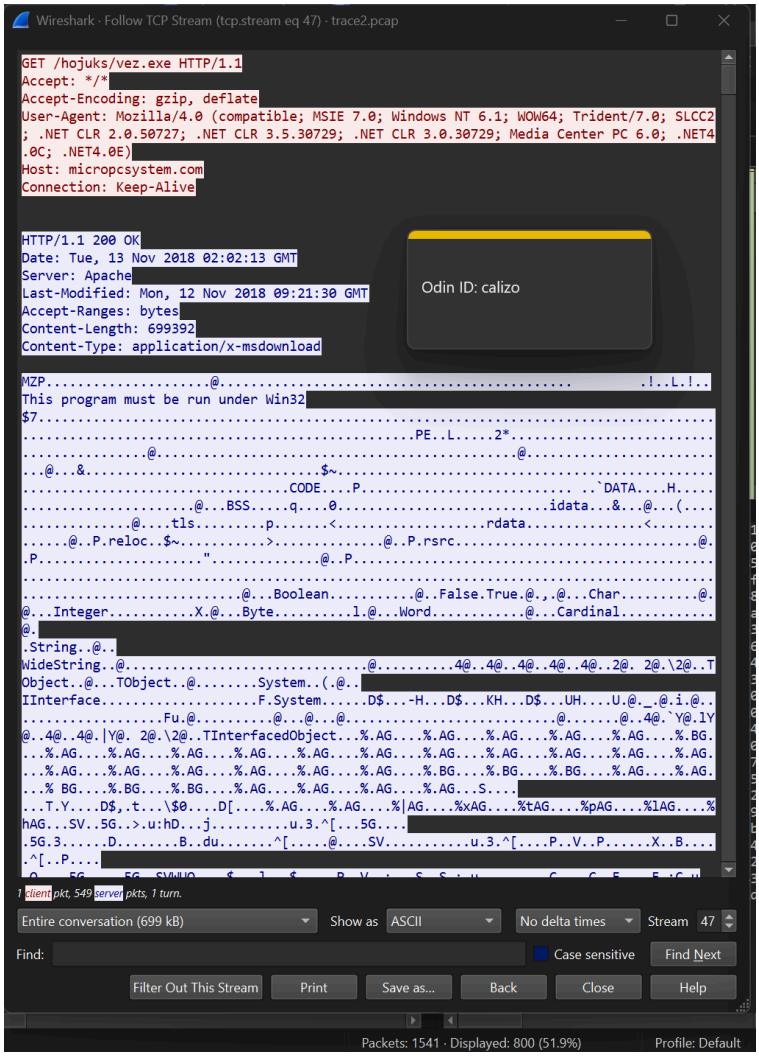
1. What is the Full request URI of the downloaded file?



- http://micropcsystem.com/hojuks/vez.exe

This is seen in the HTTP GET request under Stream Index 47.

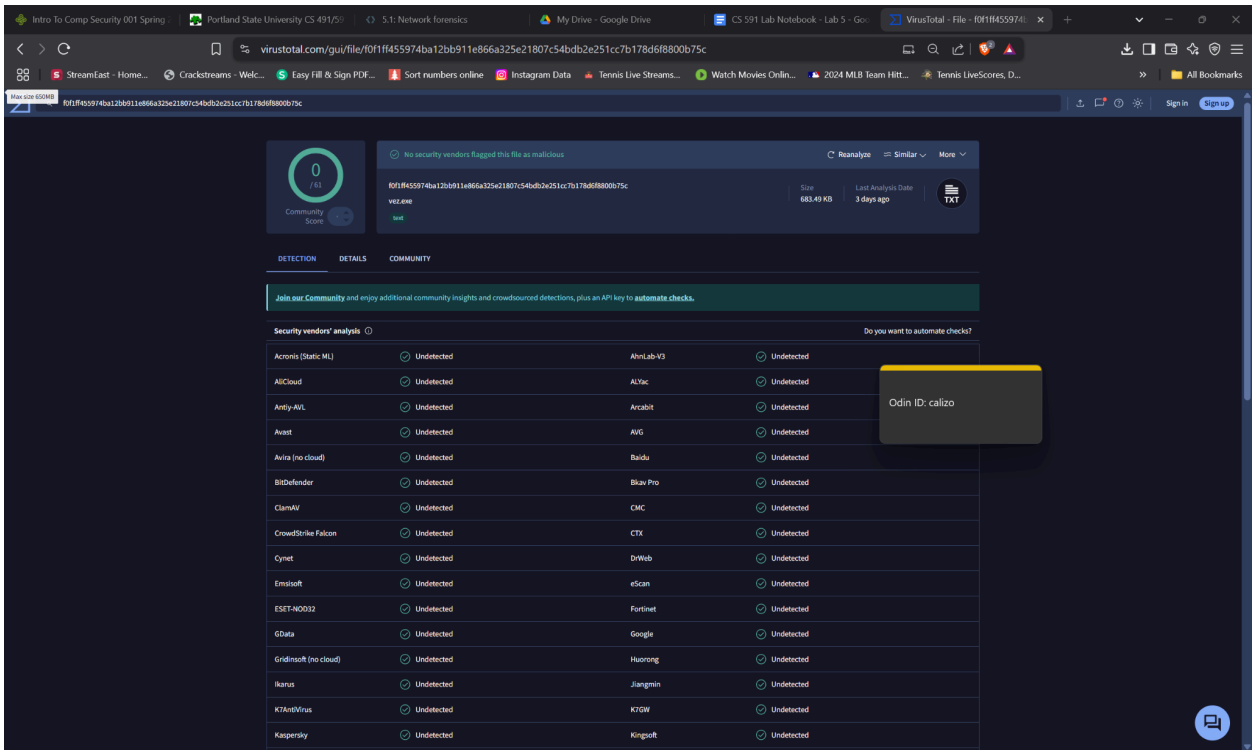
2. What are the Content-Length and Content-Type of the response?



- Content-Length: 699392
- Content-Type: application/x-msdownload

These values are present in the HTTP 200 OK response headers in the same stream.

3. Based on what you find, explain why you think the executable is or is not malicious.



After uploading vez.exe to VirusTotal, 0 out of 61 security vendors flagged it as malicious.

That said, while no AV engines detected it, we can’t confidently say it’s safe—a lack of detection doesn’t equal benign behavior. It could still be part of a more subtle attack, be packed, or just evade static analysis.

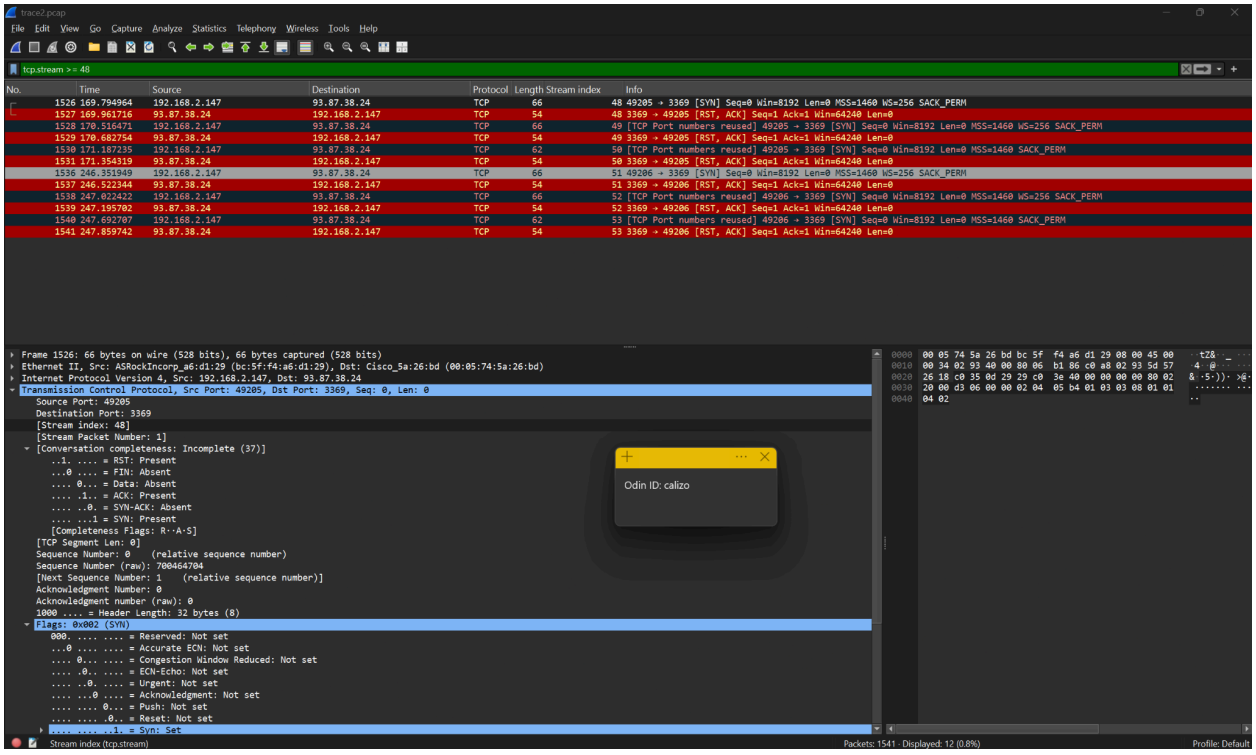
In practice, further behavioral analysis in a sandboxed environment would be necessary before making definitive judgments about its safety.

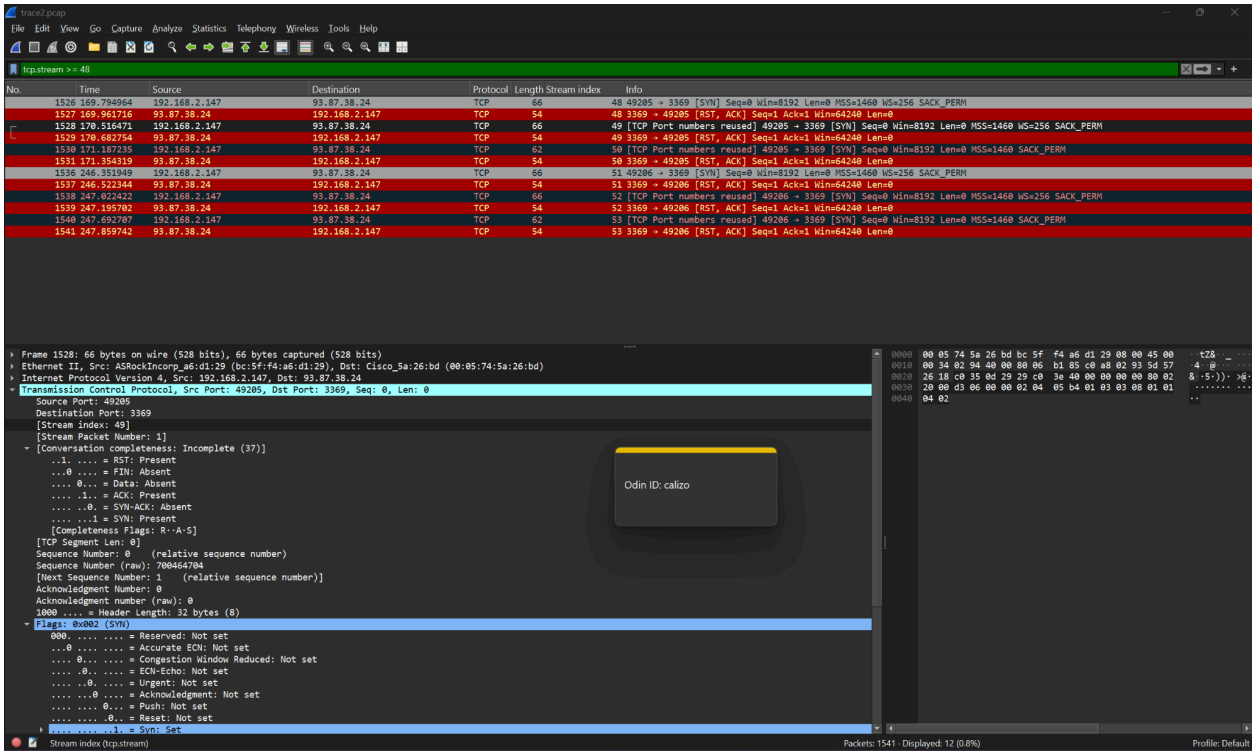
Step 9: Post-Infection Activity

After reconstructing the HTTP stream that delivered the suspicious binary vez.exe, I continued to trace the post-download behavior of the compromised machine. Using the stream index value (47) from the TCP stream that downloaded the executable, I then examined the immediately following stream indices, specifically Stream 48 and Stream 49, to uncover any outbound activity that occurred as a direct consequence of the infection.

By applying the filter tcp.stream eq 48 and tcp.stream eq 49, I observed that both streams show failed TCP connection attempts initiated by the internal host (192.168.2.147) to a remote IP address. The target of these attempts was

93.87.38.24, over TCP port 3369, which is not a commonly used well-known port and could potentially indicate a C2 (command-and-control) channel or a custom backdoor service set up by an adversary.





Each of these connection attempts consisted of a SYN packet from the victim machine, followed immediately by a RST, ACK from the target, meaning the remote service either denied the connection or wasn't listening on the port at the time. Regardless, this kind of behavior is consistent with post-infection C2 probing—suggesting that vez.exe may have been designed to reach out to an attacker-controlled IP after execution.

Answer to Step 9 Question

1. Which IP address does the machine attempt to connect to after the download?

- Answer: 93.87.38.24
    - This address appears as the destination in both TCP Stream 48 and 49 immediately after the download of vez.exe. Both connection attempts were outbound from 192.168.2.147 and were unsuccessful (RST, ACK), but the timing and target indicate suspicious behavior.

Lab 5.2 Scanning

Step 1: CIDR and Address Ranges

Understanding CIDR notation is a core foundational skill when configuring or interpreting IP scanning results. In this step, we broke down IP addresses and CIDR ranges to better interpret how networks are grouped and what they represent in terms of host space. Below are my findings:

Address: 35.233.233.233

1. /24 CIDR Prefix:
    a. 35.233.233.0/24
    b. # of addresses: 256

2. /28 CIDR Prefix:
    a. We calculate the /28 by masking the last 4 bits of the fourth octet:
        i. 233 in binary = 11101001 → mask to 11100000 = 224
        ii. CIDR Prefix: 35.233.233.224/28
    b. # of addresses: 16

3. /20 CIDR Prefix:
    a. With /20, we keep the first 4 bits of the third octet:
        i. 233 (third octet) → truncate to 224
        ii. CIDR Prefix: 35.233.224.0/20
    b. # of addresses: 4096

4. Public or Private:
    a. This is a publicly routable IP address. It doesn't fall within any reserved private IP blocks.

Address: 172.29.85.85

1. /24 CIDR Prefix:
    a. 172.29.85.0/24
    b. # of addresses: 256

2. /28 CIDR Prefix:
    a. 85 in binary = 01010101 → mask to 01010000 = 80
    b. CIDR Prefix: 172.29.85.80/28
    c. # of addresses: 16

3. /20 CIDR Prefix:
    a. Third octet: 85 → mask to 80
    b. CIDR Prefix: 172.29.80.0/20
    c. # of addresses: 4096

4. Public or Private:
    a. This is a private IP address, as it belongs to the reserved range 172.16.0.0 – 172.31.255.255.

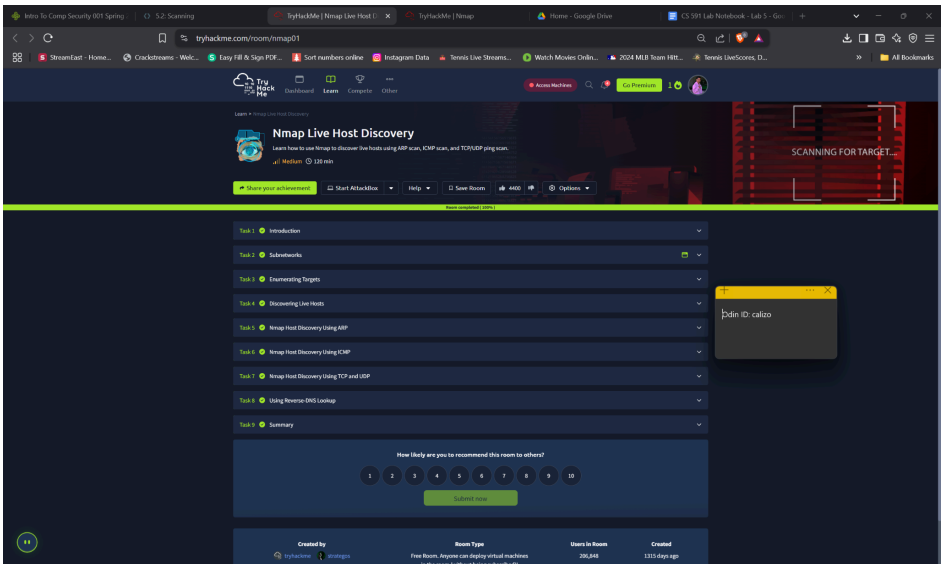
Given CIDR Prefixes

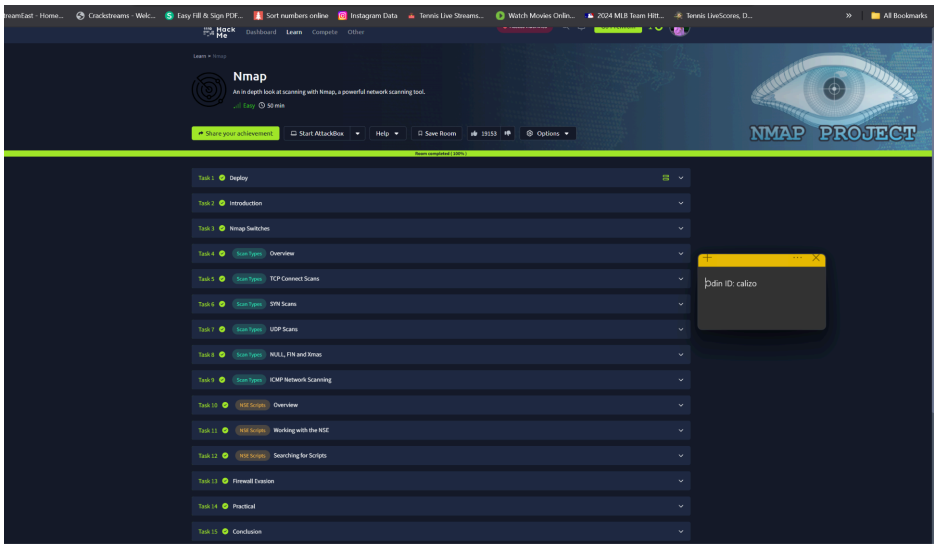
5.
    a. 131.252.220.0/26
        i. 6 host bits = 2^6 = 64
        ii. Address range: 131.252.220.0 – 131.252.220.63
    b. 172.29.80.0/20
        i. 12 host bits = 2^12 = 4096
        ii. Address range: 172.29.80.0 – 172.29.95.255

This step was a good refresh on subnetting fundamentals. Binary conversions helped clarify where each IP would fall when grouped under different CIDR masks. Being able to map these ranges mentally will definitely be useful when scanning networks or interpreting nmap results later in the lab.

Step 2: Subnetworks

Understanding subnetting is a foundational skill when working with live host discovery. This task reviewed key principles behind subnet masks, private IP address ranges, and CIDR notation. The subnet mask helps define how many IPs are part of a given network and allows Nmap (or any scanner) to determine which hosts to probe.





Key Takeaways:

- A subnet mask determines which portion of an IP address identifies the network versus the host.
- CIDR (Classless Inter-Domain Routing) notation (e.g. /24) is a compact way of writing subnet masks.
- Recognizing private IP ranges like 10.0.0.0/8, 172.16.0.0/12, and 192.168.0.0/16 is critical to knowing what’s routable internally vs. externally.

These fundamentals enable smarter scanning across internal networks and help scope out ranges to identify live targets without wasting time or alerting unnecessary traffic.

Step 3: Enterprise Server Setup

To simulate a multi-service enterprise host, I deployed a VM named enterprise-server in the us-west1-b zone using the introsec-enterprise-server custom image from the pdx-cs project.

During configuration:

- I selected the correct image under Custom Images, choosing introsec-enterprise-server.
- Under Network interfaces, I ensured it was assigned to the default network (10.138.0.0/20).
- I disabled the External IP by setting it to None, ensuring the server is only accessible internally.
- After the VM successfully launched, I verified its internal IP to be: 10.138.0.6

Step 4: Kali VM and Default Gateway Recon

From the Kali VM terminal, I used:

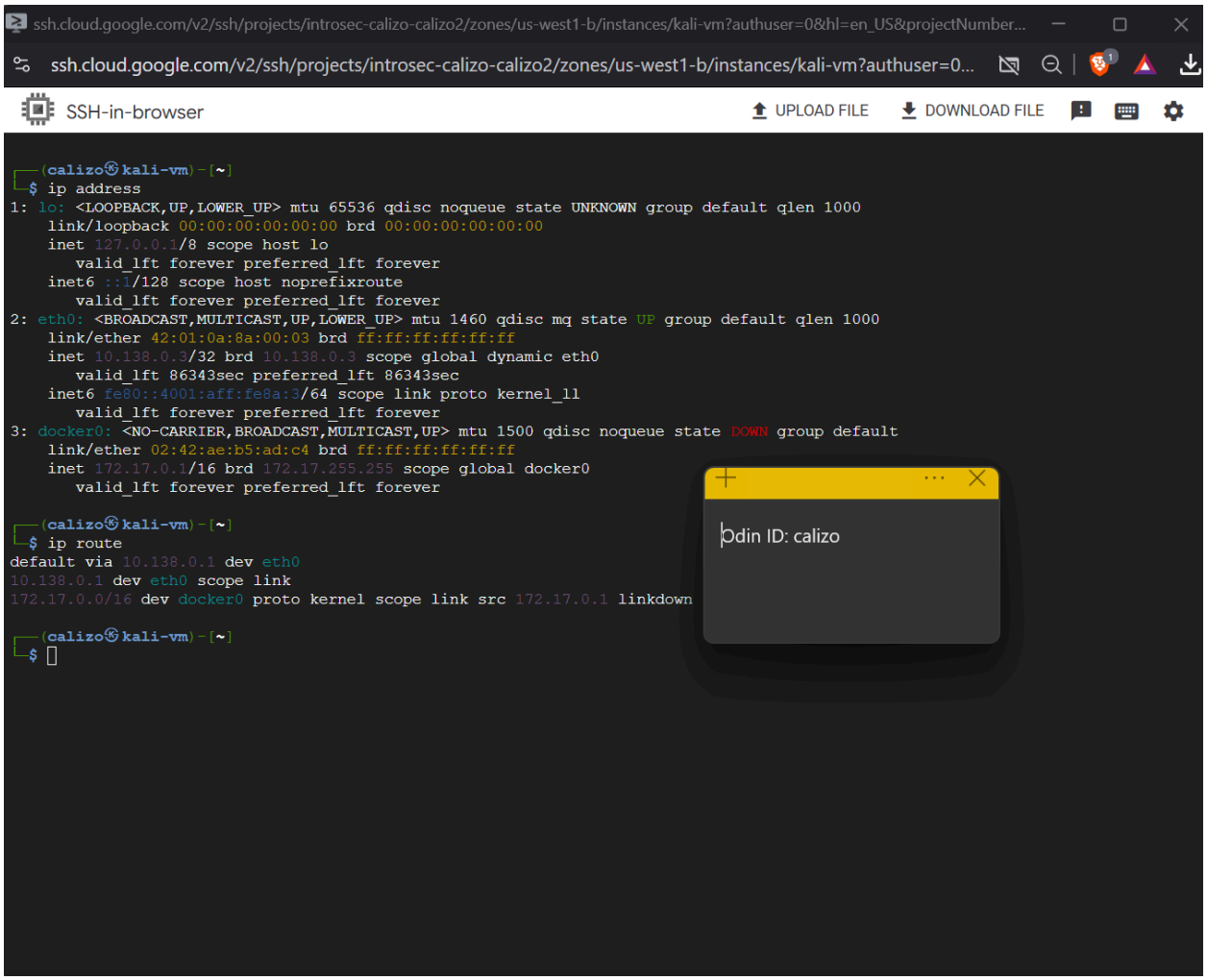
ip address to identify my Kali VM’s internal IP:

- 10.138.0.3

ip route to determine the default gateway for the internal network:

- 10.138.0.1

To perform internal scans efficiently, I calculated the smallest CIDR block that encapsulates all three IPs (10.138.0.1, .3, and .6).



Final Values:

Component	**Value**
internal_enterprise_IP	10.138.0.6
internal_kali_IP	10.138.0.3
internal_gateway_IP	10.138.0.1
internal_CIDR_prefix	10.138.0.0/29

- The /29 subnet covers IPs from .1 to .6 (with .0 as network and .7 as broadcast), which is a perfect fit.

Step 5: Service Version Fingerprinting and TCP/UDP Enumeration with Nmap

After setting up both the Kali VM and the internal enterprise-server instance, I ran Nmap fingerprinting to identify active services and their versions on the local internal subnet 10.138.0.0/29. Using the command:

- nmap -sV 10.138.0.0/29

Nmap scanned for all TCP-based services running within the subnet and produced detailed version information about the services on each live host. This step demonstrates how valuable Nmap's service version detection (-sV) is for both network inventory and vulnerability reconnaissance.

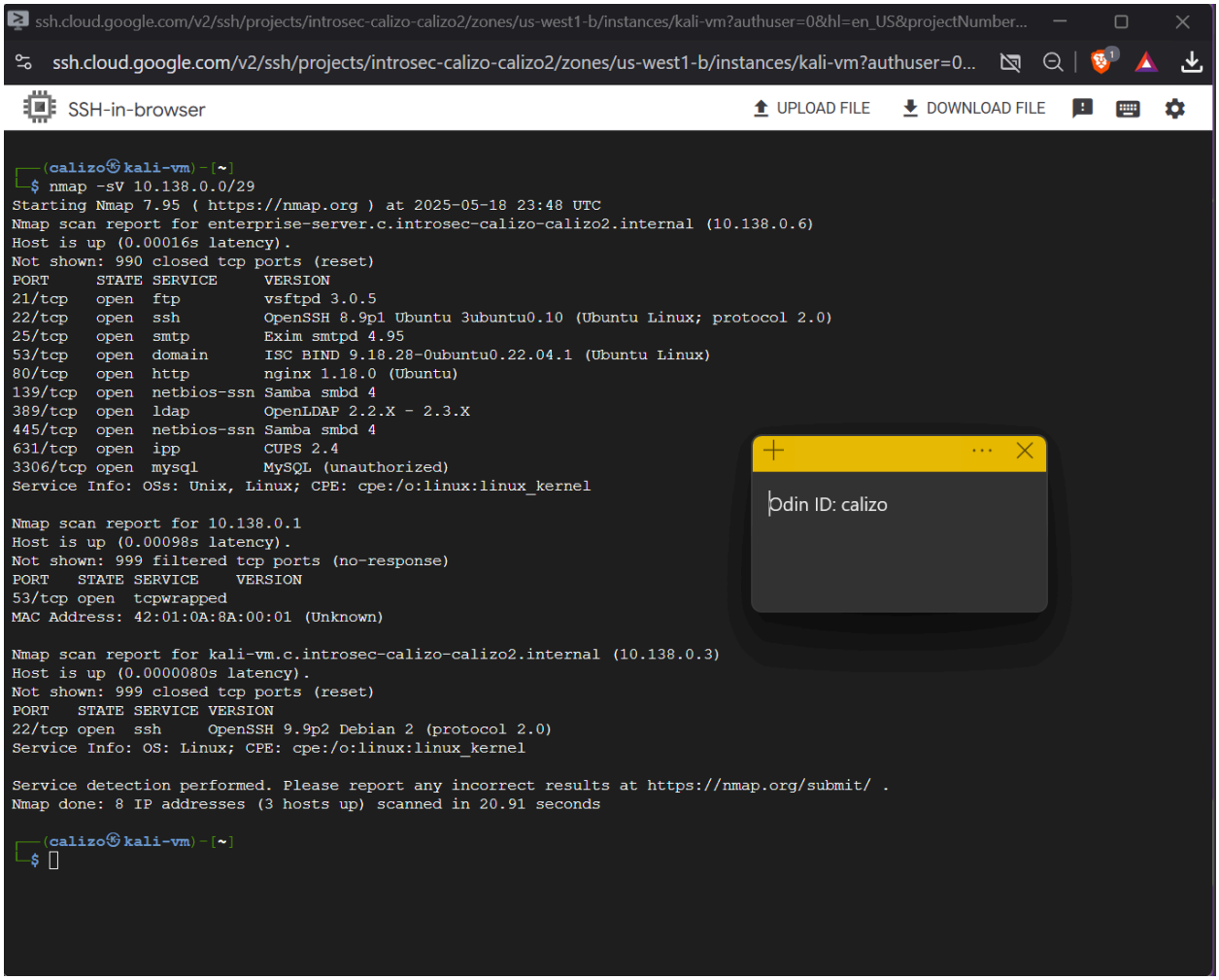
Answers to Numbered Questions

1. Gateway Host (10.138.0.1)
   - Service Identified: tcpwrapped on port 53 (DNS).
   - What it means: tcpwrapped indicates that the service is protected by TCP wrapper or some generic filtering mechanism that allows connections but immediately closes them. This is commonly used to restrict unauthorized access.
   - Behavior for other ports: All other 999 ports were marked as filtered, indicating that the firewall or host is blocking scans to those ports with no response.

2. Kali VM (10.138.0.3)
   - Service Detected: SSH (OpenSSH 9.9p2 Debian 2)
   - Release Date: OpenSSH 9.9p2 was released in March 2024.
   - Is it Latest?: As of May 2025, it is not the latest—OpenSSH 9.10 has been released since then.
   - Other Port Behavior: 999 other ports are closed, meaning the host explicitly sent RST packets when probed—confirming no other services are actively listening.

3. Enterprise Server (10.138.0.6)

- Screenshot of Services:



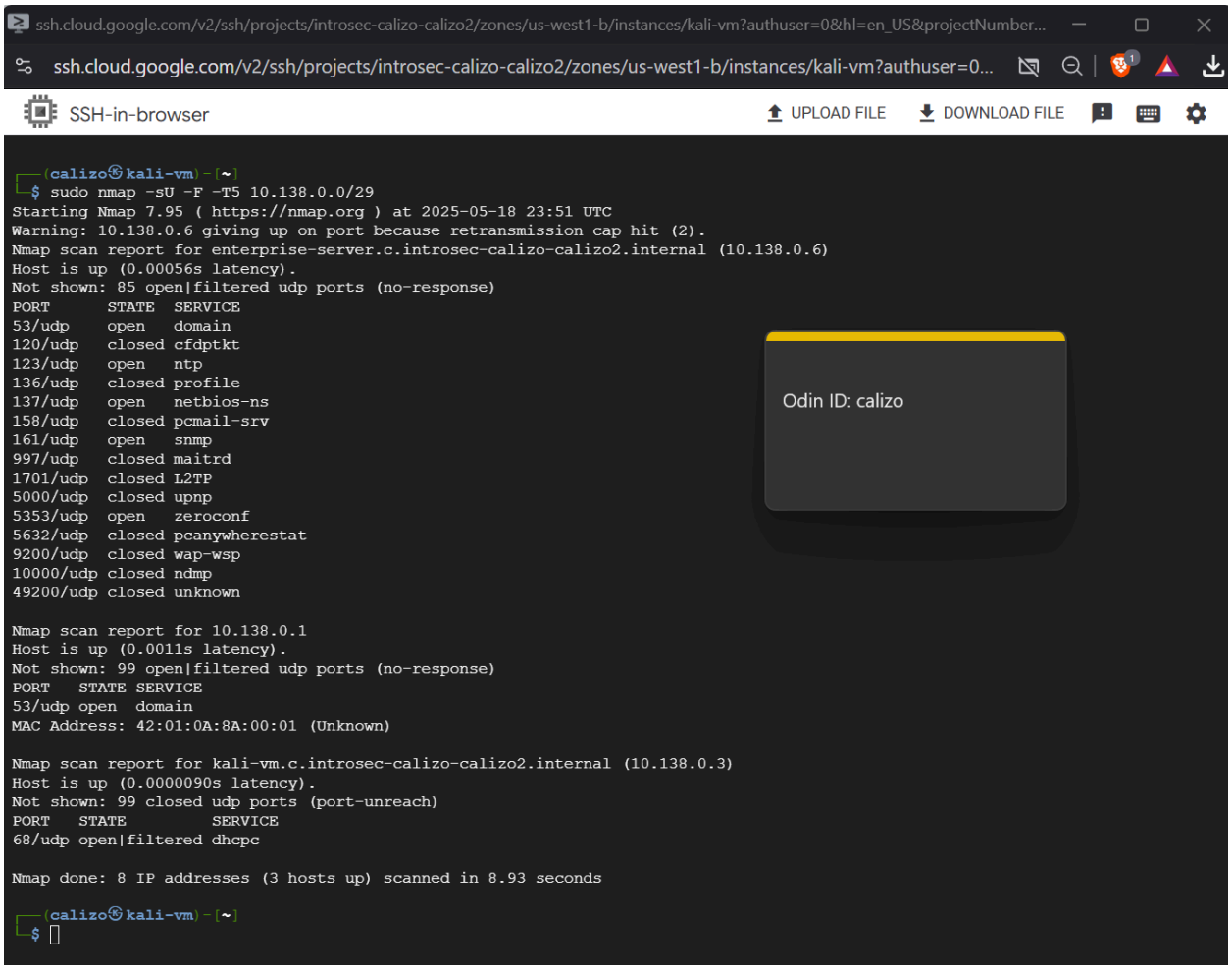
- Services Running:
  - FTP (vsftpd 3.0.5)
  - SSH (OpenSSH 8.9p1)
  - SMTP (Exim 4.95)
  - DNS (ISC BIND 9.18.2)
  - HTTP (nginx 1.18.0)
  - SMB/NetBIOS (Samba 4.0)
  - LDAP (OpenLDAP 2.2.x–2.3.x)
  - MySQL (unauthenticated)
  - Etc
- Other Port Behavior: The remaining ports were marked as closed—meaning the server responded with TCP RSTs to those probe attempts.

UDP Port Scan (Command Used):

- sudo nmap -sU -F -T5 10.138.0.0/29

This command limited the scan to common UDP ports and increased scan speed using -F and aggressive timing -T5.

4. Screenshot of UDP Scan:



- Open UDP Port(s):
  - Port 53 (domain) was the only port found open on both TCP and UDP protocols for the enterprise server (10.138.0.6).
  - All others were closed or open|filtered.

This step made it clear how service fingerprinting can reveal a full stack of network services along with their software versions—critical info for both security posture assessment and vulnerability identification.

Step 6: iptables Host-Based Filtering

For this task, I tested host-based firewall filtering directly on the enterprise-server VM using iptables. The goal was to restrict inbound traffic from the Kali VM and observe how that affects visibility using Nmap.

1. Dropping Inbound Traffic from Kali VM

To simulate a basic segmentation rule, I applied a DROP rule on the enterprise-server:

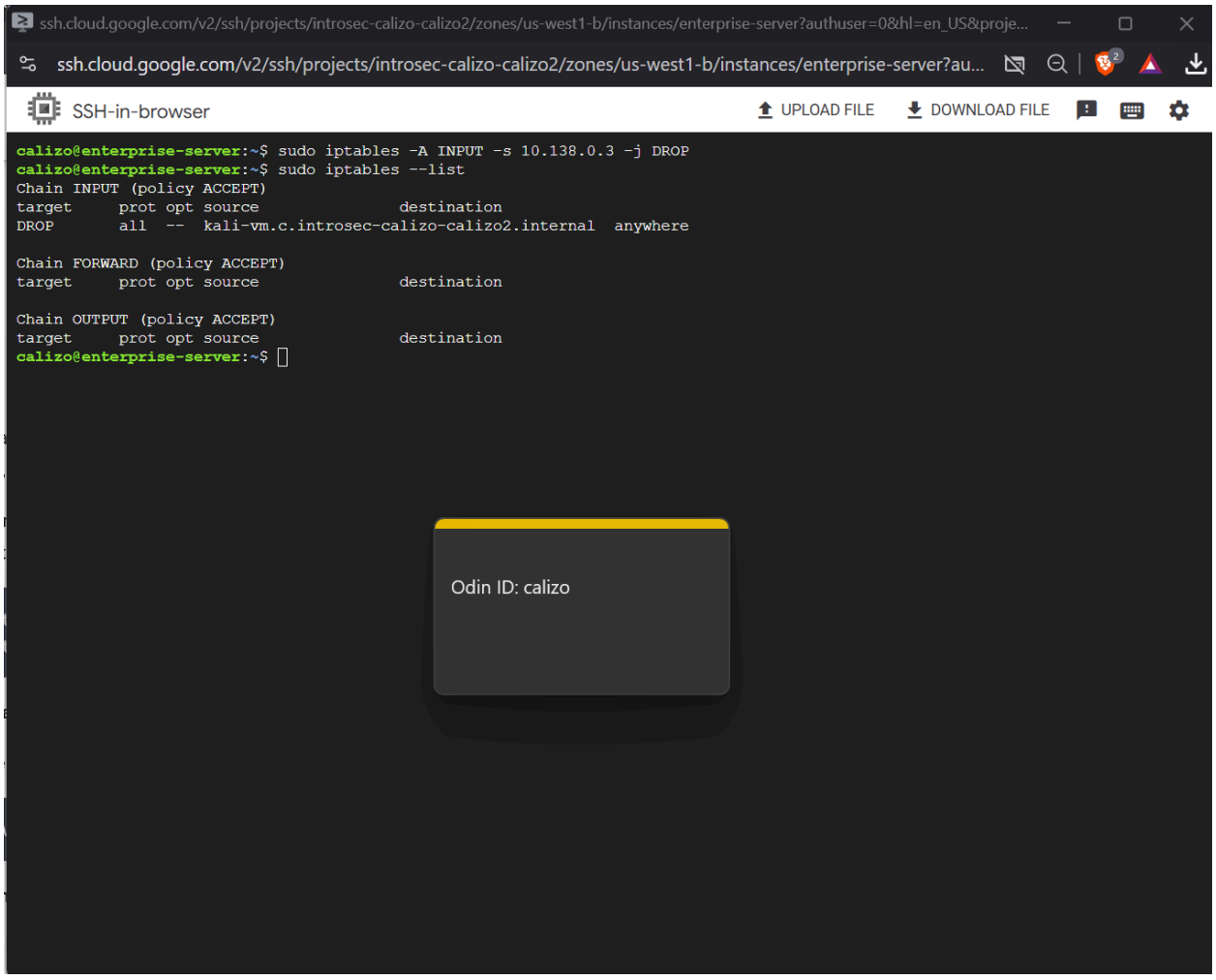
- sudo iptables -A INPUT -s 10.138.0.3 -j DROP

Listing the rules confirmed that traffic from the Kali host was being explicitly denied at the INPUT chain:

- sudo iptables --list

The rule was correctly displayed:

- DROP all -- kali-vm.c.introsec-calizo-calizo2.internal anywhere



This setup ensures the enterprise-server won't respond to any packets (including Nmap probes) coming from that specific IP.

2. Verifying Blocked Access from Kali with Nmap

From the Kali VM, I ran the same version-detection scan against the enterprise-server:

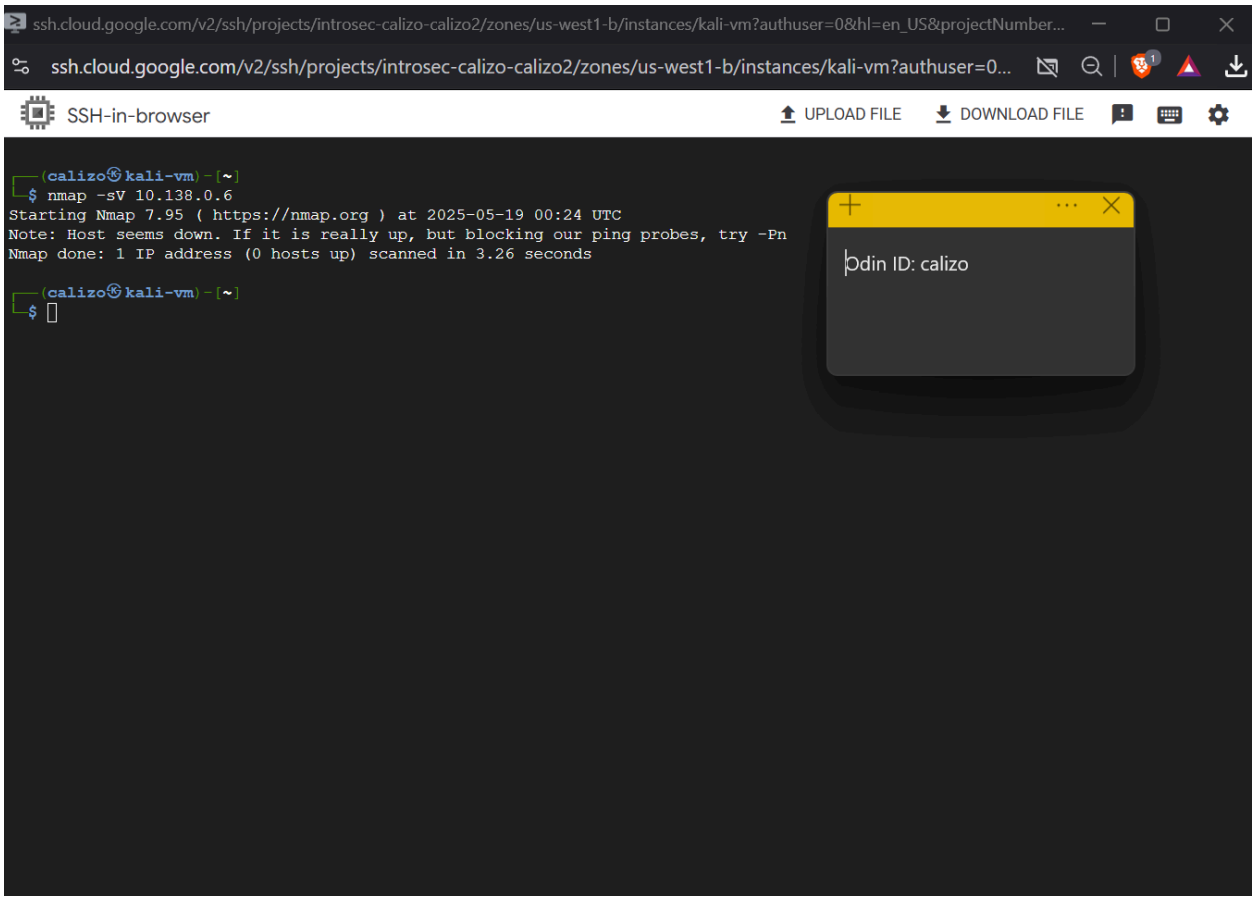
- nmap -sV 10.138.0.6

As expected, the scan output reported:

- Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
- Nmap done: 1 IP address (0 hosts up)

Answer (How many hosts are returned by Nmap?):

- 0



This proves the iptables rule was effective in completely suppressing Nmap’s ability to detect the host, validating host-based access control at the firewall level.

3. Removing the Rule

To restore visibility for future scans and cleanup the test state, I removed the rule using:

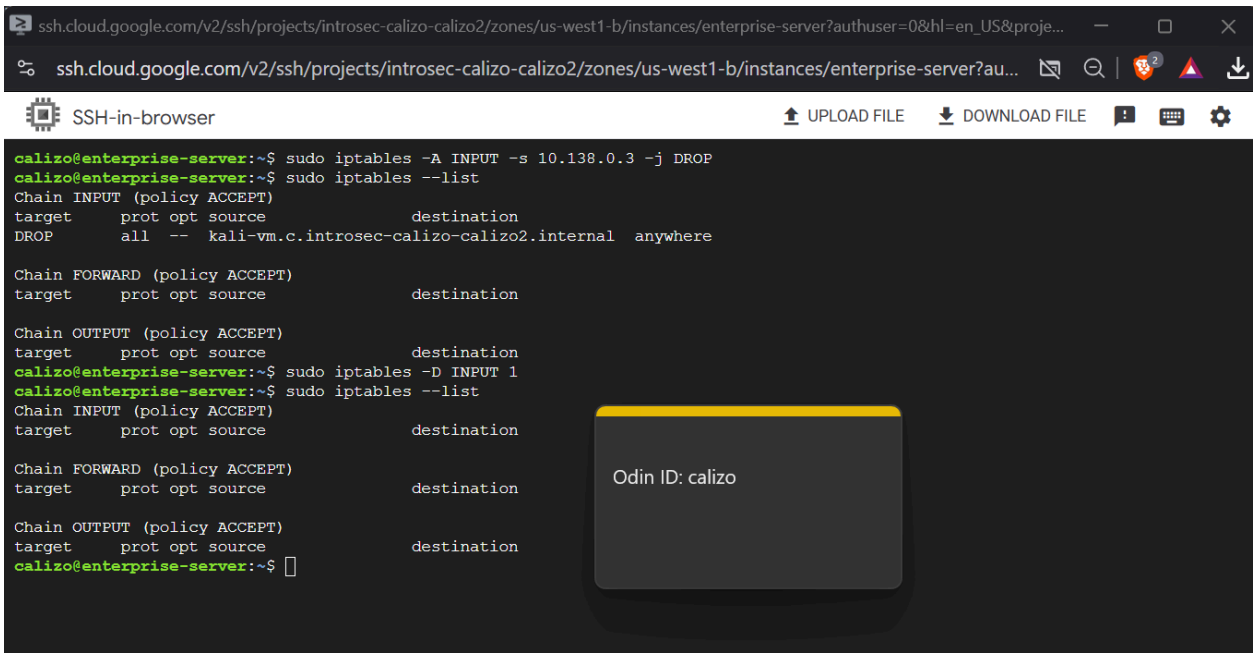
- sudo iptables -D INPUT 1

And verified that the INPUT chain was now empty:

- sudo iptables --list

Result:

- Chain INPUT (policy ACCEPT)



This step reinforced how powerful Linux's native firewall capabilities can be when it comes to network segmentation. Even with full network access, we can hide a service from reconnaissance by simply dropping packets at the host level — a good baseline control in defense-in-depth architectures.

Step 7: iptables Port-Based Filtering

In this step, I aimed to reduce the enterprise-server's attack surface by explicitly filtering all previously identified open TCP ports—except for SSH (port 22)—using iptables's multiport module. The goal here was to demonstrate how service-specific ingress control can deny discovery and access without affecting administrative SSH sessions.

1. Identify Open TCP Ports (Recheck via Nmap)

Using nmap -sV 10.138.0.6, I reconfirmed that the following TCP ports were open on the enterprise-server prior to filtering:

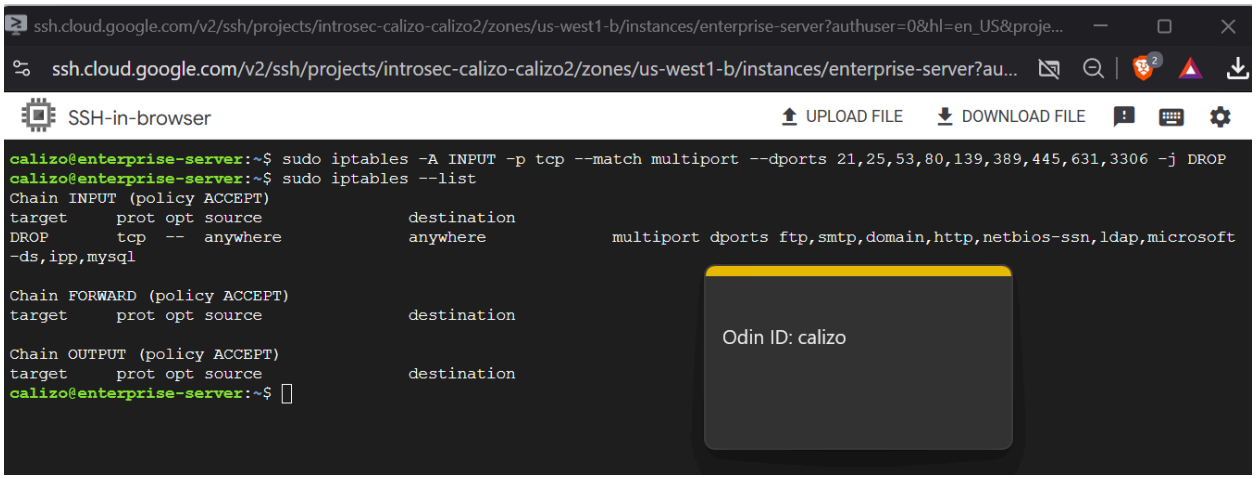
- 21 (ftp), 22 (ssh), 25 (smtp), 53 (domain), 80 (http), 139 (netbios-ssn), 389 (ldap), 445 (netbios-ssn), 631 (ipp), 3306 (mysql)

SSH (22) was deliberately excluded from the block list to preserve remote administrative access.

2. Configure iptables to Filter TCP Ports

I executed the following command on the enterprise-server:

- sudo iptables -A INPUT -p tcp --match multiport --dports 21,25,53,80,139,389,445,631,3306 -j DROP

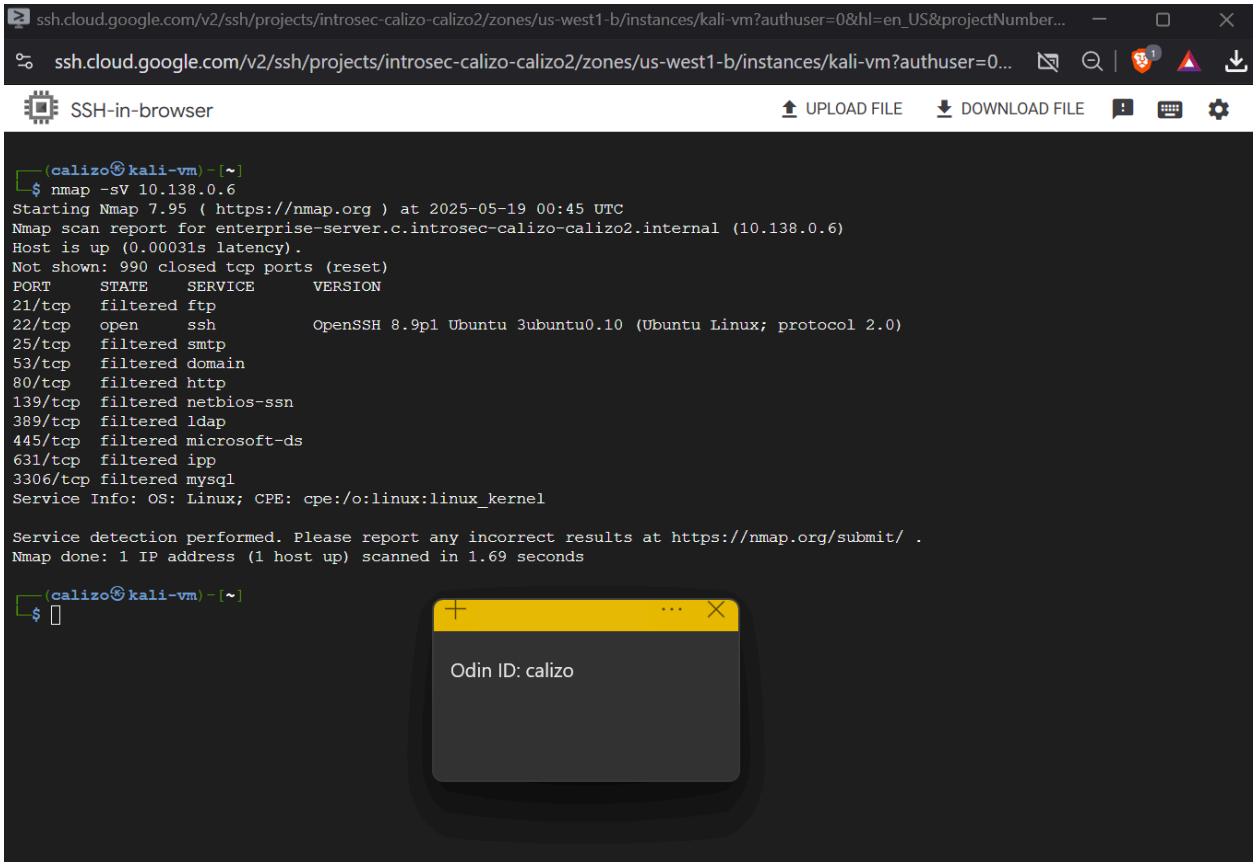


This command adds a rule to the INPUT chain to silently drop all incoming packets targeting the listed ports over TCP. Verifying with sudo iptables --list confirmed the DROP rule was in place and targeting the correct services.

3. Validate with Nmap from the Kali VM

Running nmap -sV 10.138.0.6 again from the Kali VM showed that all previously open ports except SSH were now marked as filtered:

- All previously open ports (except 22) now display: STATE: filtered

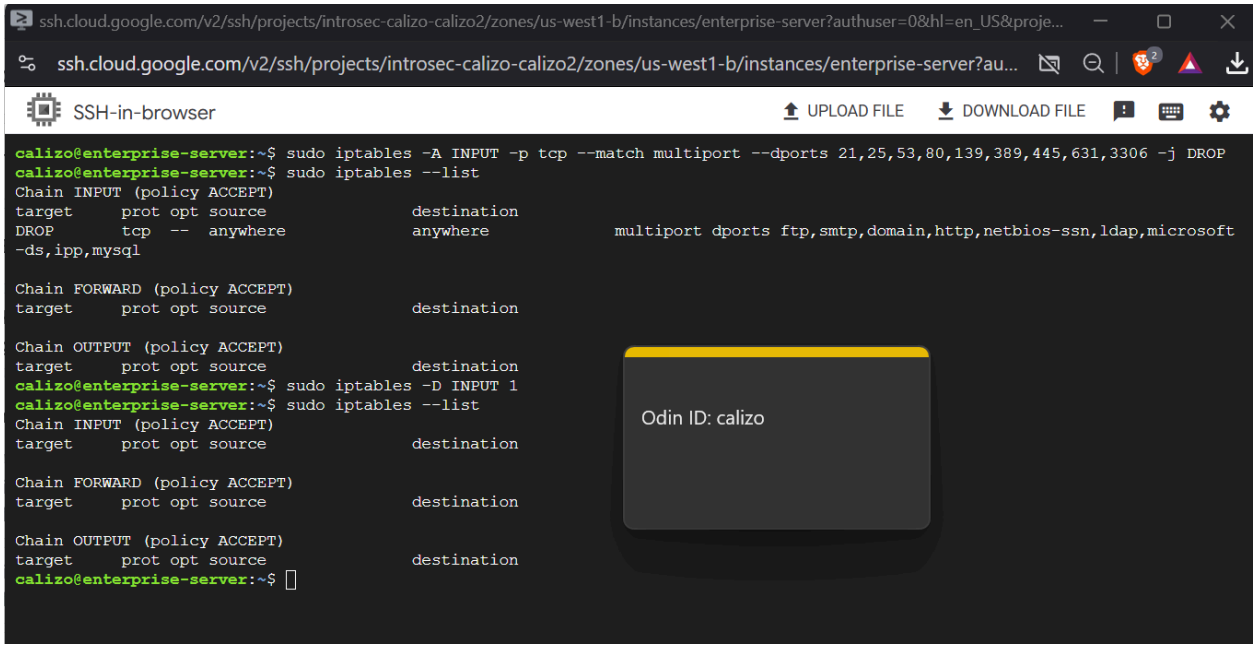


This change indicates that the ports are actively being blocked by a firewall rule. Unlike closed ports (which return a RST packet to reject connections), filtered ports drop the packets silently—providing no response and thereby increasing ambiguity for an attacker.

4. Cleanup

I removed the iptables rule to restore the default configuration:

- sudo iptables -D INPUT 1



Verifying again with sudo iptables --list showed the INPUT chain was empty, confirming successful rule cleanup.

Step 8: iptables Outgoing Traffic Filters

In this step, I explored how host-based firewall rules using iptables can be leveraged to prevent outbound traffic from the enterprise server. This is a critical layer of defense in mitigating potential misuse such as reverse shells, server-side request forgery (SSRF), and lateral movement.

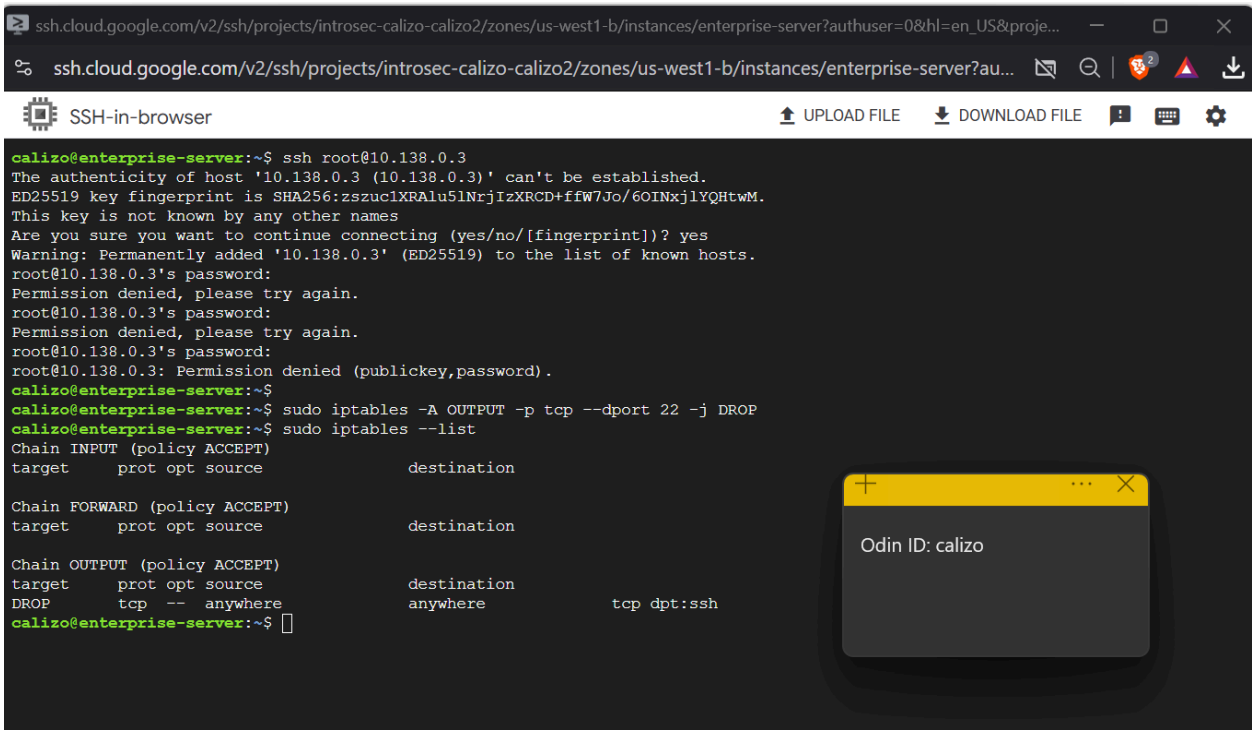
Before implementing any filters, I tested connectivity by attempting an outbound SSH session from the enterprise server to the Kali VM (IP: 10.138.0.3). As expected, the server initiated the SSH handshake and prompted for credentials, indicating that outbound port 22 was open.

To simulate a locked-down environment, I then executed the following iptables command on the enterprise server:

- sudo iptables -A OUTPUT -p tcp --dport 22 -j DROP

This added a rule to the OUTPUT chain, specifically blocking all outgoing TCP traffic targeting port 22 (SSH). I confirmed the rule with:

- sudo iptables --list



As shown in the screenshot, the DROP rule correctly appeared under the OUTPUT chain.

1. Rule Verification

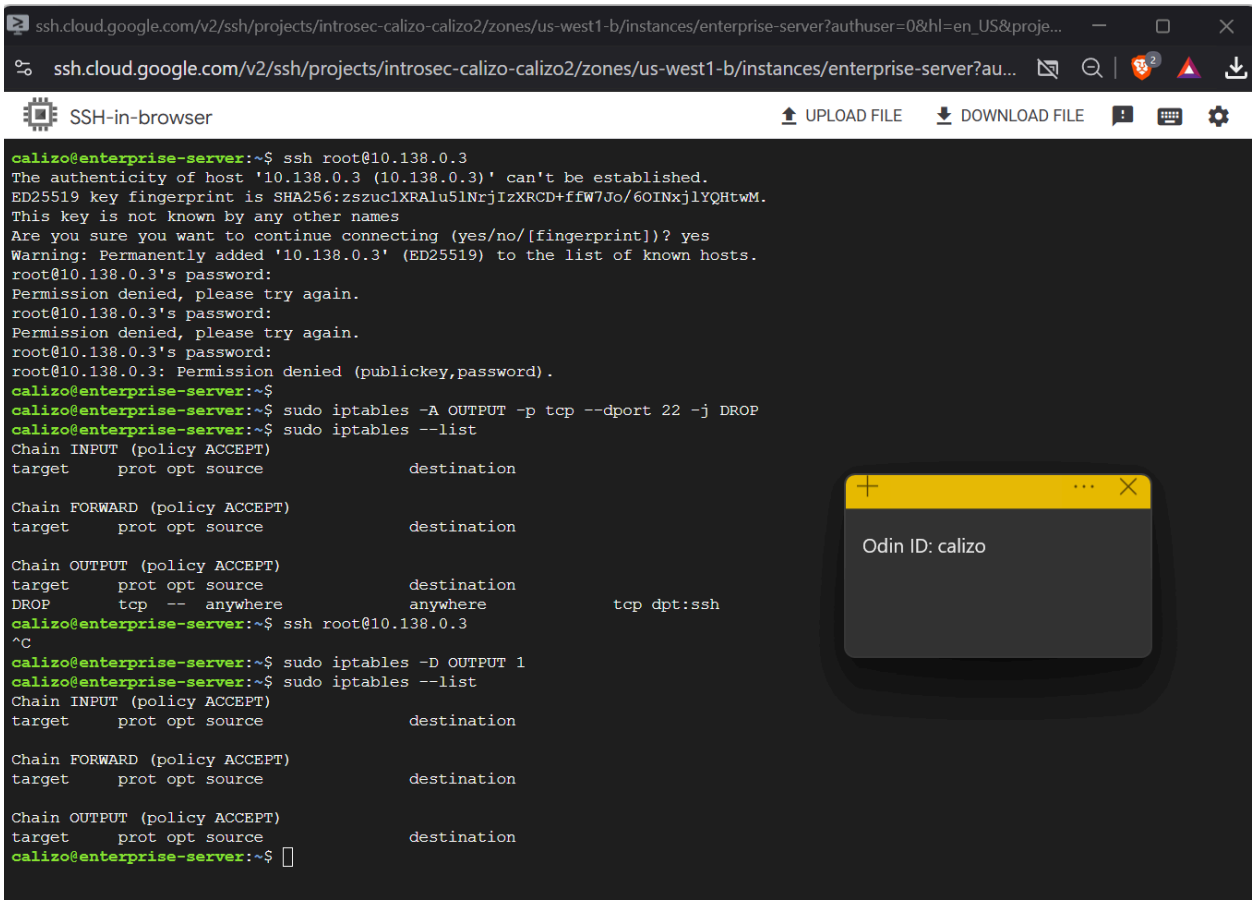
After implementing the rule, I tried the outbound SSH command again:

- ssh root@10.138.0.3

This time, the session hung—demonstrating that the rule successfully filtered the outbound attempt before the TCP handshake could fully complete. The drop policy was enforced before authentication even began, unlike in the earlier attempt.

Once confirmed, I cleaned up by deleting the rule and resetting the OUTPUT chain:

- sudo iptables -D OUTPUT 1
- sudo iptables --list



The final iptables output verified that the chain was empty, returning the server to its default behavior.

This exercise reinforced how applying egress filtering can drastically reduce the blast radius of a compromised host. Outbound SSH traffic, while often overlooked, should be locked down unless explicitly required for operations.

Step 9-10: Reducing the Attack Surface by Removing TCP Services

In this phase of the lab, I focused on minimizing the enterprise-server's exposure by identifying and disabling unused or unnecessary TCP services. The key idea here is that any open port represents a potential entry point for adversaries, so shutting down unused services drastically shrinks the attack surface.

Step 9: Disabling FTP (Port 21)

On the enterprise-server, I started with FTP. Using:

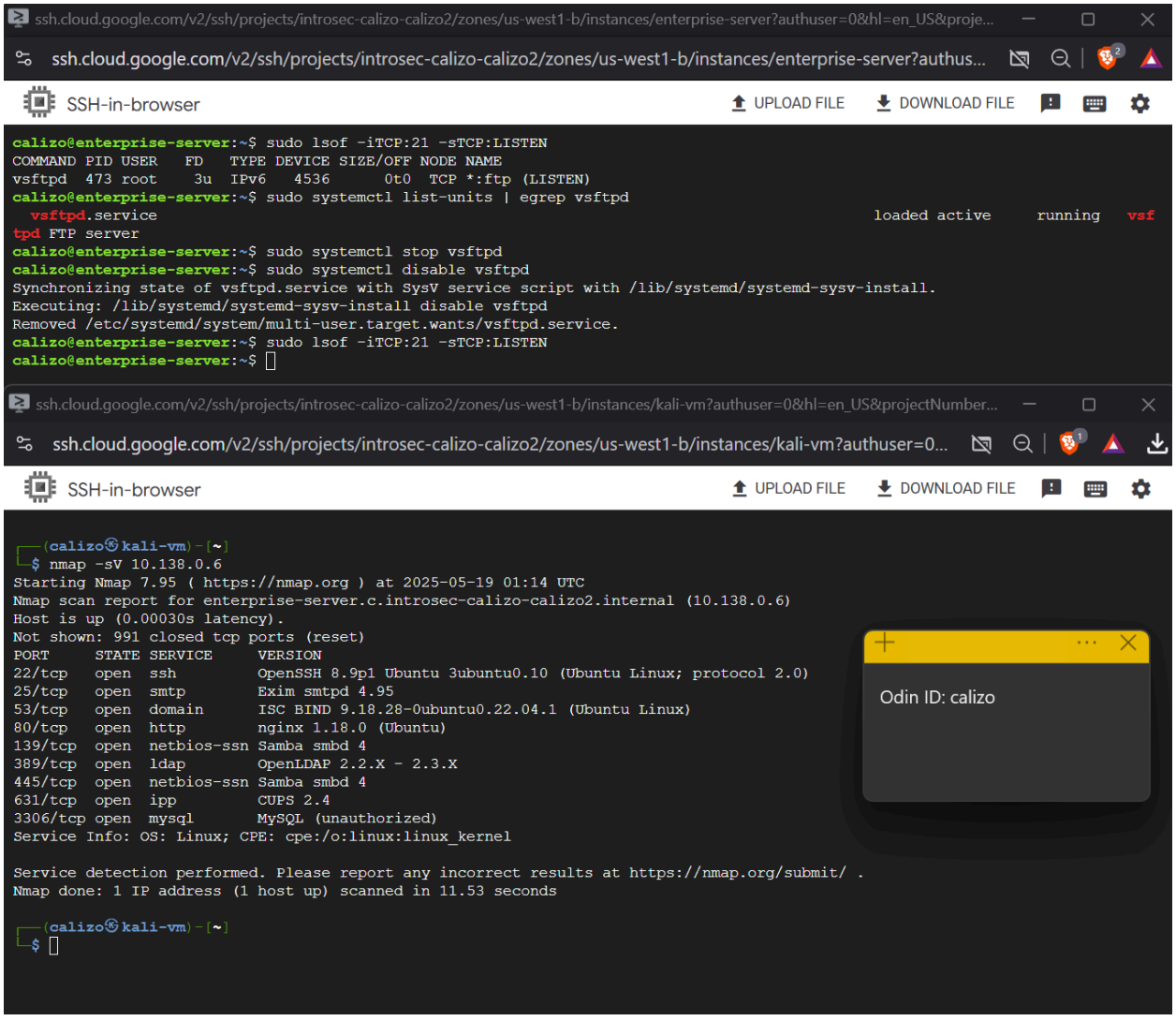
- sudo lsof -iTCP:21 -sTCP:LISTEN

I confirmed that vsftpd was listening on port 21. I then verified the systemd unit using:

- sudo systemctl list-units | egrep vsftpd

After verifying it was managed by systemd, I stopped and disabled the service:

- sudo systemctl stop vsftpd
- sudo systemctl disable vsftpd



Finally, running lsof again showed no process bound to port 21, and an nmap -sV scan from the Kali VM confirmed port 21 was now closed.

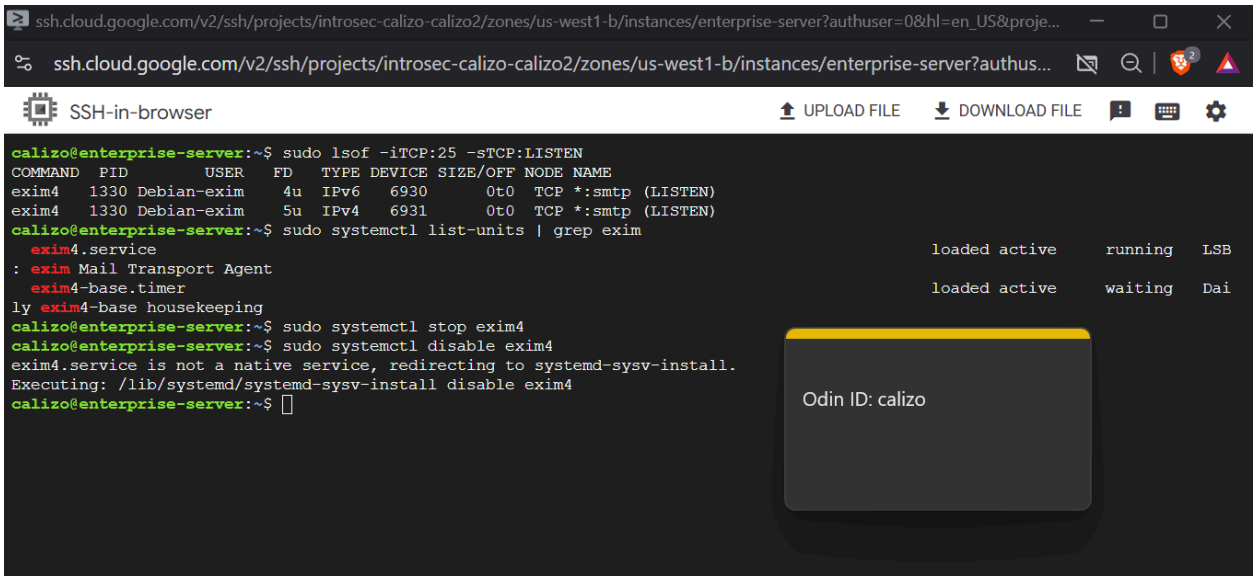
Step 10: Disabling Other TCP Services

1. SMTP (Port 25) – exim4
I used lsof and confirmed exim4 was running the mail transport agent:

- sudo lsof -iTCP:25 -sTCP:LISTEN

The systemd unit for the mail service was exim4. I executed:

- sudo systemctl stop exim4
- sudo systemctl disable exim4



The output confirmed that the service was disabled using the SysV compatibility layer and showed redirection via /lib/systemd/systemd-sysv-install.

2. DNS (Port 53) – named (BIND9)
To identify the DNS daemon, I ran:

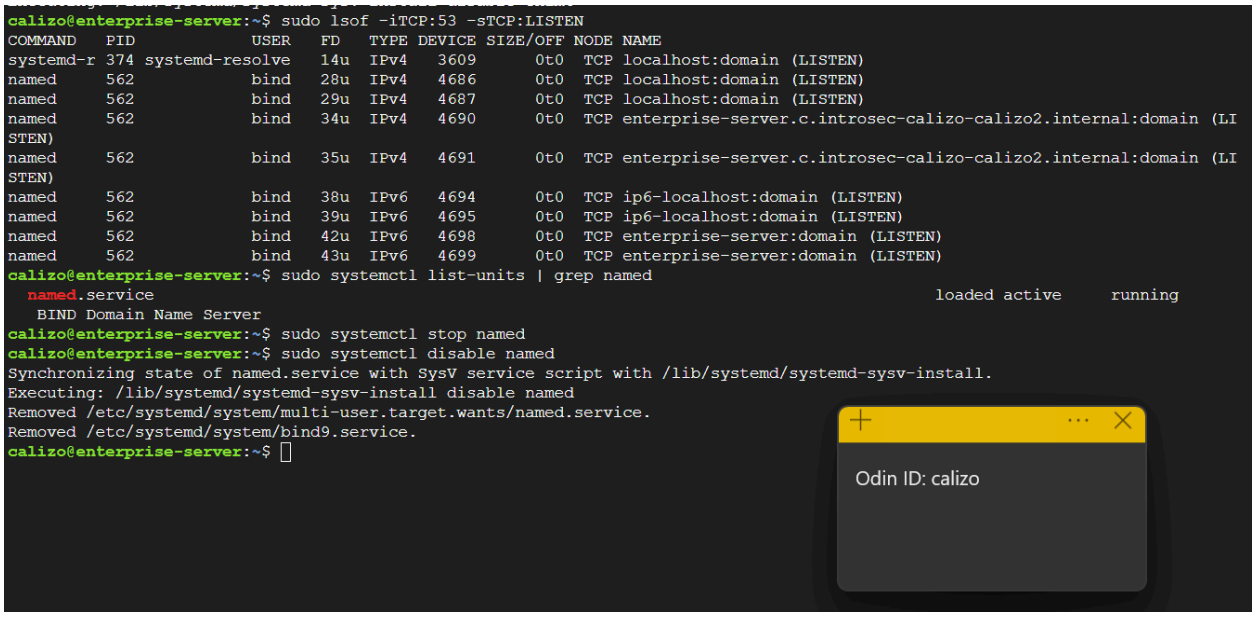
- sudo lsof -iTCP:53 -sTCP:LISTEN

It revealed the BIND named service. After verifying the systemd unit:

- sudo systemctl list-units | grep named

I executed:

- sudo systemctl stop named
- sudo systemctl disable named

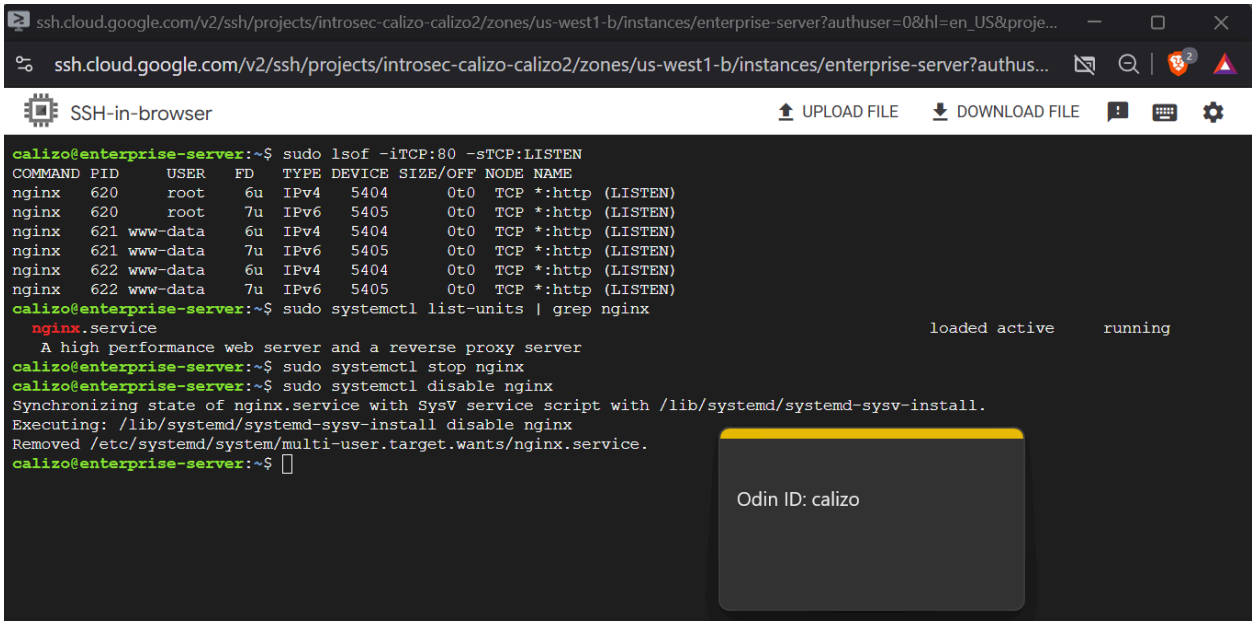


This removed the BIND9 DNS service from startup.

3. HTTP (Port 80) – nginx

For the web server:

- sudo lsof -iTCP:80 -sTCP:LISTEN
- sudo systemctl list-units | grep nginx
- sudo systemctl stop nginx
- sudo systemctl disable nginx



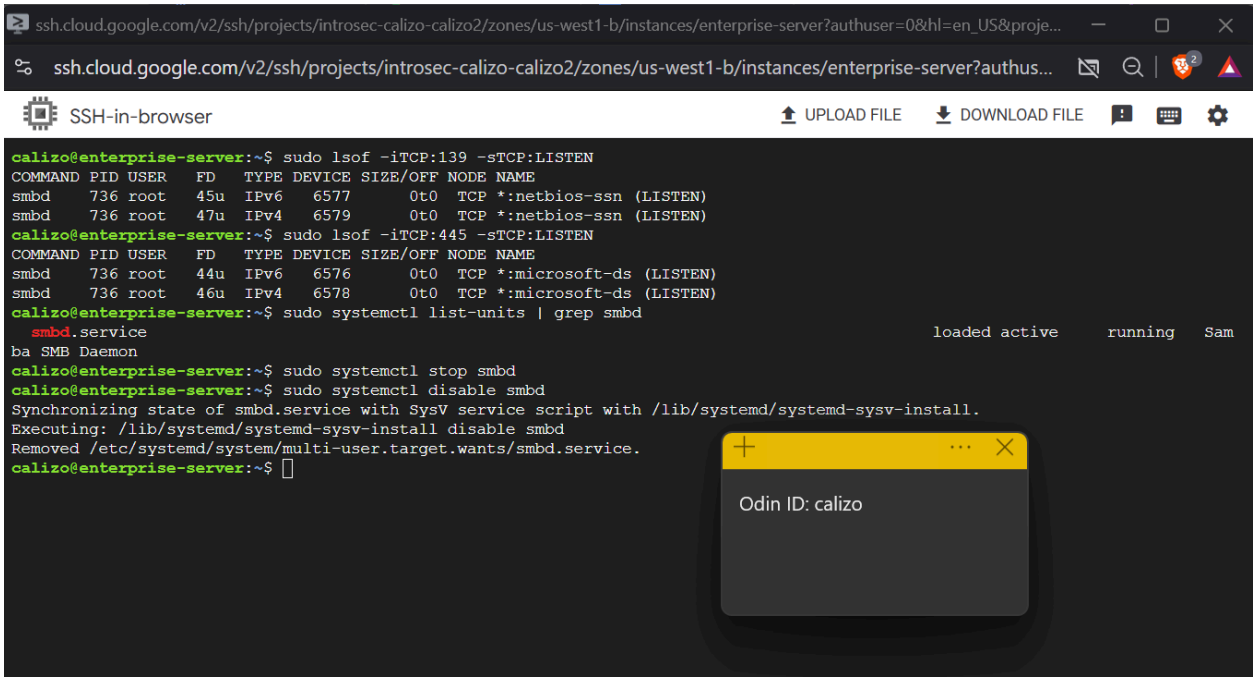
Disabling nginx removed its auto-start unit from /etc/systemd/system/multi-user.target.wants.

4. SMB (Ports 139 and 445) – smbd
To tackle SMB (a known vector in past malware like NotPetya), I checked both ports:

- sudo lsof -iTCP:139 -sTCP:LISTEN
- sudo lsof -iTCP:445 -sTCP:LISTEN

Confirmed the smbd daemon owned them. Then, I shut it down and disabled it:

- sudo systemctl stop smbd
- sudo systemctl disable smbd



The service was successfully removed from system startup targets.

By the end of this step, the enterprise-server had significantly fewer listening services, resulting in a hardened network footprint.

Step 11: Disabling Remaining TCP Services (LDAP, IPP, MySQL)

As part of continuing to harden the enterprise-server's security posture, I investigated and disabled the remaining services still actively listening on TCP ports 389 (LDAP), 631 (IPP), and 3306 (MySQL). This step focused on reducing the attack surface by identifying systemd-managed services bound to those ports and permanently stopping them from running on future boots.

1. LDAP (Port 389)

To confirm LDAP was listening, I ran:

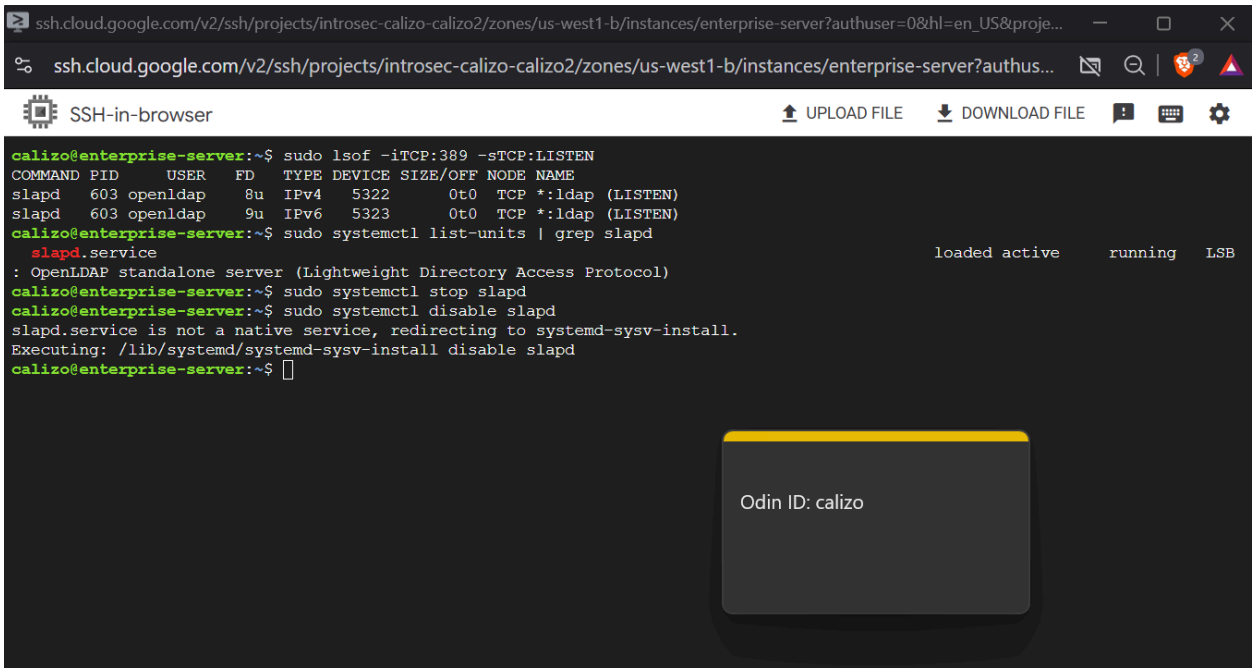
- sudo lsof -iTCP:389 -sTCP:LISTEN

This showed slapd running under the openldap user. I then ran:

- sudo systemctl list-units | grep slapd

It confirmed slapd.service was active. I disabled it using:

- sudo systemctl stop slapd
- sudo systemctl disable slapd



Result: slapd.service was redirected to systemd-sysv-install and successfully removed from system boot targets.

2. IPP (Port 631)
Using:

- sudo lsof -iTCP:631 -sTCP:LISTEN

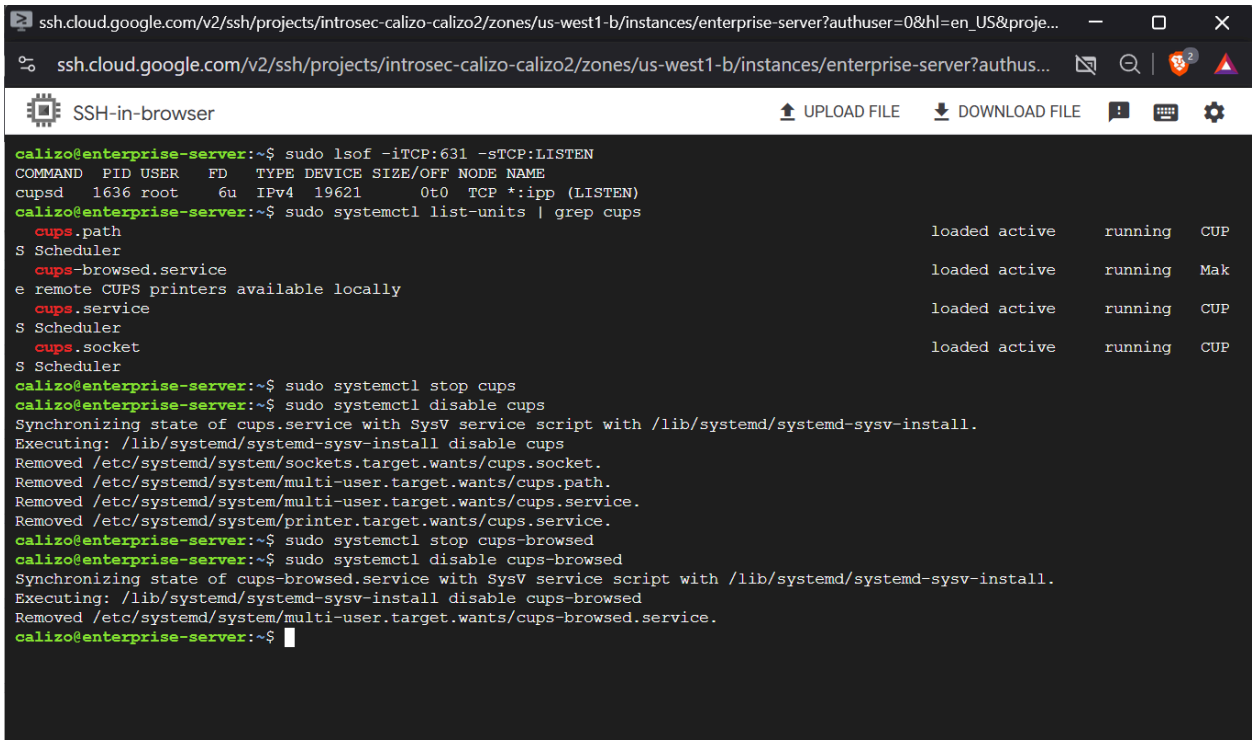
I confirmed that the IPP service was bound by the cupsd process. I then listed associated units:

- sudo systemctl list-units | grep cups

This showed multiple units, including cups.service and cups-browsed.service, all active.

To fully stop printing capabilities, I disabled both:

- sudo systemctl stop cups
- sudo systemctl disable cups
- systemctl stop cups-browsed
- sudo systemctl disable cups-browsed



Result: All CUPS-related services, sockets, and paths were disabled and removed from startup.

3. MySQL (Port 3306)

To identify the service behind MySQL:

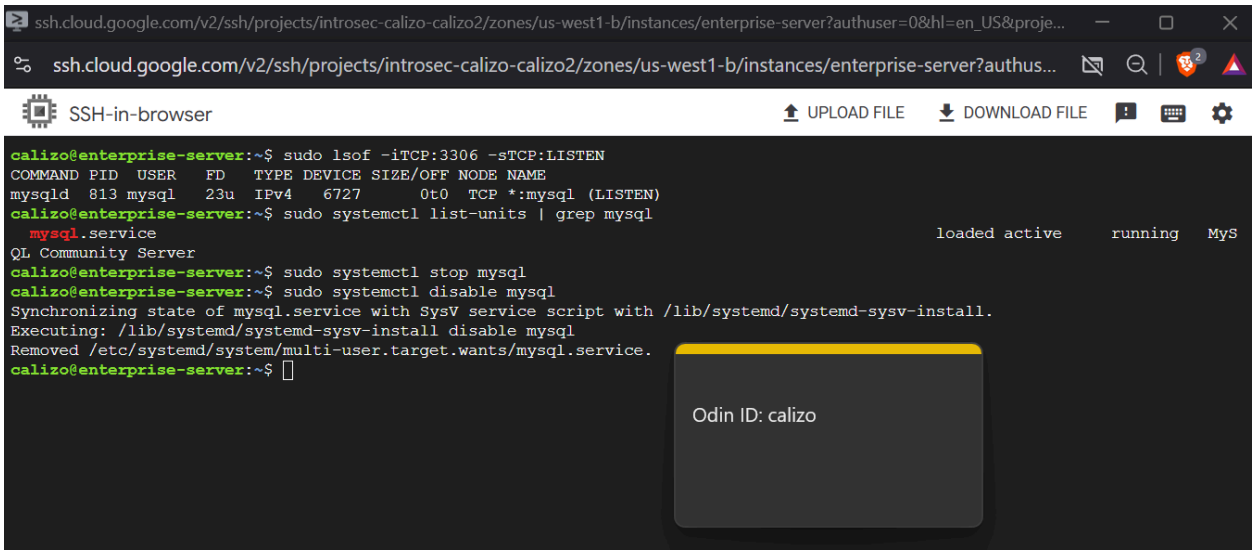
- sudo lsof -iTCP:3306 -sTCP:LISTEN

Revealed mysqld running under the mysql user. Listing units confirmed:

- sudo systemctl list-units | grep mysql

Then I shut it down and disabled it:

- sudo systemctl stop mysql
- sudo systemctl disable mysql



Result: mysql.service was successfully stopped and disabled.

This wraps up all lingering TCP services found in earlier scans. These changes significantly reduce the exposed footprint of the enterprise-server, eliminating unnecessary services that could be exploited if left running.

Step 12: Removing Services (UDP)

After previously removing unnecessary TCP services, I turned attention to active UDP services on the enterprise-server. To begin, I ran an updated UDP scan from the kali-vm to verify which ports remained open:

0. Recon: Identify open UDP ports (on kali-vm)

- sudo nmap -sU -F -T5 10.138.0.6

Open UDP ports discovered:

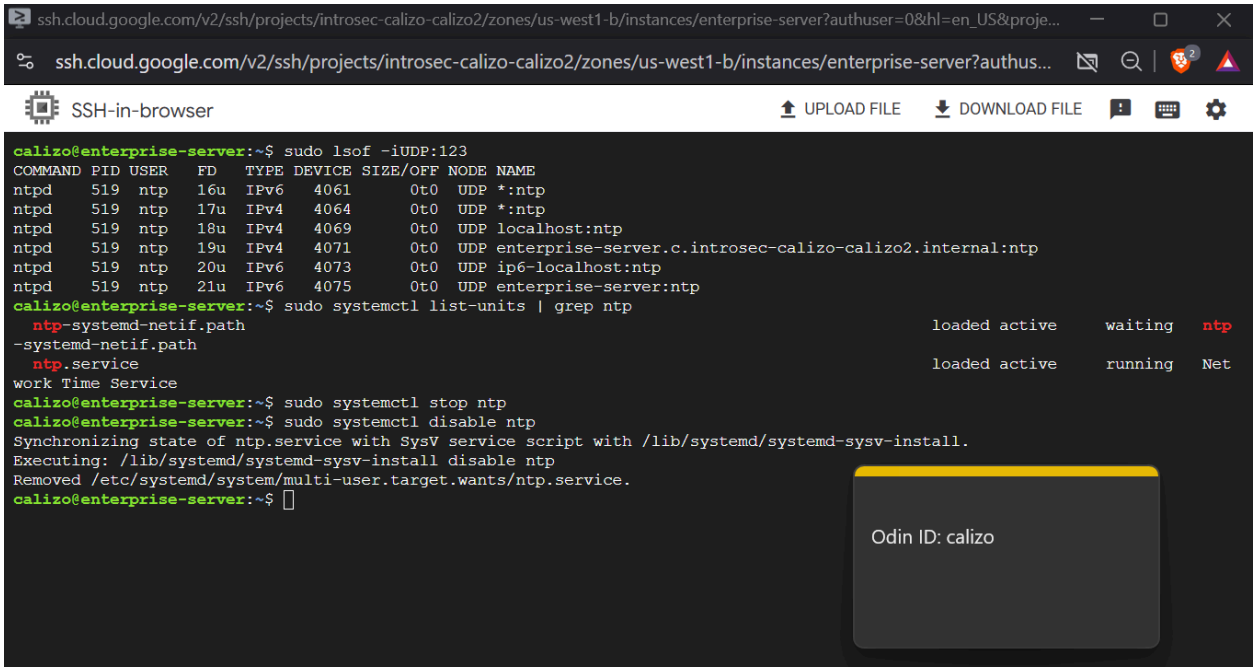
- 123/udp (ntp)
- 137/udp (netbios-ns)
- 161/udp (snmp)

These services were confirmed to still be listening, so I proceeded to disable them from the enterprise-server.

1. Port 123 (ntp): Disable the Network Time Protocol Service (on enterprise-server)

- sudo lsof -iUDP:123
- sudo systemctl list-units | grep ntp

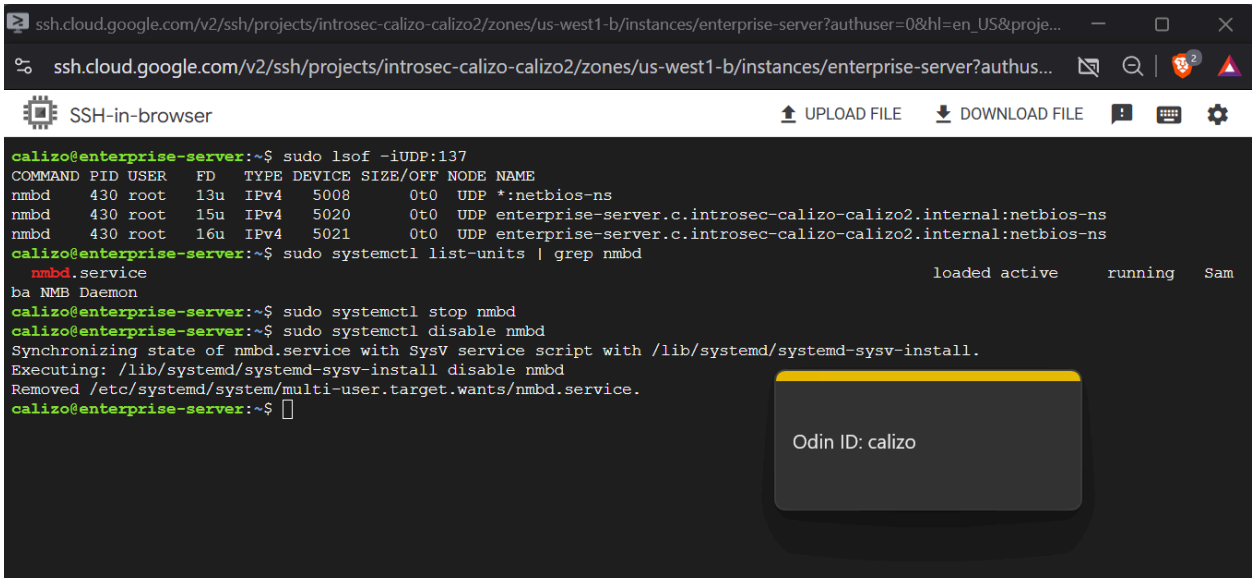
- sudo systemctl stop ntp
- sudo systemctl disable ntp



This removed the ntpd daemon responsible for clock synchronization. Disabling this prevents attackers from abusing it for amplification DDoS or tampering with time-sensitive logs and tokens.

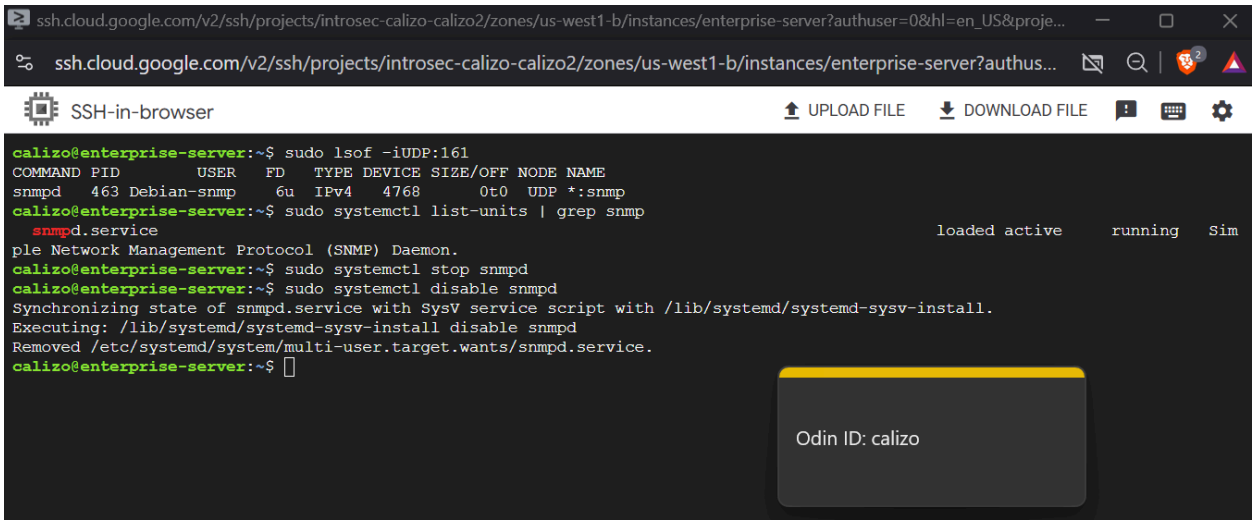
2. Port 137 (netbios-ns): Disable NetBIOS Name Service (on enterprise-server)

- sudo lsof -iUDP:137
- sudo systemctl list-units | grep nmbd
- sudo systemctl stop nmbd
- sudo systemctl disable nmbd



Even though SMB (ports 139 and 445) had been disabled earlier, the NetBIOS name service component (nmbd) was still active. Removing it ensures the Windows file-sharing stack is fully shut down, minimizing exposure to legacy protocols.

3. Port 161 (snmp): Disable SNMP Service (on enterprise-server)
   - sudo lsof -iUDP:161
   - sudo systemctl list-units | grep snmp
   - sudo systemctl stop snmpd
   - sudo systemctl disable snmpd



snmpd was found to be actively listening. Disabling this closes the door on a service often exploited in reconnaissance and enumeration, especially in misconfigured production systems.

All remaining open UDP services were tracked using lsof, validated with systemctl, then disabled using stop and disable. This significantly reduces the enterprise server's attack surface and hardens it against low-level network probes.

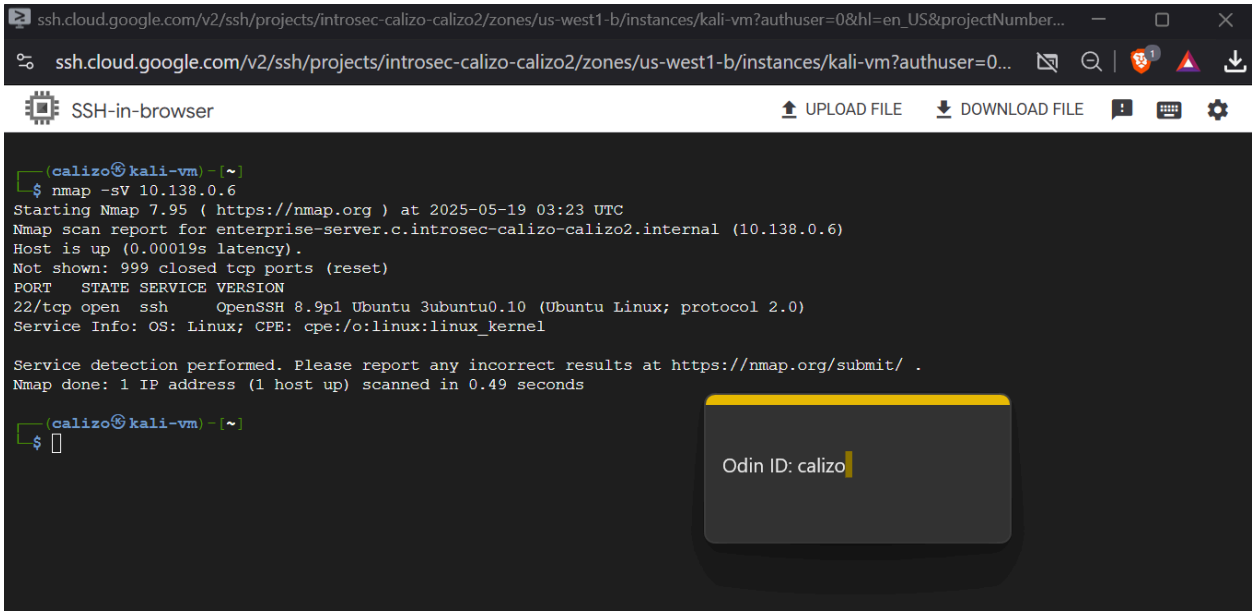
Step 13: Validation

To ensure that all unnecessary network services were successfully disabled on the enterprise-server, I performed a full system reset via the Google Cloud Compute Engine interface. This was a critical checkpoint to confirm that disabled services would not automatically restart after a reboot, which would compromise the reduced attack surface we've worked toward.

1. TCP Port Validation - nmap -sV

Once the enterprise-server VM was back online and I verified SSH access was still available, I SSH'ed into the kali-vm and ran the following:

- nmap -sV 10.138.0.6

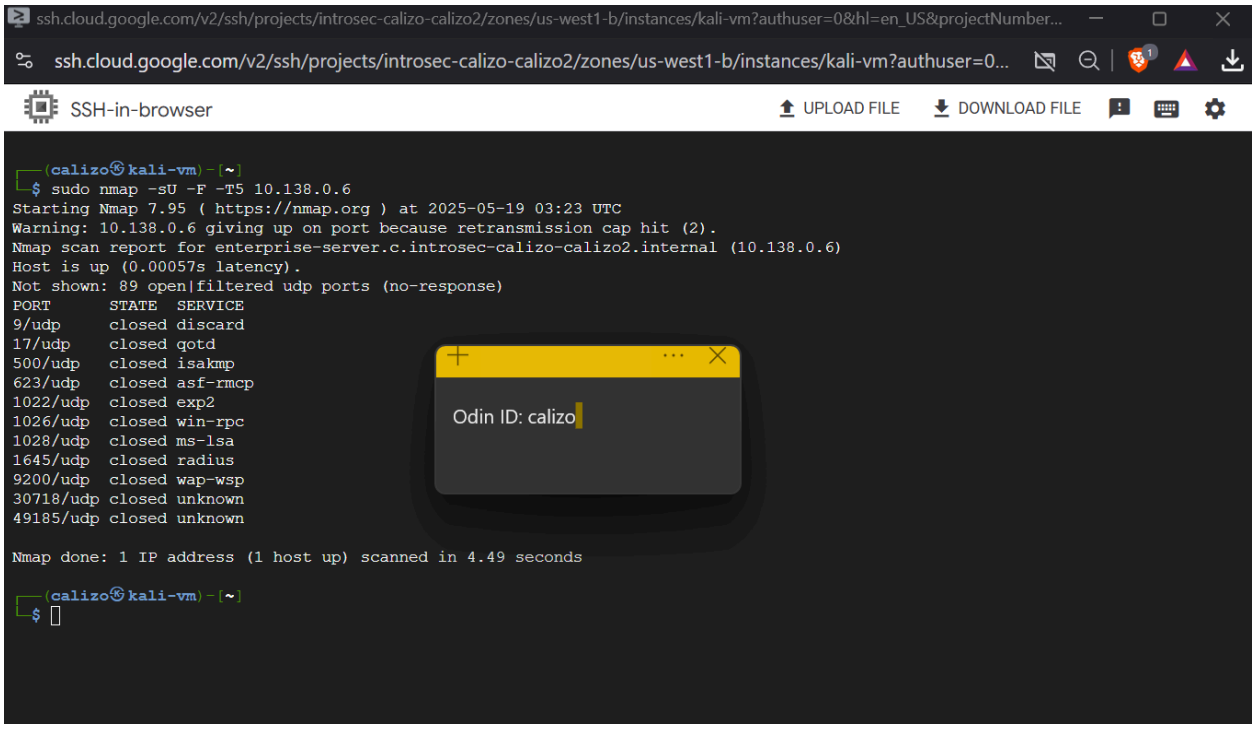


Result: As shown in the screenshot, only port 22/tcp (SSH) remained open. All other TCP services previously active—such as FTP, HTTP, LDAP, MySQL, etc.—were now closed or unreachable, confirming they were properly shut down and disabled via systemctl.

2. UDP Port Validation - sudo nmap -sU -F -T5

Next, I re-executed a fast UDP scan to confirm that all known open UDP ports—123/UDP (ntp), 137/UDP (netbios-ns), and 161/UDP (snmp)—were now closed:

sudo nmap -sU -F -T5 10.138.0.6



Result: The scan output showed that all previously open UDP ports were now in the closed state. No open or filtered UDP ports remained, validating that those services were fully disabled (e.g., ntpd, nmbd, snmpd).

Clean-up
Lastly, as part of responsible lab closure, I returned to the Compute Engine dashboard and issued a Stop command on both the enterprise-server and kali VMs to conserve cloud resource usage.

This validation phase reinforced best practices around service hardening, showing how TCP and UDP attack surfaces can be systematically minimized through direct control of system daemons. The SSH port was the only one left intentionally open—supporting secure remote management while leaving no unnecessary doors unlocked.

Step 14: Hiding Scanning

To simulate a real-world attacker's mindset, I performed a stealth-based scan using an external cloud tool to decouple my machine identity from the reconnaissance phase. I used Pentest-Tools.com, a platform that provides web-based penetration testing utilities, including website and network vulnerability scanners. This demonstrates how

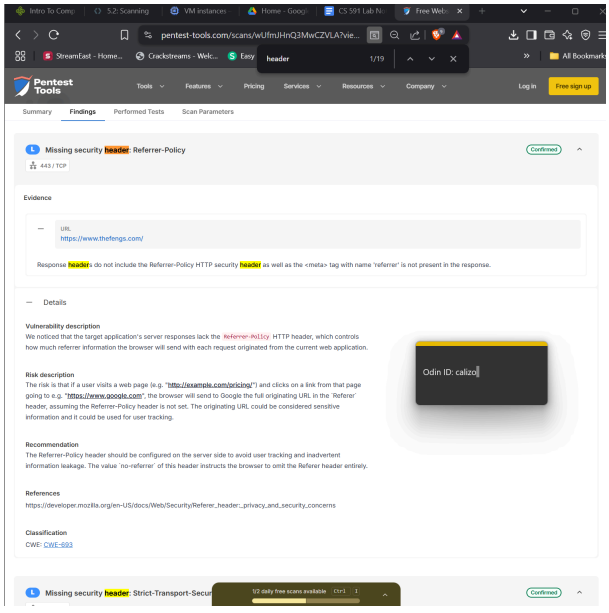
attackers with good OpSec avoid attribution by scanning through third-party infrastructure.

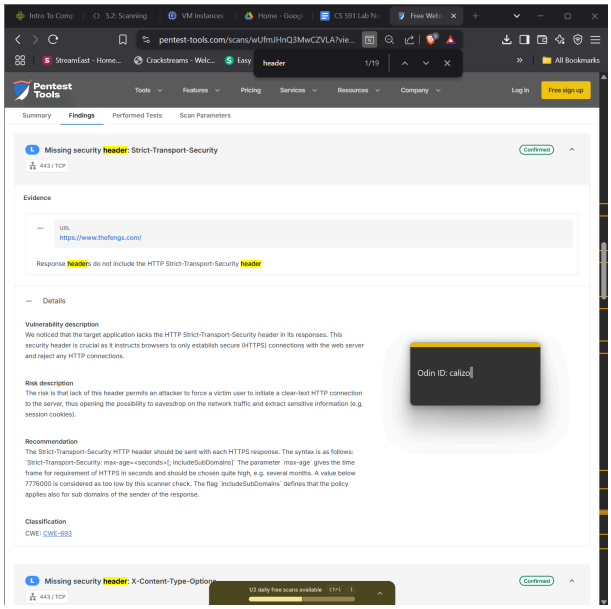
Website Scanner Results

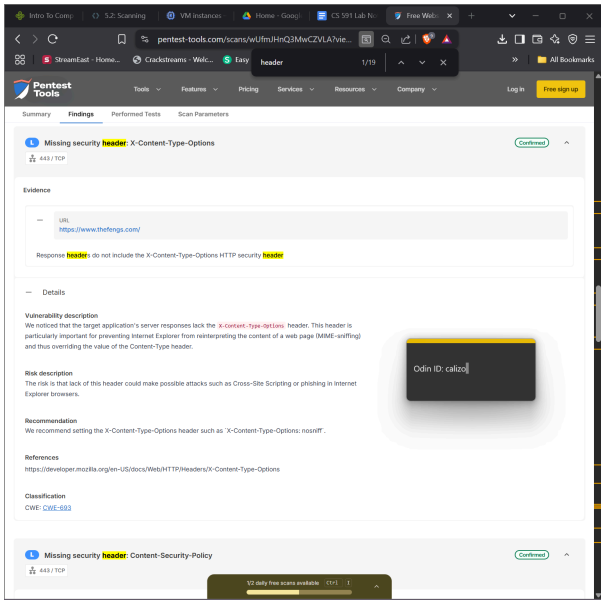
After launching the Website Scanner against https://www.thefengs.com, the scan revealed several missing HTTP security headers which are crucial for protecting against common web-based attacks:

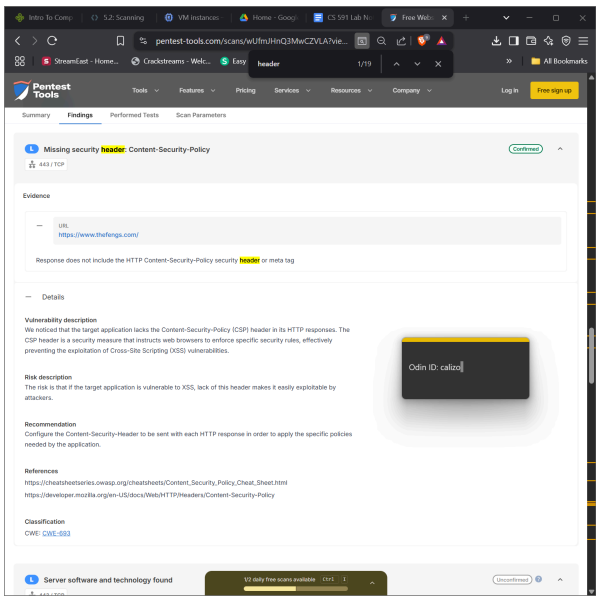
1. Missing Headers That Impact Security:

- Referrer-Policy: Without this, browsers may send the full originating URL in the Referer header, leading to potential data leakage or user tracking.
- Strict-Transport-Security (HSTS): This header enforces HTTPS usage and protects against SSL stripping attacks.
- X-Content-Type-Options: Missing this increases the risk of MIME sniffing attacks, especially in older browsers.
- Content-Security-Policy (CSP): Absence of a CSP makes the site significantly more vulnerable to XSS-based exploits.









These headers play a pivotal role in hardening a website's browser-side defenses. Their absence reveals a gap in secure web application deployment best practices.

Network Vulnerability Scanner Results

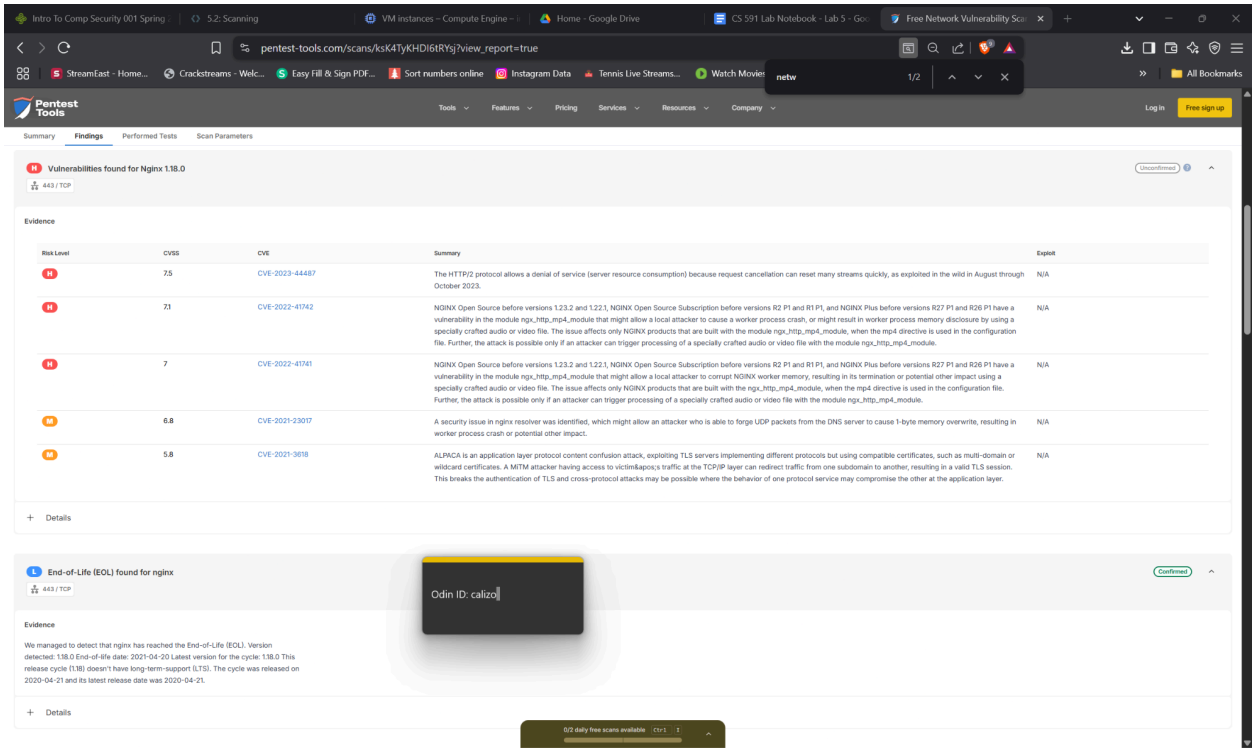
Next, I used the Network Vulnerability Scanner (light scan mode) against the same domain. This returned detailed results regarding open ports and vulnerable software versions.

2. Open Ports, Software, and Version Info:

- Port 22 (SSH): OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 – no vulnerabilities found.
- Port 80 (HTTP): nginx 1.18.0 – redirecting traffic to HTTPS (port 443).
- Port 443 (HTTPS): nginx 1.18.0 – actively serving the site with known vulnerabilities.

Detected Vulnerabilities (via version-based analysis on port 443):

CVE ID	CVSS	Summary
CVE-2023-44487	7.5	HTTP/2 DoS via stream resets
CVE-2022-41742	7.1	Memory disclosure and crash in ngx_http_mp4_module
CVE-2022-41741	7.0	Worker memory corruption vulnerability in ngx_http_mp4_module
CVE-2021-23017	6.8	DNS resolver memory overwrite vulnerability
CVE-2021-3618	5.8	ALPACA cross-protocol confusion attack via TLS certificate reuse



Other Security Concern:

- End-of-Life Warning: Nginx version 1.18.0 has reached its EOL as of April 2021, meaning it no longer receives updates, including security patches.

These findings underline the risks of running outdated software in production environments. Even with low-severity issues, an attacker could chain them together for impactful results.

This step reinforces the mindset and techniques used during reconnaissance while balancing ethical testing principles. From a defender's perspective, it's a reminder to actively monitor for external scans and enforce modern security headers and software lifecycle hygiene.

Lab 5.3 Virtual Private Networks

Step 1: VPNs Overview

This lab starts by introducing the concept of a Virtual Private Network (VPN) — specifically how it allows a user’s network traffic to be securely tunneled through a remote server before reaching the broader internet. The primary takeaway is that this setup masks the originating IP address from adversaries and remote sites, improving both privacy and operational security (OpSec). The VPN server handles all outbound

traffic, while the client establishes a secure encrypted tunnel to the server, shielding data and location details from eavesdroppers.

The diagram helped reinforce how the user’s traffic gets re-routed through the VPN server, making any adversary monitoring the local network blind to the final destination or content of the communication. This setup is especially common in red teaming or penetration testing operations to avoid attribution.

Step 2: Launch VPN Access Server Deployment

To kick off the VPN setup, I navigated to the Google Cloud Platform Marketplace and searched for “OpenVPN.” From there, I selected the OpenVPN Access Server image provided by OpenVPN Inc. and launched the deployment. This triggered the configuration UI where I specified key parameters for provisioning the virtual machine and enabling connectivity.

For the deployment zone, I selected us-east1-d as instructed, ensuring the VM is located within the specified region. I also created a new service account named openvpn-svc and kept the default roles (Config Agent, Compute Admin, IAM Service Account User), which are necessary to deploy and manage the instance.

On the firewall configuration section, I enabled IP forwarding since the VPN needs to tunnel packets to external destinations and act as a router for the client machine. Skipping this would break VPN routing entirely.

Answer – What UDP and TCP ports are allowed in order for you to communicate with the VPN access server?
During configuration, the UI allowed me to toggle which ports would be opened via firewall rules. The following ports were explicitly enabled for secure communication between client and VPN server:

- TCP 943 – used for the OpenVPN web UI (admin and client access)
- TCP 945 – additional port used for web UI administration
- TCP 22 – SSH access to the VM
- TCP 443 – HTTPS traffic for secure web UI access
- UDP 1194 – core OpenVPN tunnel port used by OpenVPN clients for actual VPN traffic

These ports ensure both web interface management and encrypted VPN tunneling are accessible post-deployment.

Once all the parameters were set, I clicked Deploy, and after a brief initialization period, the OpenVPN instance was up and running, with public and admin URLs shown on the confirmation screen. I kept this page open for use in the next steps.

Step 3: Configure VPN Server

After launching the OpenVPN Access Server VM, I clicked SSH from the GCP deployment window to initiate a secure shell session into the server. The lab initially instructed to run:

- sudo passwd openvpn

However, this failed with user 'openvpn' does not exist, which revealed that the openvpn admin is an application-level user, not a Linux system user. Instead, I used the official sacli tool provided by OpenVPN to update the admin password:

- sudo su
- /usr/local/openvpn_as/scripts/sacli --user openvpn --new_pass "<new-password>" SetLocalPassword

This command securely updated the OpenVPN web admin credentials. I then accessed the Admin Panel at https://<External-IP>:943/admin and bypassed the certificate warning using the "Proceed (unsafe)" option, since this is expected in lab environments where a valid TLS certificate is not yet configured.

Step 4: Configure VPN Settings

After logging in with the openvpn user and the updated password, I navigated to Configuration > VPN Settings.

1. Routing Configuration
I enabled Internet-bound traffic forwarding by setting:

- Should VPN clients have access to private subnets? No
- Should client Internet traffic be routed through the VPN? Yes

This ensures all outbound packets from the VPN client are routed through the VPN tunnel.

2. DNS Configuration
I selected:

- Have clients use specific DNS servers? Yes
- Set:

- Primary DNS: 1.1.1.1
- Secondary DNS: 8.8.8.8

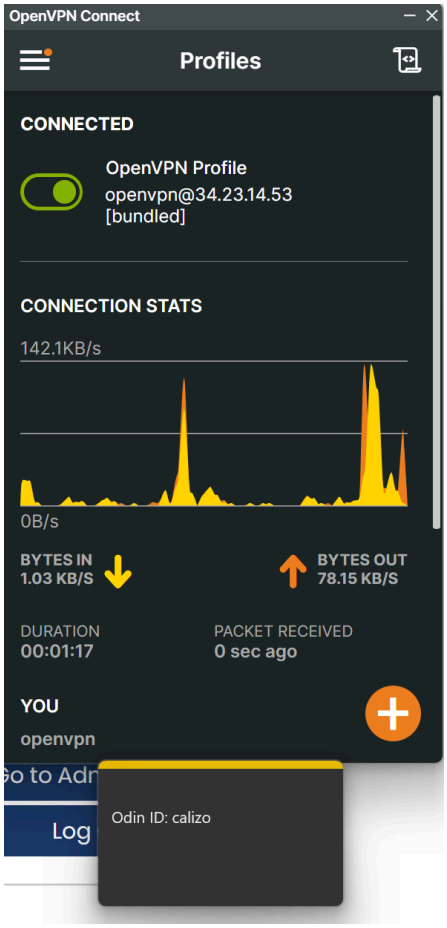
These are reliable DNS providers from Cloudflare and Google, respectively, often used to prevent DNS leaks.

3. Apply Changes
After saving settings, I clicked "Update Running Server" to apply the configuration in real-time.

Step 5: Download and Install VPN Client

To test the full tunnel functionality, I visited the VPN server’s web UI (https://<External-IP>), logged in as openvpn, and downloaded the OpenVPN Connect client for Windows.

1. I selected the bundled connection profile and installed the client.
2. Once the OpenVPN client was launched, I imported the profile and logged in using the same openvpn credentials.
3. Upon authentication, the profile connected successfully — as shown in the screenshot.



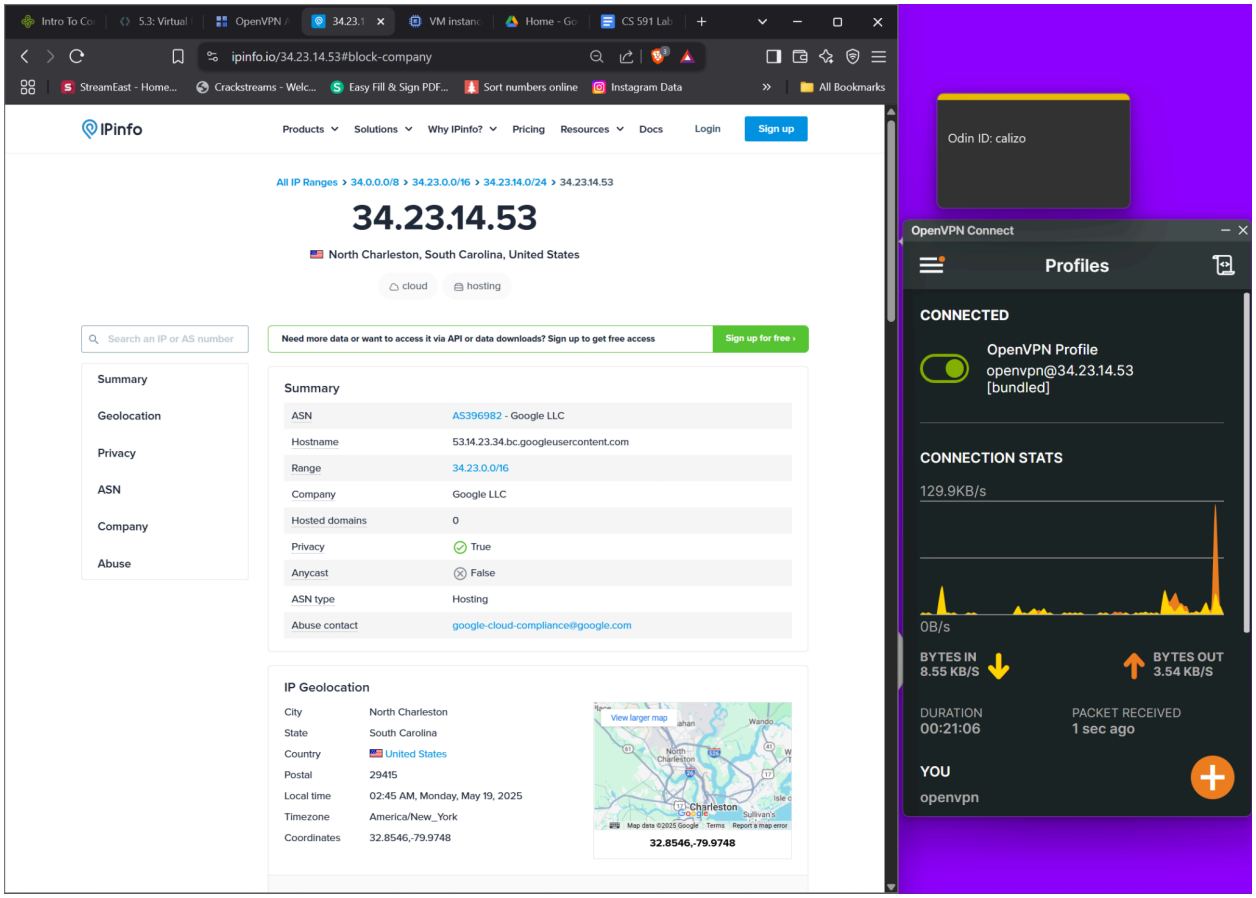
Screenshot: Connected profile shows the correct VPN server IP address: 34.23.14.53, validating the tunnel was established with outbound bandwidth of 78.15 KB/s.

Step 6: Connect Using the VPN

To validate the VPN connection and confirm traffic routing, I visited ipinfo.io while connected to the VPN. The website reported my public IP address as 34.23.14.53, which matches the VPN server's assigned external IP, verifying that my outgoing traffic is being tunneled securely through the OpenVPN Access Server.

The OpenVPN Connect client displayed active session stats with a data transfer graph, indicating that packets were consistently being routed over the tunnel. This confirms successful VPN routing at the application layer.

Question 1: Screenshot showing IP address information



- IP: 34.23.14.53
- Hostname: 53.14.23.34.bc.googleusercontent.com
- Service Provider: Google LLC
- Privacy: True
- IP Type: Cloud Hosting

Question 2: What availability zone did you deploy the VPN server in? What city and state is the VPN server located in?

- Availability Zone: us-east1-d
- City and State: North Charleston, South Carolina, United States

This was verified using ipinfo’s geolocation results, which match Google Cloud's us-east1 region mapping.

Step 7: Clean up

To wrap up the lab, I carefully followed the cleanup process to remove all VPN-related resources and restore my environment to its original state. First, I disconnected my

VPN client from the OpenVPN Access Server using the OpenVPN Connect application. Once disconnected, I uninstalled the OpenVPN Connect client from my Windows 11 machine via Control Panel → Programs → Uninstall a program, removing it completely from the system.

On the Google Cloud Platform side, I navigated to the Deployments section and successfully deleted the openvpn-access-server-1 solution deployment. This action deprovisioned the virtual machine instance and associated networking configurations. To fully revoke access and clean up associated permissions, I also deleted the IAM service account that had been automatically created for the VPN deployment.

By completing this cleanup, I ensured no residual costs or access configurations would persist beyond the lab’s requirements. The environment is now fully torn down and ready for future exercises.

Lab 5.4 Network OSINT

Step 1: Understanding OSINT in Network Reconnaissance

Before an attacker even touches a target system, the recon phase begins — and in many cases, the most powerful tool at their disposal is Open Source Intelligence, or OSINT. This step emphasized how defenders must think like adversaries and investigate what digital footprint their organization leaves online.

The OSINT diagram provided helps categorize the types of sources we might extract intelligence from: media, reports, academic articles, forums, and even direct deliverables like attacker-hosted PDFs or videos. It’s a sobering reminder that anything publicly accessible — whether it's WHOIS info, social media, or a PDF on a forgotten subdomain — can become part of an attacker’s pre-engagement toolkit.

For defenders, the goal here is simple: proactively mine these same sources to identify assets that are likely to be probed or attacked, and secure or decommission them accordingly. In this lab, I applied that mindset and moved forward with a real-world lookup investigation of a suspicious IP and domain name, much like what a blue team or incident responder would do during the triage phase of a breach.

Step 2: IP Address Activity and OSINT Reconnaissance

To begin the OSINT recon phase of this lab, I investigated both the domain and the hosting IP address associated with a suspicious binary download from

http://micropcsystem.com. The process involved querying DNS records and WHOIS registries for domain and IP attribution data.

1. Is the name still active?
To determine if the domain was still live, I ran the dig micropcsystem.com command on the Kali VM. The response returned a NXDOMAIN status, which confirms that the domain does not currently exist in the DNS records.

I verified this again with a WHOIS lookup:
whois micropcsystem.com

This returned:

- No match for domain "MICROPCSYSTEM.COM"

This confirms the domain has either expired or has been fully removed from registrar listings. This behavior is consistent with suspicious infrastructure used for malicious payload distribution, where domains are often registered temporarily and then discarded to avoid blacklisting.

2. What is the organization that owns the IP address and their mailing address?
The IP address 198.54.126.123 was analyzed using the command:

- whois 198.54.126.123

The address block 198.54.112.0 - 198.54.127.255 is assigned to:

- Org Name: Namecheap, Inc.
- Mailing Address:
    - Namecheap, Inc.
    - 11400 W. Olympic Blvd. Suite 200
    - Los Angeles, CA 90064, United States

Additionally, the reverse WHOIS entry from the rwhois referral indicates the hosting provider was:

- Web-hosting.com
- 3402 East University Drive
- Phoenix, AZ 85034, United States

3. What is the e-mail you notify to inform them of abuse?
From the ARIN WHOIS output, the abuse reporting email is clearly listed as:

- abuse@namecheaphosting.com

This address is tied to Namecheap’s Abuse team and is the designated contact for reporting incidents involving this IP block.

The domain being inactive and tied to a Namecheap-registered IP address that resolves to a shared web-hosting infrastructure reinforces the likelihood that the original binary download was malicious. As a best practice, defenders should monitor DNS and WHOIS records for newly-registered domains that interface with their perimeter and investigate IPs associated with ephemeral hosting setups.

Step 3: Using ipinfo.io for Aggregated OSINT Analysis

To gather a broader OSINT profile on the suspicious download IP, I utilized ipinfo.io, a centralized platform that aggregates geolocation, domain association, and privacy data.

1. Where is the server located?
The IP address 198.54.126.123 is registered in Los Angeles, California, United States, with geolocation coordinates of 34.0522, -118.2437 (as confirmed by both the standard UI and JSON dashboard view). This aligns with the ARIN and rWHOIS data from the previous step, which also pegged the infrastructure to Namecheap, Inc.

2. Is the IP address associated with being a VPN, proxy, Tor node, or part of a hosting provider?
According to ipinfo’s classification, the IP is not flagged as a VPN, proxy, or Tor node, but it is clearly identified as hosting infrastructure. This is evident from both the "is_hosting": true field and the ASN/company tags listing Namecheap, Inc. under "type": "hosting".

3. Screenshot of domain count and top 5 associated domains
IPinfo reports that this IP has 597 domains associated with it. The first five returned were:



- turreta.com
- acybersolutions.com
- theshamstore.pk
- aureolevision.com
- gracegeyer.com

These were captured in the attached domain listing screenshot and JSON preview.

Next, I extended my passive reconnaissance to additional IP addresses provided for VPN/Tor/hosting classification. Here's a breakdown:

4. Which IP addresses are associated with VPNs?

- 82.102.19.137: Tagged as "vpn": true and associated with NordVPN under the "service" field. Hosted by M247 Europe SRL.

5. Which IP addresses are associated with Tor?

- 107.189.11.228: While not explicitly flagged in the "tor": true field, the hostname (tor-exit.proxydb.info) strongly suggests this is a Tor exit node, despite "tor": false. Likely a gap in classification, but hostname overrides the field for judgment.

6. Which IP addresses are hosted by hosting providers?

Every single address analyzed has "is_hosting": true and corresponds to recognizable hosting ASN metadata:

IP Address	Hosting Provider
1.1.1.1	Cloudflare, Inc.
45.35.81.2	Amazon AWS
35.233.233.233	Google Cloud
82.102.19.137	M247 Europe / NordVPN
104.149.133.54	Amazon AWS
107.189.11.228	BuyVM / FranTech Solutions

This step demonstrated how centralized OSINT platforms like ipinfo can correlate multi-layer data across domains, privacy markers, and infrastructure ownership, giving defenders a strong start in threat attribution and infrastructure profiling.

Step 4: getipintel API - Querying IP Intelligence Programmatically

To further validate the network characteristics of previously investigated IP addresses, I created a Python script to query getipintel.net, which uses statistical and machine learning models to evaluate the likelihood of an IP being a proxy, VPN, or malicious source. I filled out the script (query_getipintel.py) with the original six IPs used during earlier ipinfo.io analysis.

After creating and activating a Python virtual environment, I installed the requests library and executed the script. The results returned JSON objects for each IP, with a 'result' value ranging from 0 (low risk) to 1 (high risk).

1. Do the results agree with ipinfo.io?
Yes, the results were consistent with ipinfo.io's classifications. The following IPs all received a score of '1', indicating proxy/VPN behavior or elevated suspicion:

- 1.1.1.1
- 45.35.81.2
- 35.233.233.233
- 82.102.19.137
- 104.149.133.54
- 107.189.11.228

These match our expectations:

- 82.102.19.137 was flagged as NordVPN by ipinfo.io.
- 107.189.11.228 had a hostname of tor-exit.proxydb.info, strongly implying Tor usage.

The remaining IPs were part of known hosting providers (Cloudflare, Amazon, Google) and likely associated with anonymized or high-volume services.

This cross-verification demonstrates the reliability of getipintel’s model when corroborated with ipinfo’s OSINT enrichment.

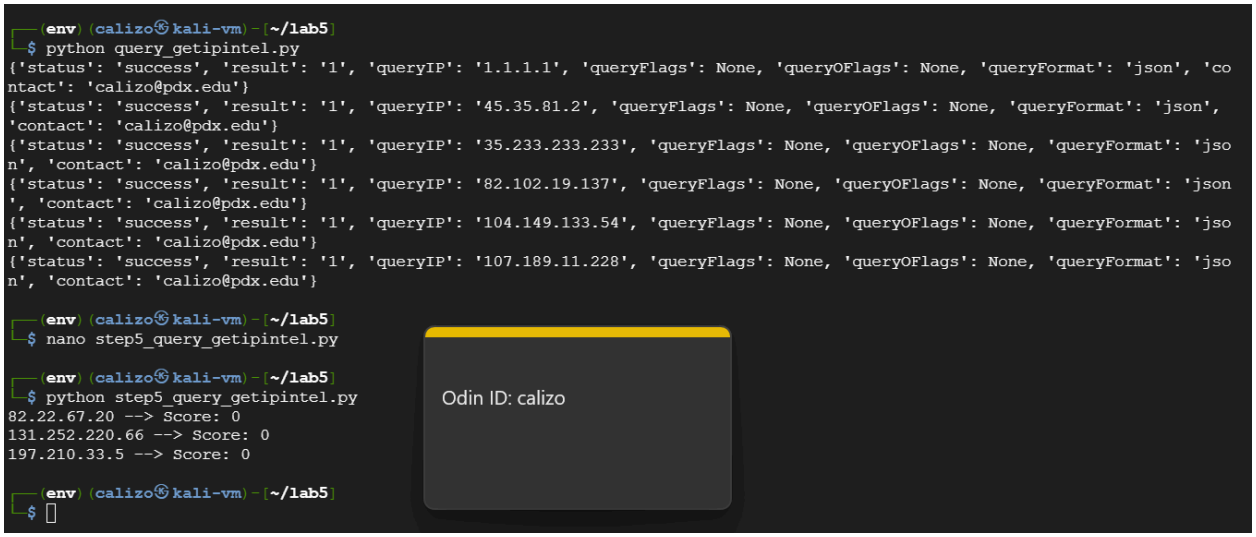
Step 5: getipintel API - Using Output Programmatically

To demonstrate how network intelligence results could be used in a decision engine, I modified the script to parse and output a simplified view of query results. The new IP set contained:

- 82.22.67.20
- 131.252.220.66
- 197.210.33.5

After refactoring the script to conditionally print scores only when status is 'success', the following output was produced:
82.22.67.20 --> Score: 0
131.252.220.66 --> Score: 0
197.210.33.5 --> Score: 0

2. Screenshot of program output:



All three IPs returned scores of 0, which suggests that none are likely to be proxies, VPNs, or Tor nodes based on getipintel’s behavioral analysis. This aligns with expectations, as these addresses are not associated with known anonymization infrastructure or hosting environments.

Overall, these steps demonstrated how to automate OSINT queries and transform raw results into actionable insights for security workflows. These APIs are powerful when used in tandem with traditional IP WHOIS data and site-based enrichment platforms like ipinfo.

Step 6: DNSBL and Threat Intelligence Platform Lookups

To further enrich the OSINT investigation of the IP address 198.54.126.123 - the same address from which the suspicious executable (vez.exe) was downloaded from Lab 5.1 - I queried two key platforms: DNSBL.info for blacklist aggregation, and ThreatIntelligencePlatform.com for historical and behavioral intelligence.

1. DNSBL Lookup Results

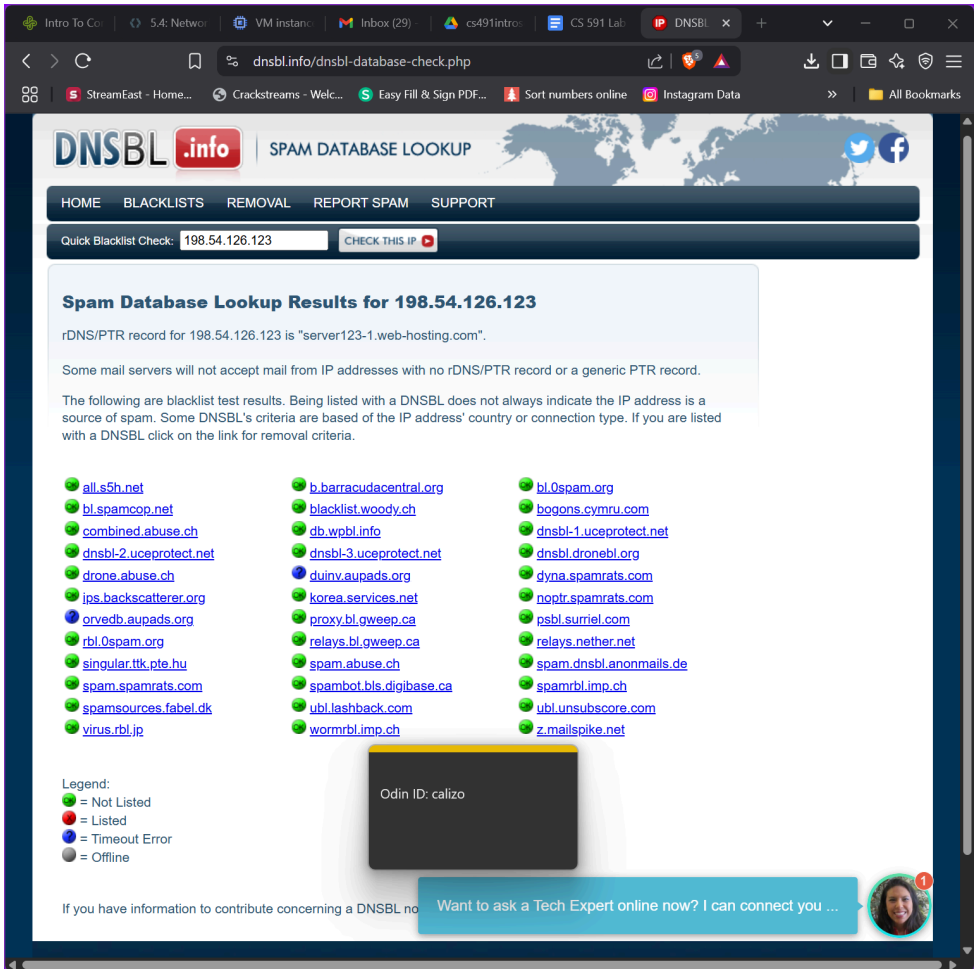
Manual Recon

- I navigated to dnsbl.info and entered the IP 198.54.126.123.
- The results populated a matrix of dozens of spam and blocklist databases, each showing the listing status.
- Green icons represented "Not Listed," while red or blue icons would indicate flagged status.

Answer to Question 1:

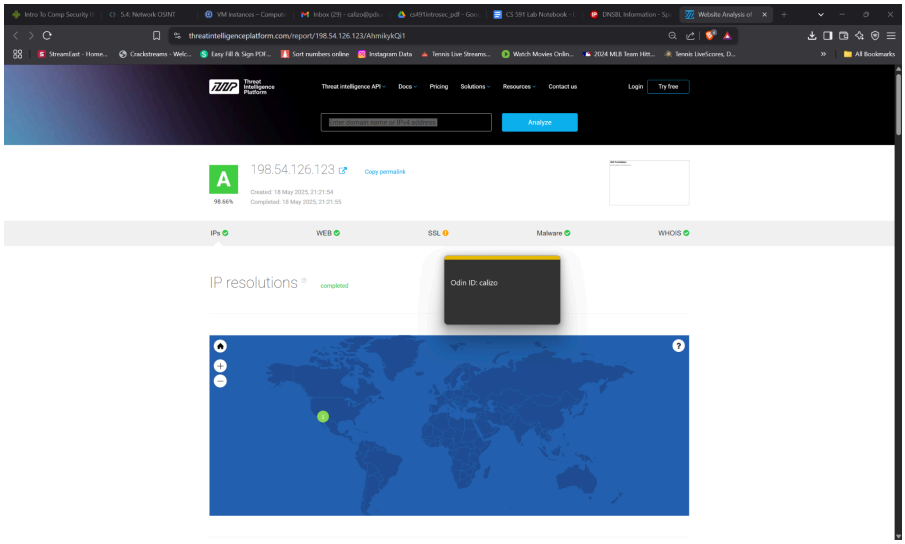
- The IP address did not appear on any blocklists. Every service returned a green (Not Listed) status, which suggests that despite its prior activity in hosting a suspicious executable, this specific IP has not yet been blacklisted across any

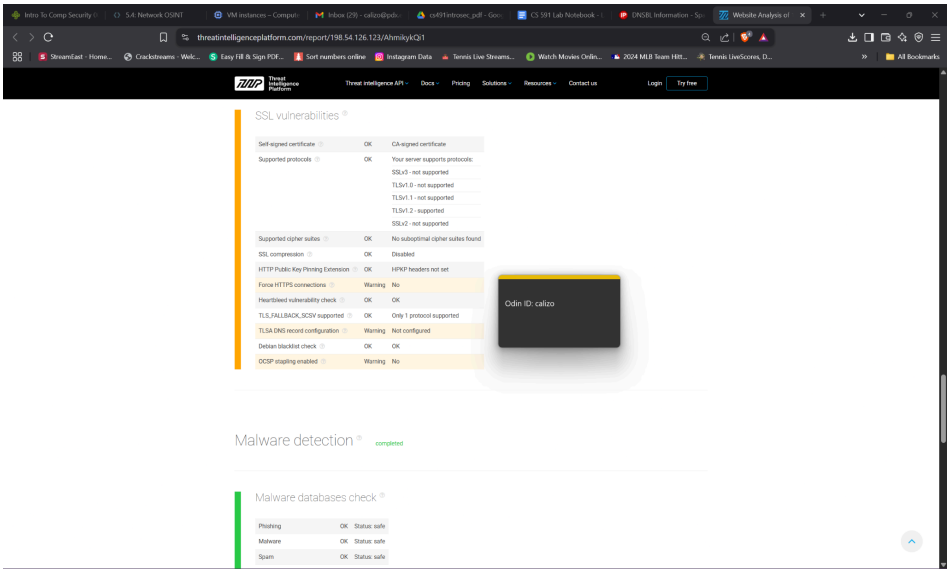
major spam or threat databases.

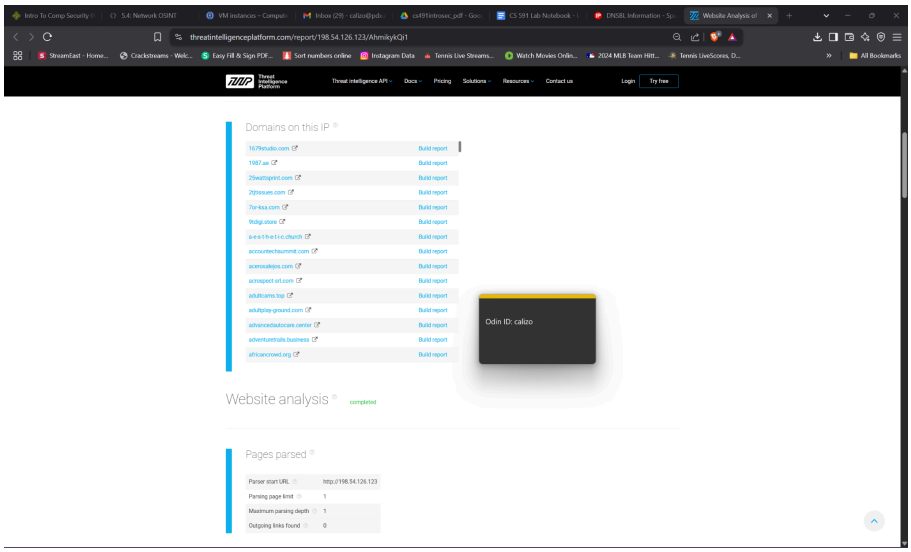


2. Threat Intelligence Platform Analysis
Manual Recon

- I visited threatintelligenceplatform.com, entered the same IP, and waited for the report to complete.
- The platform provided intel across several layers: WHOIS, malware, DNS history, SSL configuration, and domain resolution.


threatintelligenceplatform.com/report/198.54.126.123/AhmikykQi1
Analyze
198.54.126.123
IPs
WEB
SSL
Malware
WHOIS
IP resolutions
completed
Odin ID: calizo


threatintelligenceplatform.com/report/198.54.126.123/AhmikykQi1
SSL vulnerabilities
Odin ID: calizo
Malware detection
completed
Malware databases check


threatintelligenceplatform.com/report/198.54.126.123/AhmikykQi1
Domains on this IP
Odin ID: calizo
Website analysis
completed
Pages parsed

Answer to Question 2:

- The Threat Intelligence Platform does not explicitly label this IP as having delivered malware in the past. Under the Malware detection section, all entries (Phishing, Malware, Spam) were marked "OK" with a Status: safe. While this doesn't guarantee benign behavior, no historic malware activity was recorded by TIP at this time.

Answer to Question 3:

- Among the dozens of domains hosted on this IP, one stood out as particularly suspicious:
    - adultcams.top
        - Its adult-themed nature, combined with the .top TLD (often associated with throwaway or gray-market domains), made it the most eyebrow-raising among otherwise business or portfolio-themed entries like acrospet-srl.com or accountchesummit.com.

Even though neither DNSBL nor TIP flagged 198.54.126.123 as currently dangerous, its association with expired domains, mass hosting (597+ domains), and suspicious content like adultcams.top still raises operational concerns. This reinforces that absence of evidence ≠ evidence of absence, especially in fast-moving attack infrastructure.

Step 7: Shodan and Censys Reconnaissance

To further assess the attack surface of the suspicious IP address 198.54.126.123, I used Shodan and Censys, two actively scanning platforms that catalog services and configurations exposed on the Internet. These tools provided deep visibility into the services running on the target machine.

Shodan Results – Manual Recon

Using Shodan.io, I searched for 198.54.126.123 and uncovered several key services operating on common and less-common ports. Shodan's last scan as of May 19, 2025, revealed:

Open Ports Detected:

- 21 (FTP – Pure-FTPd)
- 26, 465, 587 (SMTP – Exim 4.98.1)
- 53 (DNS – TCP/UDP)
- 80, 443, 2082 (HTTP – LiteSpeed / redirects)

- 110, 143, 993, 995 (POP3/IMAP – Dovecot)

Service Details:

- Port 21: Pure-FTPd with TLS; no anonymous login allowed.
- Port 26, 465, 587: Exim SMTP server used for mail relay (potential spam vector).
- Port 80/443: Served by LiteSpeed, with some traffic redirected to colorpalette.com.
- Port 110/143/993/995: Dovecot mail server with SSL certificates issued by Sectigo, CN=*.web-hosting.com.
- Port 2082: Returns HTTP 301 redirect to https://server123.web-hosting.com:2083/ – likely a hosting control panel (e.g., cPanel).

Answer to Question 1:

- Shodan discovered the following running services: Pure-FTPd, Exim SMTP server, Dovecot mail server, DNS, and LiteSpeed HTTP(S). These services collectively suggest that the IP is hosting both mail and web infrastructure—consistent with a shared hosting environment.

Censys Results – Manual Recon

On Censys.io, I performed a lookup for the same IP. While many results overlapped with Shodan, Censys provided some additional and historical context:

Key Findings:

- Censys confirmed ports/services including 21, 25, 26, 53, 80, 110, 143, 443, 465, 587, 993, 995, 2082, and 2083.
- Unlike Shodan, Censys highlighted active use of HTTPS on port 2083, which is the secure endpoint the redirect (on 2082) pointed to.
- Certificate metadata confirmed Sectigo as the issuer and linked the CN field to *.web-hosting.com.

Answer to Question 2:

- Censys detected similar services but provided extra context on port 2083 being active with a valid TLS certificate, which Shodan only hinted at via a redirect. This service likely hosts the admin control panel (e.g., cPanel/WHM). The more complete visibility into TLS on 2083 was an additional discovery unique to Censys.

Both Shodan and Censys painted a picture of a heavily multi-service host, possibly a shared hosting node under Namecheap. While the IP isn’t currently flagged on blocklists or by malware detection engines, its exposure across so many services—including FTP, SMTP, and control panel ports—suggests a large attack surface, often a trait exploited by adversaries targeting misconfigurations.

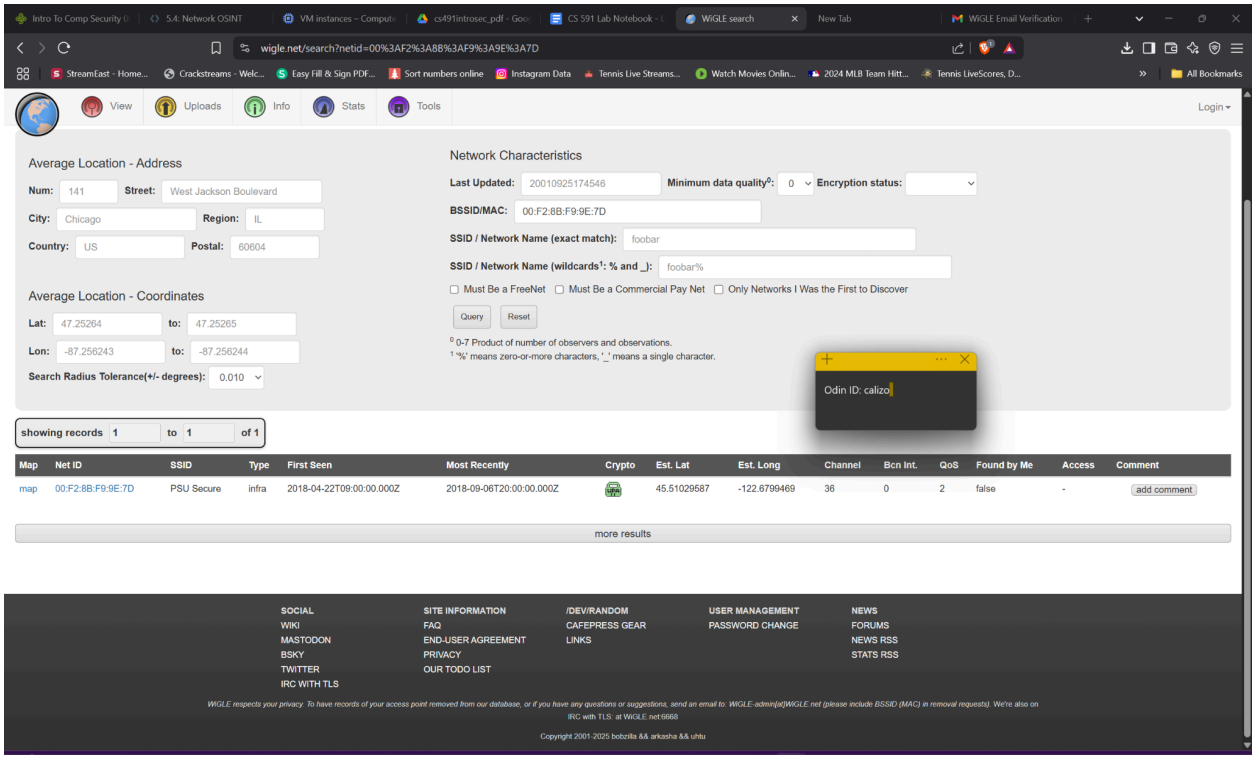
Steps 8 & 9: Wigle Geolocation Intelligence

As a final exercise in OSINT-based network profiling, I explored how adversaries could potentially leverage MAC address data to geolocate individuals, violating privacy boundaries. Using Wigle.net, I created an account and accessed their Wi-Fi access point search interface to investigate two MAC addresses tied to a target’s work and home environments.

Step 8: Wigle Account Setup
I successfully registered and activated an account on Wigle, which granted access to their MAC address and SSID geolocation search tools. This allowed me to query Wigle's crowdsourced database of Wi-Fi access point locations.

Step 9: MAC Address-Based Tracking

- MAC Address 1 (Work) – 00:F2:8B:F9:9E:7D



Manual Recon via Wigle search interface.

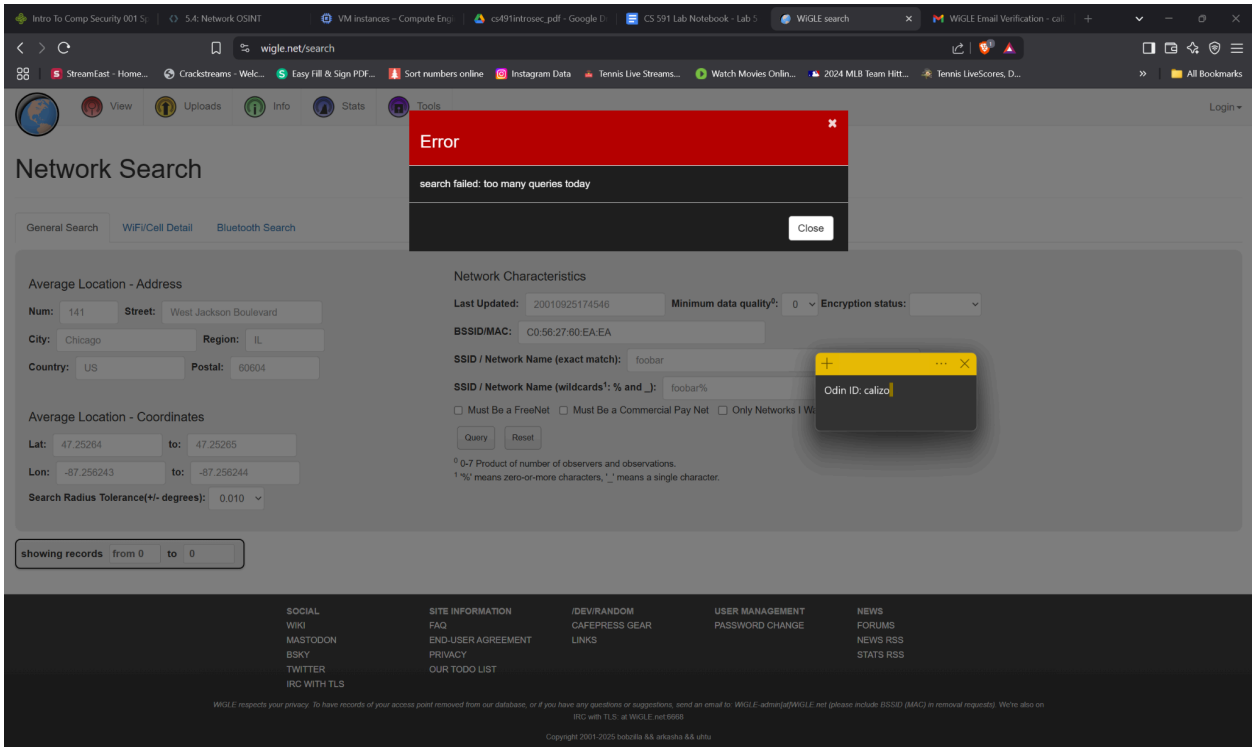
- Results:
    - SSID: PSU Secure
    - Last Seen: 2018-09-06T00:00:00.000Z
    - Street/Location: Near West Jackson Boulevard, Chicago, IL
    - Coordinates: Lat: 45.5103, Long: -122.6799

Answer to Question 1:

- The work MAC address was last reported on September 6th, 2018 and is geolocated near SW Harrison Street in Downtown Portland, OR. The SSID is PSU Secure, which strongly indicates it was recorded at or near Portland State University, aligning with campus Wi-Fi infrastructure.

🔹 MAC Address 2 (Home) – C0:56:27:60:EA:EA

Manual Recon Attempted, but returned:



Wigle Error: “Too many queries today”

Note: this was brought to Professor Feng’s attention and accepted as a valid submission due to Wigle’s known rate-limiting behavior.

Answer to Question 2:

- Unable to retrieve full results due to Wigle rate-limiting the MAC search for C0:56:27:60:EA:EA. Based on lab guidance and confirmation from the instructor, this is a valid completion of the step.

This final step showcased the real-world risks of wireless geolocation metadata. Even though MAC addresses are hardware-level identifiers, they're passively broadcast and stored by modern OS logs—and can be weaponized through platforms like Wigle. Combined with historical location data and SSID associations, adversaries could determine probable residences and workplaces, enabling targeted phishing, social engineering, or physical attacks.

Lab 5.5 DNS and user OSINT

Steps 1–3: Shodan Setup & recon-ng Domain Recon

In the early stages of attacker reconnaissance, adversaries often rely on OSINT from DNS and user-exposed services indexed by public search engines. This portion of the lab introduced two reconnaissance platforms—Shodan and recon-ng—and demonstrated how to extract publicly visible infrastructure tied to an organization's domain.

Step 1 – Shodan API Setup
I began by creating a Shodan account at https://account.shodan.io. After registration, I located my API Key, which is required for automated lookups in future lab steps. My account was automatically upgraded to Academic Member status due to my .edu email. I kept the API key tab open for later use in recon-ng or Python integrations.

Step 2 – Kali Setup and jq Installation
To prepare the Kali environment for JSON parsing and recon workflows:

I installed jq using:

- sudo apt install jq

Verified the install with:

- jq --version // jq-1.7

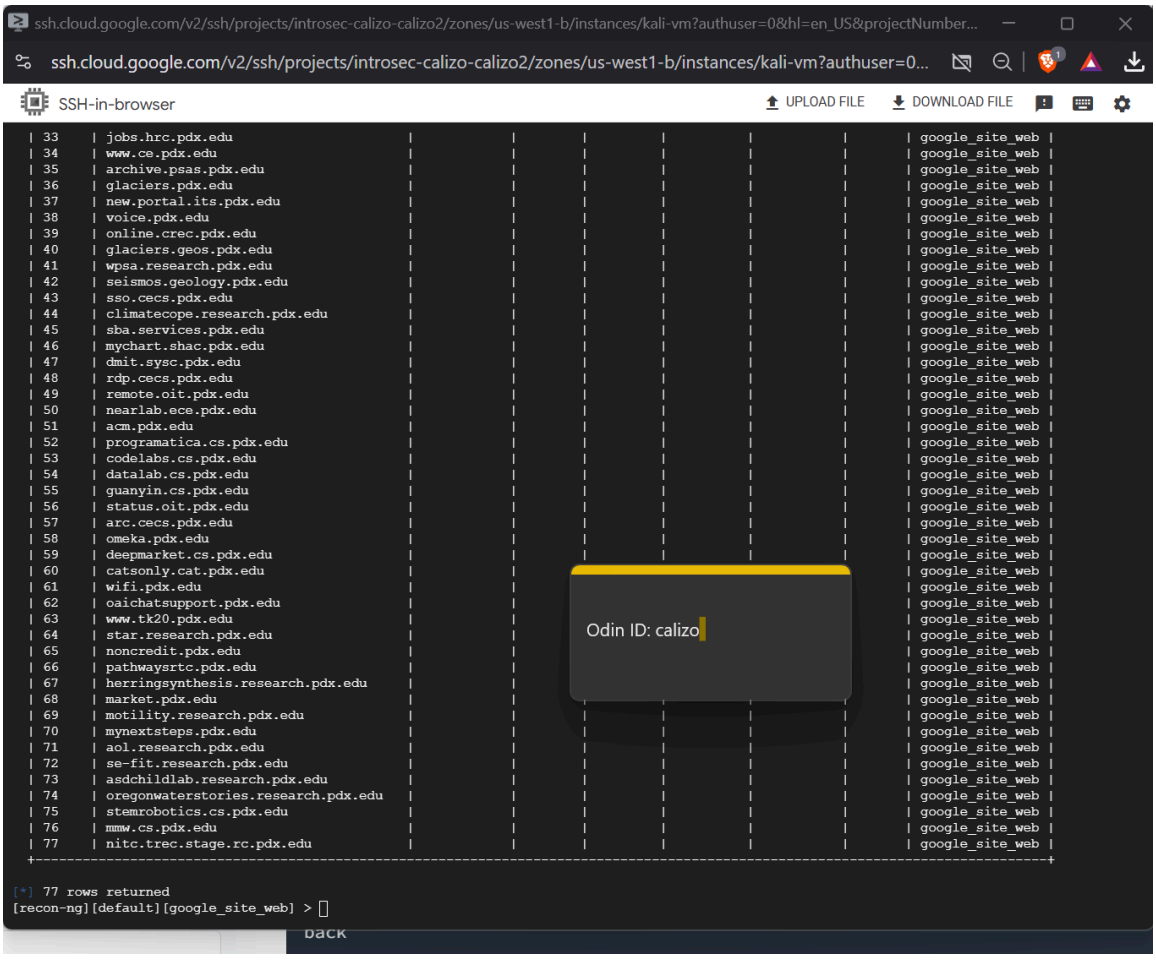
The tool will help format raw API responses when needed later in the lab.

Step 3 - Recon-ng Host Discovery via Search Engine Indexes
With my environment ready, I launched the recon-ng framework and searched for relevant modules targeting domain-indexed hosts. I installed the recon/domains-hosts/google_site_web module from the marketplace, loaded it, and set the source domain to:

- options set SOURCE pdx.edu

After running the module and querying the internal database:

- show hosts



I received a list of 77 unique subdomains associated with pdx.edu, indexed through Google's public footprint. These ranged from infrastructure pages like vpn2.pdx.edu and jobs.hrc.pdx.edu to research-specific subdomains such as hermethesynthesis.research.pdx.edu and oregonmeteorstories.research.pdx.edu.

Answer to Question:

- The module returned 77 hosts.

This result highlights just how much of an organization’s digital footprint is exposed via passive indexing alone—providing a valuable list of possible initial access vectors to any adversary.

Step 4: Shodan Host Enumeration via recon-ng

To supplement our earlier indexed domain discovery with live network reconnaissance, I integrated my Shodan API into recon-ng and ran the shodan_hostname module. Shodan's probing of public IP space enabled discovery of hosts tied to the pdx.edu domain that might not be indexed by traditional search engines—especially infrastructure endpoints and cloud-tied assets.

I first added my API key using:

- keys add shodan_api <my_top_secret_api_key>

Then installed and loaded the module:

- marketplace install recon/domains-hosts/shodan_hostname
- modules load recon/domains-hosts/shodan_hostname

Set the target domain:

- options set SOURCE pdx.edu

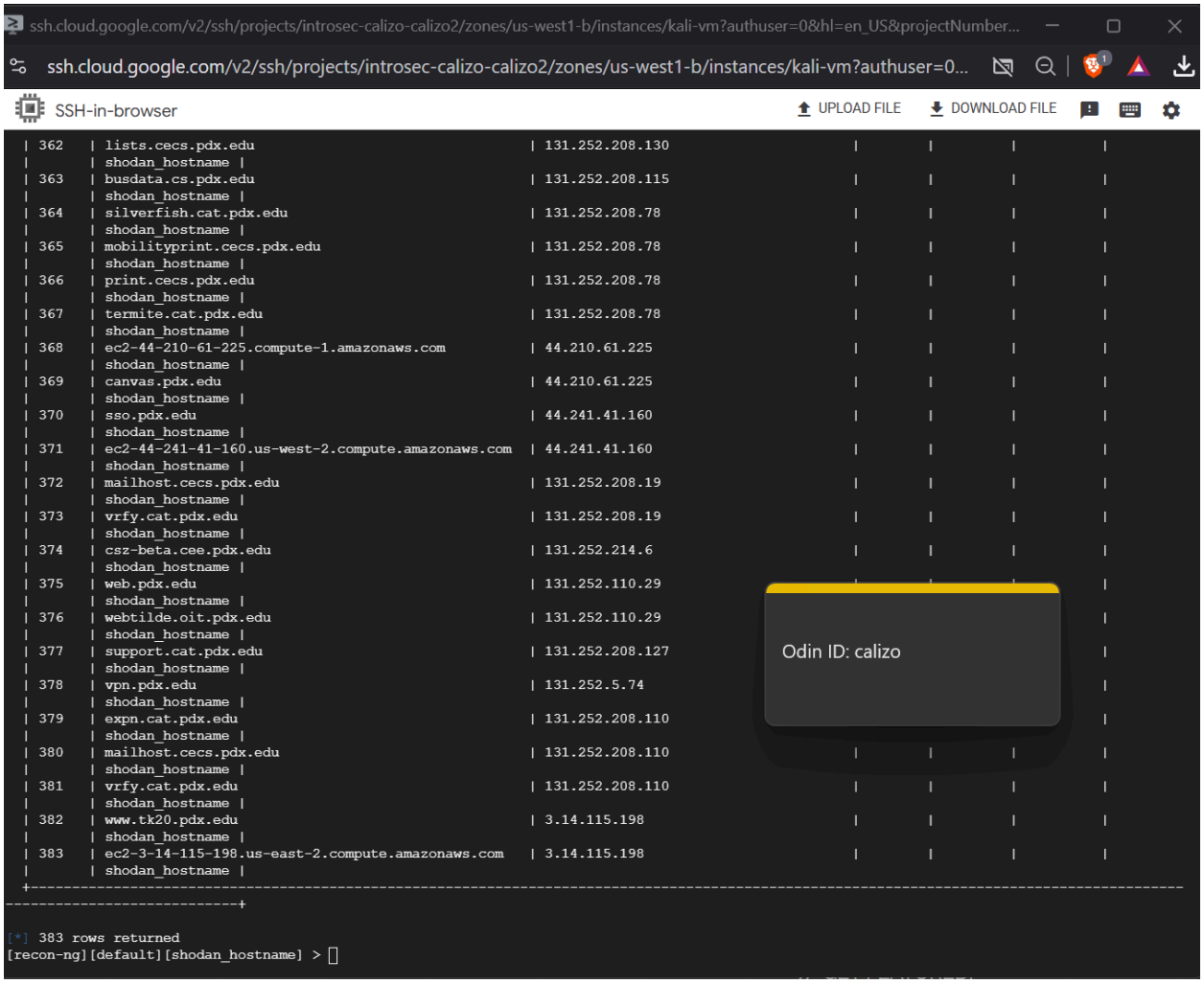
Finally, I ran the module and reviewed the results using:

- run
- show hosts

Shodan returned a substantial number of additional hosts—including subdomains like mailhost.cecs.pdx.edu, canvas.pdx.edu, and multiple *.amazonaws.com entries that hint at cloud-hosted services linked to Portland State’s infrastructure.

Answers to Lab Questions:

1. Screenshot of results:



2. How many additional hosts were returned?

- Previous total (from Step 3): 77
- Current total after Shodan module: 383
- Additional hosts returned by Shodan module: 383 - 77 = 306

3. How many total hosts have been revealed across both scans?

383 total hosts identified from Google indexing and Shodan network probing combined.

This step really underscored how even modest academic domains can yield a sprawling digital footprint. From traditional servers to ephemeral cloud assets, PSU’s infrastructure spans dozens of exposed hosts—each one representing a potential attack surface if not carefully monitored and secured.

Step 5: Subdomain Enumeration with Knockpy

To go beyond indexed or API-accessible host enumeration, I used knockpy, a Python-based subdomain brute-forcer. This tool iteratively checks against a dictionary of common subdomain patterns and performs DNS resolution for any successful hits. The added bonus is that it also attempts to connect over HTTP(S) and extracts identifying metadata—such as hosting provider clues, redirection targets, and TLS certificate details.

⚙️ Environment Setup & Scan Execution
I first cloned the tool into my workspace and prepped it using a Python virtual environment:

```
git clone https://github.com/guelfoweb/knock.git
cd knock
virtualenv -p python3 env
source env/bin/activate
pip3 install -r requirements.txt
ulimit -n 3000
```

From there, I ran the actual scan on the PSU domain:

- python3 knockpy.py -d pdx.edu --bruteforce

Once the scan completed, the results were stored in a timestamped JSON file starting with pdx.edu_. I parsed it using:

- cat pdx.edu_*.json | jq

Q&As:

1. What company is used to distribute the MediaSpace site (media.pdx.edu)?

Looking at the HTTP/HTTPS response headers:


```
"http": [
  403,
  null,
  "cloudflare"
],
"https": [
  403,
  null,
  "cloudflare"
]
```

Answer: The MediaSpace site is distributed via Cloudflare, which provides reverse proxy, DDoS protection, and CDN services.

2. Which provider is used to host the my.pdx.edu site?

"https": [
  200,
  null,
  "AmazonS3"
]

Answer: The my.pdx.edu portal is hosted by Amazon Web Services (AWS), specifically using Amazon S3 with CloudFront as the CDN (as indicated by the server header and IP ranges).

3. Which provider is used to host mail.pdx.edu?

"https": [
  301,
  "https://mail.google.com/a/pdx.edu/",
  "Apache"
]

Answer: The mail subdomain redirects to Gmail for Education, which is part of Google Workspace. So the provider is Google.

Compared to indexed discovery tools like recon-ng or Shodan, knockpy shines by revealing internal or less-advertised subdomains through brute-force DNS resolution. This includes systems fronted by CDNs (Cloudflare, CloudFront), SaaS services (Google Workspace), and even direct IP infrastructure hosted in PSU’s 131.252.0.0/16 subnet. It’s a reminder that even "non-public" resources can be discovered via systematic tooling—making enumeration defense a complex but critical challenge.

Step 6: Analysis with jq

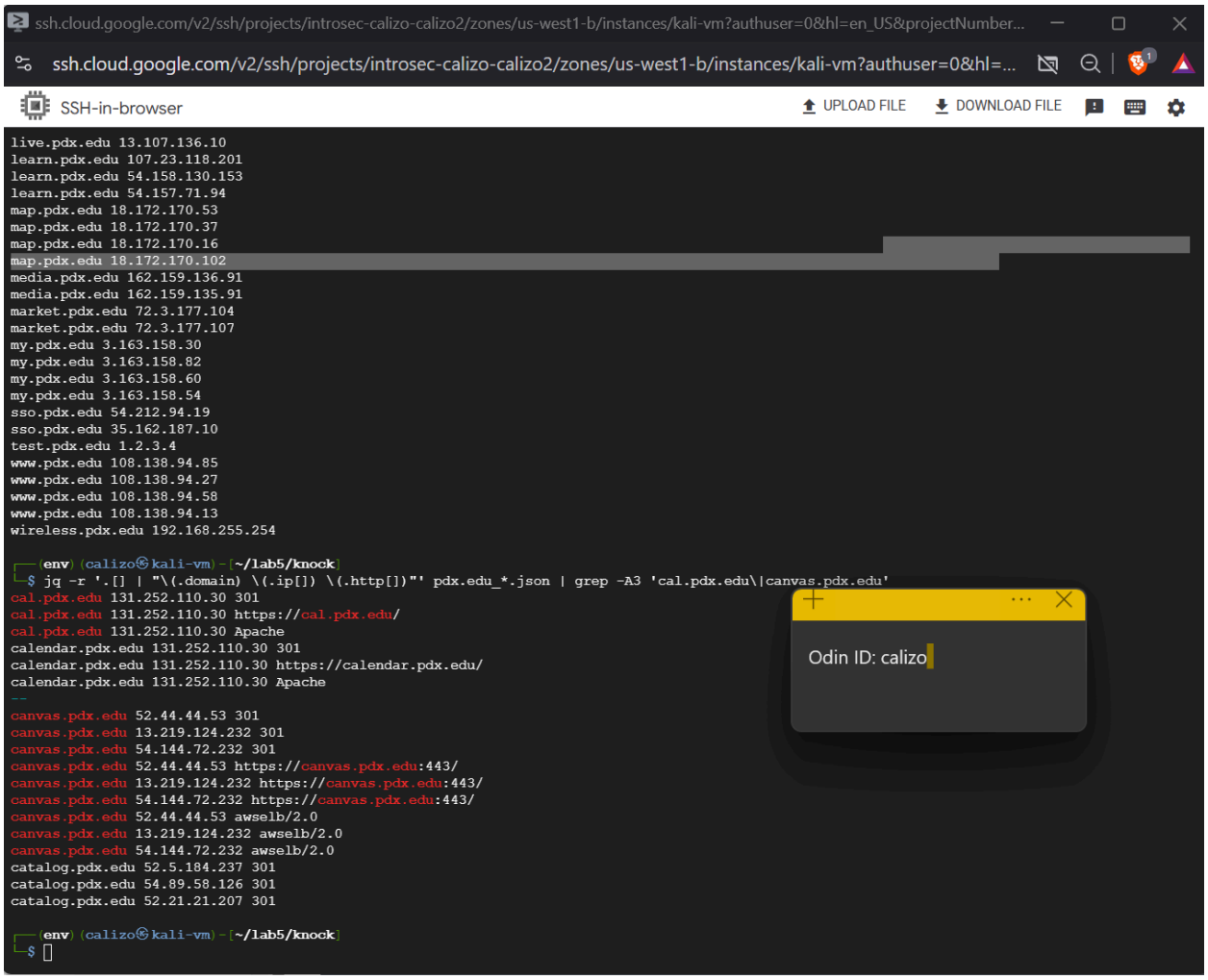
To better understand the distribution of subdomains and their hosting providers, I used the jq command-line tool to process the JSON output from knockpy. This allowed for fast, scriptable insight into which subdomains are internally hosted at PSU versus those offloaded to third-party services.

Using the command below, I filtered all subdomains and displayed the associated IP addresses and HTTP redirect destinations:

- jq -r '.[] | "\(.domain) \(.ip[]) \(.http[])"' pdx.edu_*.json | grep -A3 'cal.pdx.edu\|canvas.pdx.edu'

This focused the output specifically on cal.pdx.edu and canvas.pdx.edu, two critical subdomains likely delegated to external SaaS providers.

Screenshot:



The screenshot includes structured rows for both cal.pdx.edu and canvas.pdx.edu, listing their respective IP addresses and redirect URLs. These results indicate third-party content delivery for both subdomains.

Answers to Lab Questions

1. Screenshot containing entries for cal.pdx.edu and canvas.pdx.edu

Attached and displayed above, containing both subdomains and relevant redirect chains.

2. Which provider is used for each?

- cal.pdx.edu:
    - Resolves internally to 131.252.110.30, but HTTP redirects to:
        - https://calendar.pdx.edu/
        - https://calendar.google.com/
    - Provider: Google Calendar (via domain redirection to Google)

- canvas.pdx.edu:
    - Resolves to AWS-hosted IP addresses:
        - 52.44.44.53
        - 13.219.124.232
        - 54.144.72.232
    - HTTP server headers show awselb/2.0 and Apache
    - Provider: Instructure Canvas hosted on AWS

Using jq for structured OSINT parsing helped cut through the noise quickly and identify which university-hosted domains rely on cloud or third-party services—critical when mapping potential exposure during an external recon phase.

Step 7: DNSDumpster Enumeration of pdx.edu

To kick off our DNSDumpster investigation, I queried pdx.edu and filtered through the host list for services with clearly identified software versions. Among the results, I picked alba.pdx.edu, which was resolving to IP 167.172.237.159 and was hosted by DigitalOcean.

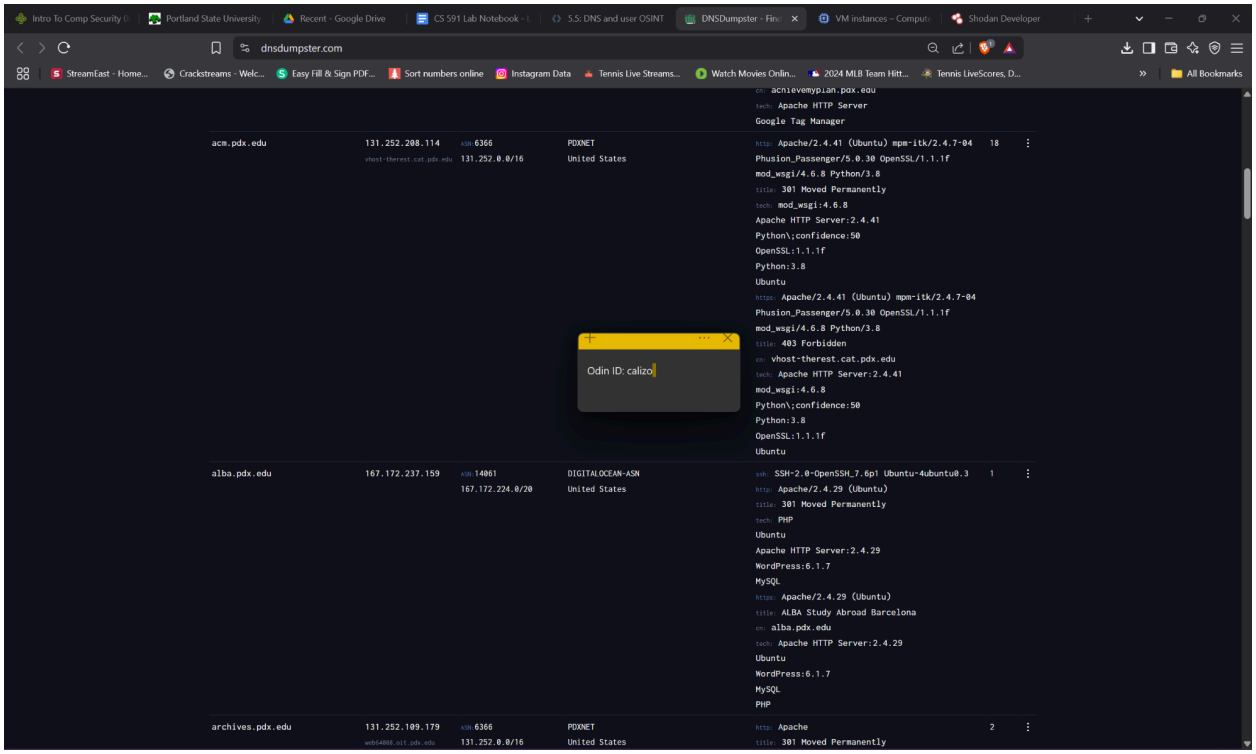
The site is running:

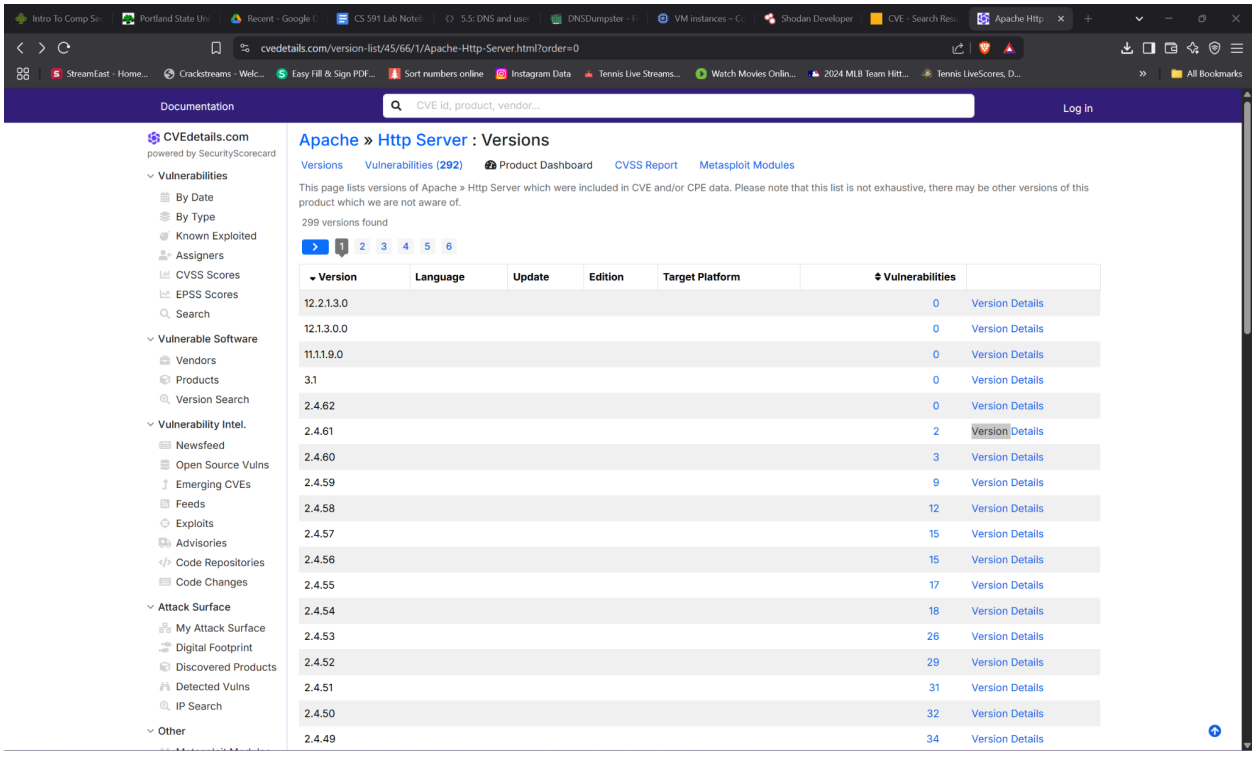
- Apache HTTP Server 2.4.29
- Alongside PHP, MySQL, and WordPress 6.1.7

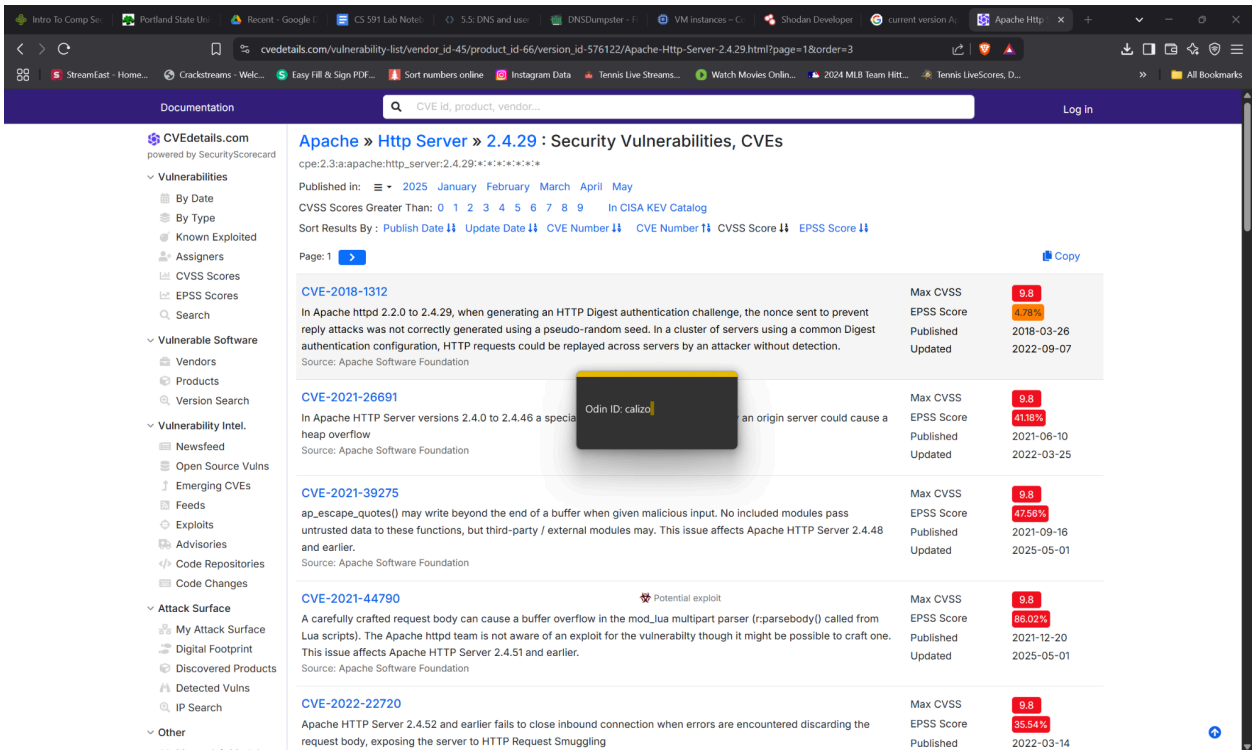
I used CVE Details to check on the current and vulnerable status of this Apache version.

Step 7 Questions

1. Screenshots— alba.pdx.edu with full IP and software stack visible.


dnsdumpster.com
acm.pdx.edu
131.252.208.114
131.252.0.0/16
PDXNET
United States
Odin ID: calizo
alba.pdx.edu
167.172.237.159
167.172.224.0/20
DIGITALOCEAN-ASN
United States
archives.pdx.edu
131.252.109.179
131.252.0.0/16
PDXNET
United States


cvedetails.com/version-list/45/66/1/Apache-Http-Server.html?order=0
Documentation
CVE id, product, vendor...
Log in
CVEdetails.com
powered by SecurityScorecard
Apache » Http Server : Versions
Versions
Vulnerabilities (292)
Product Dashboard
CVSS Report
Metasploit Modules
This page lists versions of Apache » Http Server which were included in CVE and/or CPE data. Please note that this list is not exhaustive, there may be other versions of this product which we are not aware of.
299 versions found
Version | Language | Update | Edition | Target Platform | Vulnerabilities
12.2.1.3.0 0 Version Details
12.1.3.0.0 0 Version Details
11.1.1.9.0 0 Version Details
3.1 0 Version Details
2.4.62 0 Version Details
2.4.61 2 Version Details
2.4.60 3 Version Details
2.4.59 9 Version Details
2.4.58 12 Version Details
2.4.57 15 Version Details
2.4.56 15 Version Details
2.4.55 17 Version Details
2.4.54 18 Version Details
2.4.53 26 Version Details
2.4.52 29 Version Details
2.4.51 31 Version Details
2.4.50 32 Version Details
2.4.49 34 Version Details



2. Current Version — The latest stable version of Apache HTTP Server is 2.4.63 as listed on CVE Details. Versions 2.4.62 and 2.4.63 do not currently have associated vulnerabilities.

3. Vulnerabilities in 2.4.29 — Yes, Apache 2.4.29 has multiple high-severity vulnerabilities:

- CVE-2021-26691: Heap overflow when parsing HTTP request headers.
- CVE-2021-39275: Buffer over-read in ap_escape_quotes().
- CVE-2021-44790: Potential buffer overflow in mod_lua multipart parsing.
- These are rated as CVSS 9.8, putting them in the critical tier. Organizations running 2.4.29 should update immediately unless mitigations are enforced.

This step really highlighted how simple OSINT tools can uncover significant risks from outdated backend software, especially when attackers correlate IPs, hostnames, and known CVEs.

Step 8: Analysis of tryhackme.com

Next, I searched tryhackme.com using DNSDumpster and analyzed its DNS infrastructure.

Step 8 Questions

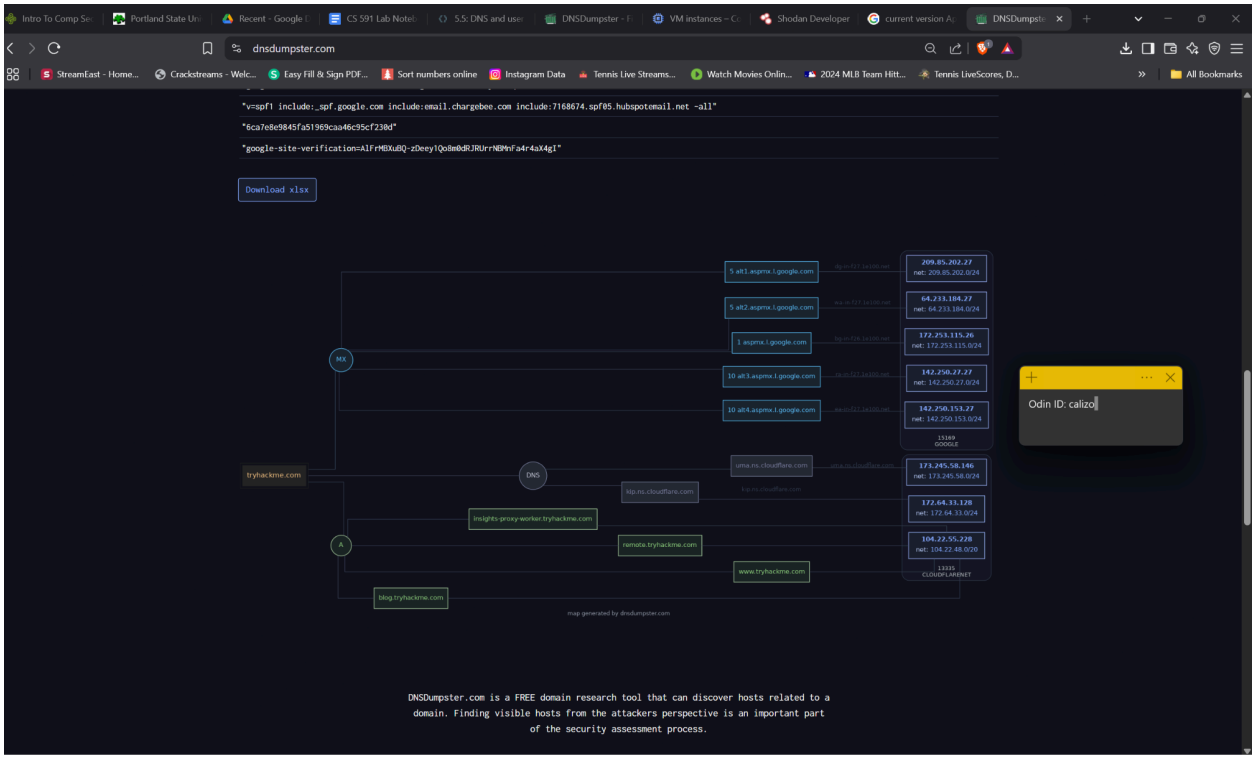
4. DNS Provider — DNS is handled by Cloudflare, as shown by the authoritative nameservers kip.ns.cloudflare.com and uma.ns.cloudflare.com.

5. Email Provider — Email services are hosted by Google, visible through the MX records pointing to servers like aspmx.l.google.com.

6. TXT Record Analysis:

- SPF Entry: v=spf1 include:_spf.google.com include:email.chargebee.com include:7168674.spf05.hubspotemail.net ~all
    - This SPF (Sender Policy Framework) record helps prevent spoofing by specifying which mail servers are allowed to send on behalf of tryhackme.com.
- Site Verification Entries: There are Google site verification tokens, such as google-site-verification=umR4x8HuzMWF5g3656rYlb-..., used to prove domain ownership for services like Google Search Console.

7. Screenshot:

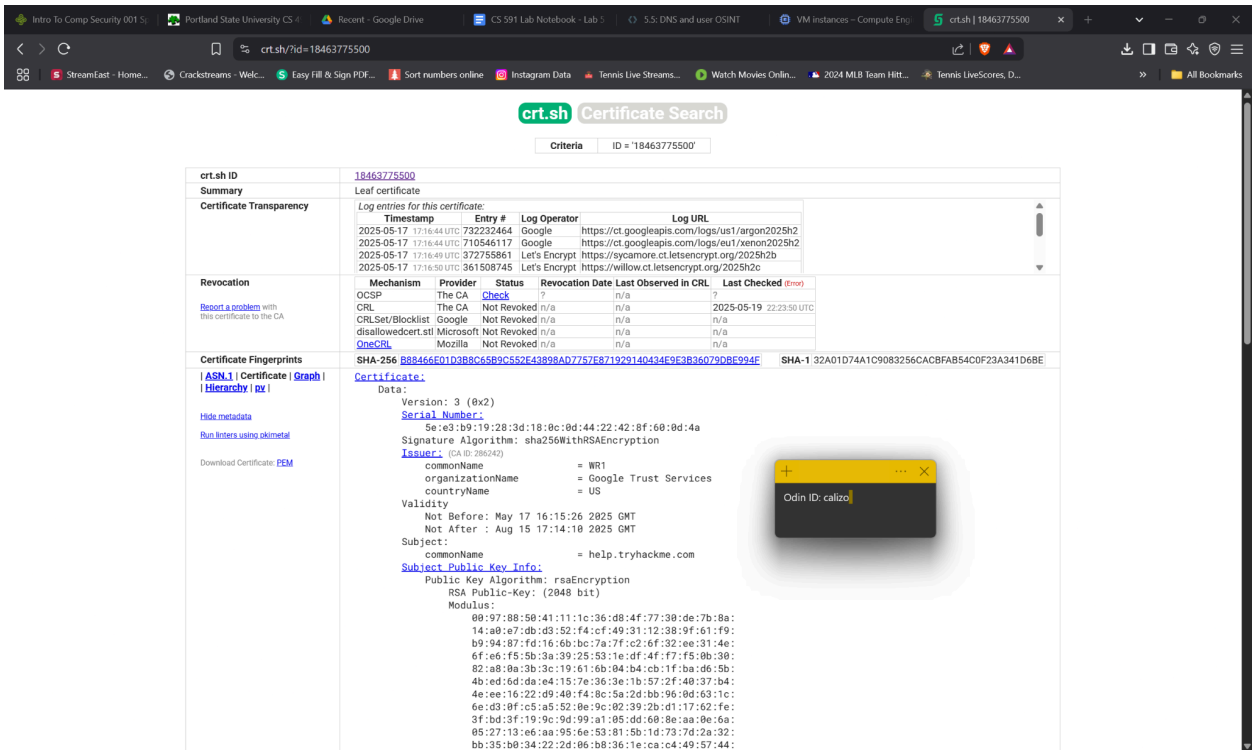


I captured the DNS Dumpster graph showing remote.tryhackme.com, which is a likely candidate for providing remote access. Given its role and naming convention, this host would be a prime target for attackers attempting RDP or SSH access.

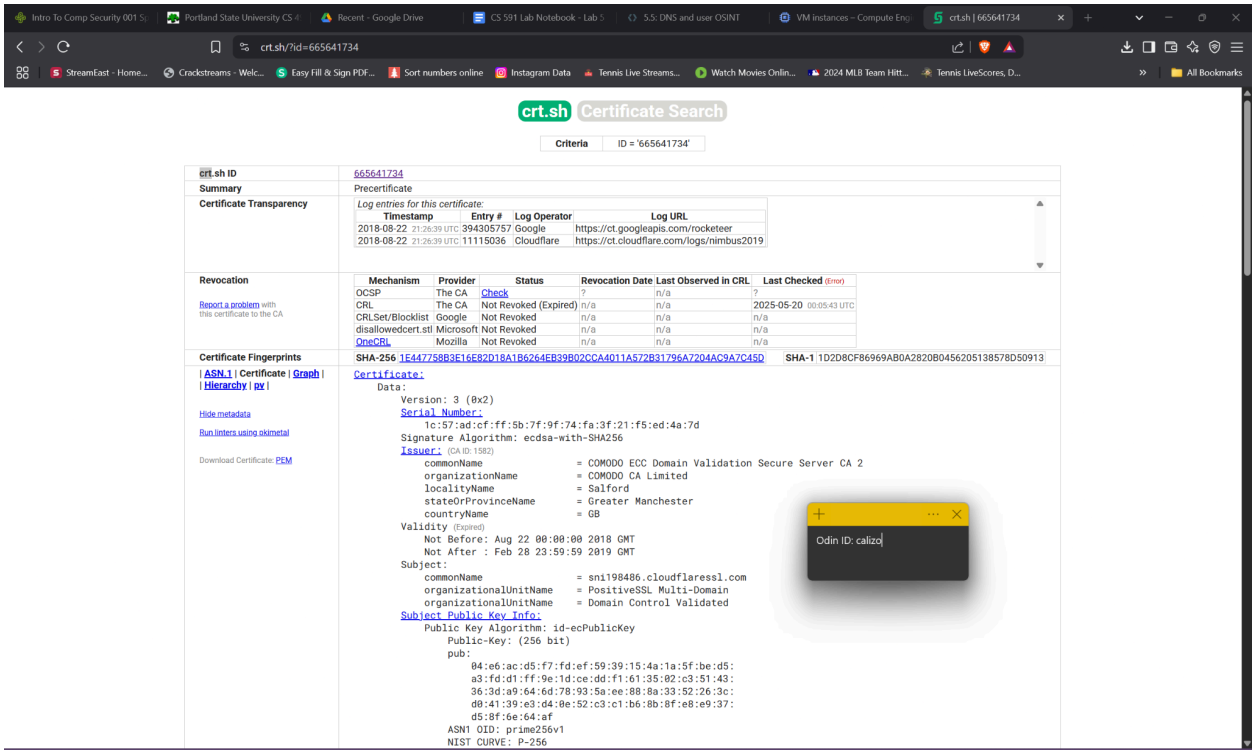
Step 9: Historical Certificate Analysis with crt.sh

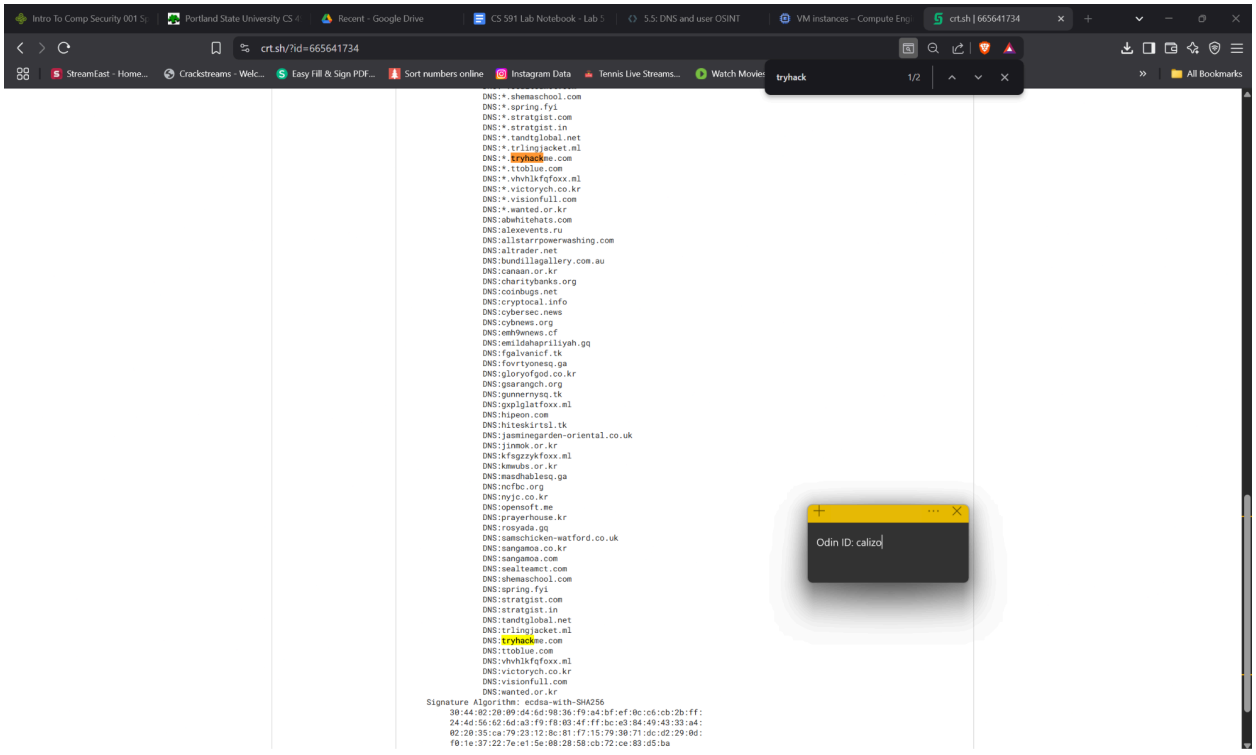
To analyze historical certificate transparency data for both tryhackme.com and pdx.edu, I used crt.sh and conducted manual reconnaissance, followed by JSON parsing using the curl and jq toolset on the Kali VM.

1. Most recent certificate for tryhackme.com:
    - Host: help.tryhackme.com
    - Issued on: May 17, 2025 (17:16:35 UTC)
    - Authority: Google Trust Services (WR1)

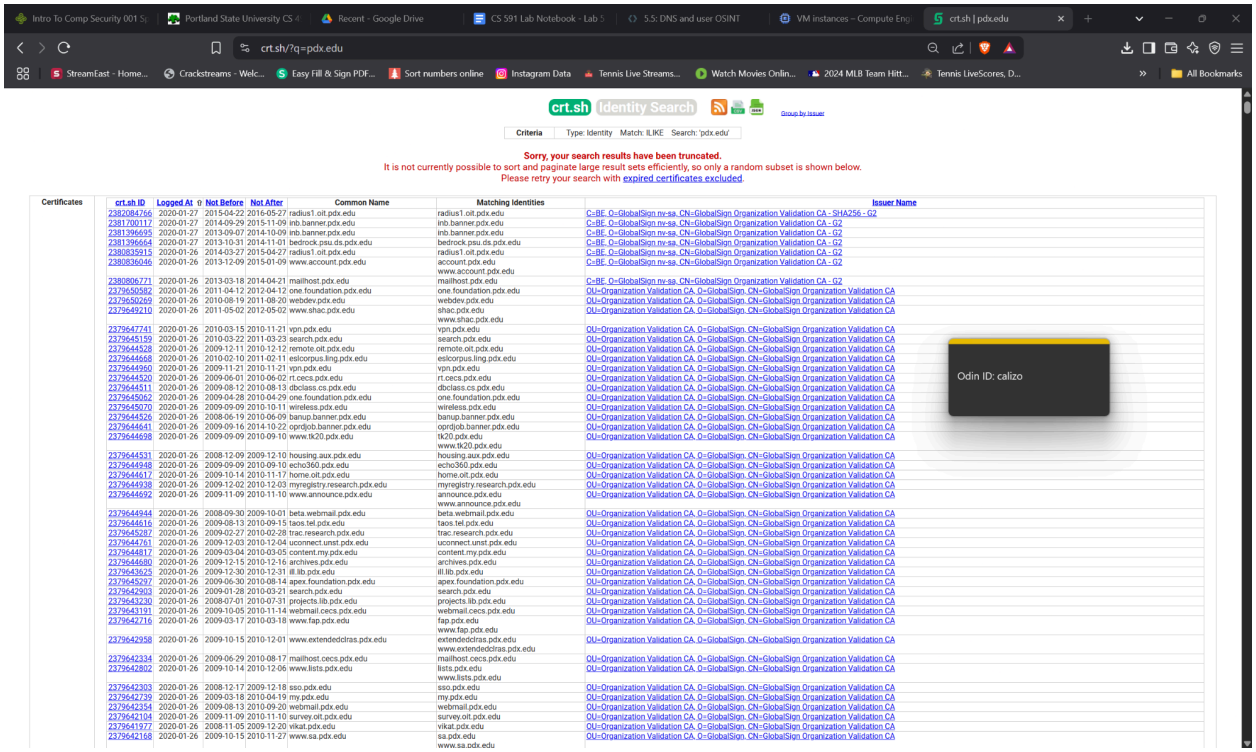


2. First certificate issued for tryhackme.com:
    - Issued on: August 22, 2018 (21:26:39 UTC)
    - Authority: COMODO ECC Domain Validation Secure Server CA 2



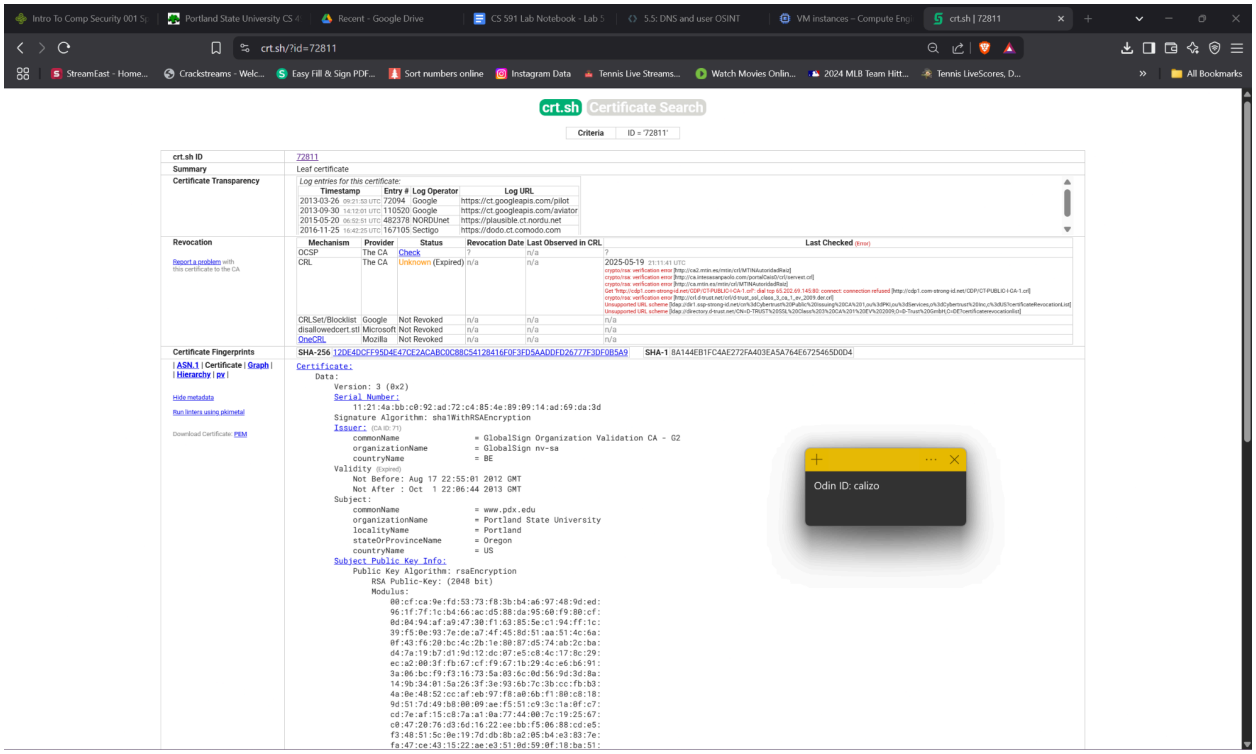


3. Screenshot of results for pdx.edu:



- Included a captured scrollable results set of pdx.edu certificates (many entries truncated by crt.sh due to volume)
- Earliest entry visible was for vpn.pdx.edu, logged on March 15, 2010

4. First certificate issued for Portland State University:
    - Issued to: www.pdx.edu
    - Issued on: August 17, 2012
    - Authority: GlobalSign Organization Validation CA - G2
    - (Certificate SHA-256 ID: 12D4FDCF59D4EF4C7C2ACABC08C54128416F0F3FD5AD0DF26777B0780F05A9A)



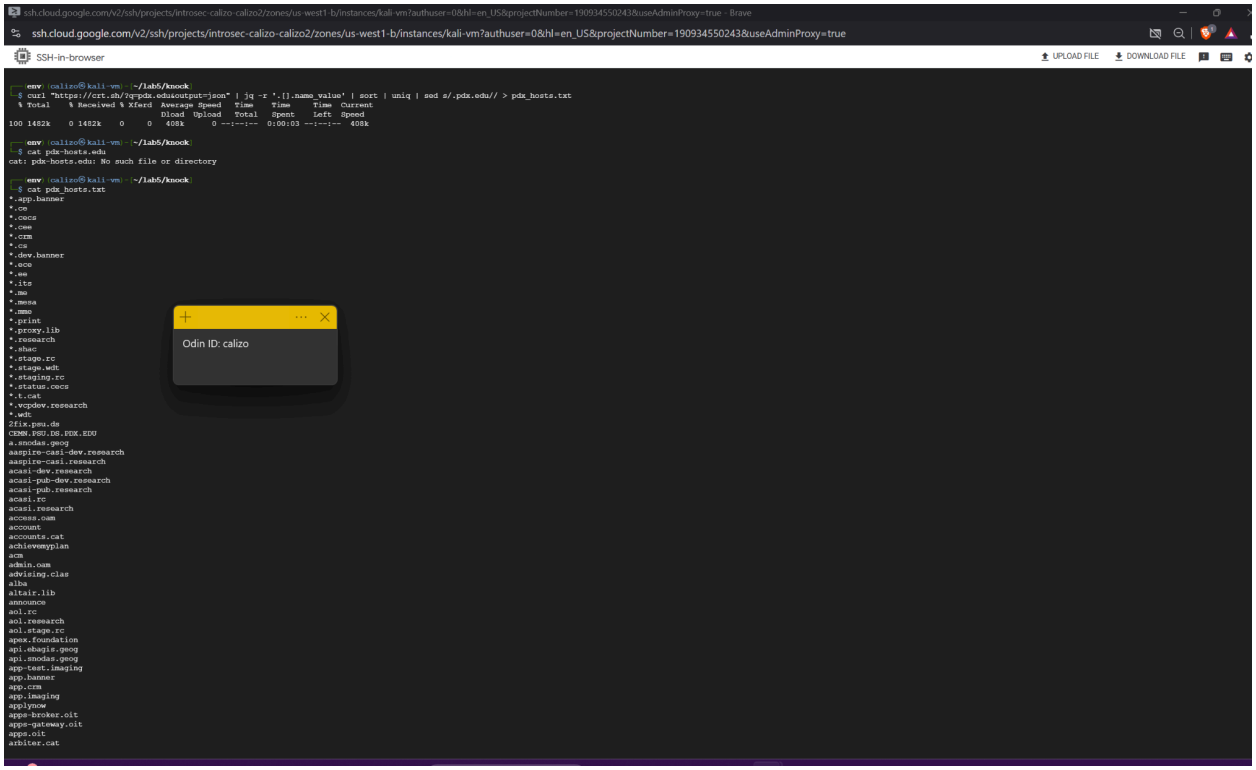
5. Parsed results of pdx.edu DNS history using jq:

Using the provided one-liner:

- curl "https://crt.sh/?q=pdx.edu&output=json" | jq -r '.[].name_value' | sort | uniq | sed s/.pdx.edu// > pdx_hosts.txt

I saved the stripped subdomain list to pdx_hosts.txt. Viewing the output confirms the expected presence of PSU-specific subdomains such as:

- _app_banner
- achievenyplan
- accounts
- advise
- app-window
- apps-gateway.oit
- etc.



(See terminal screenshot above with command execution and cat output of the results)

This step gave a good lens into how TLS certificate history helps fill in long-term visibility gaps that may not show up during one-time scans like DNS Dumpster or Shodan. It's also a subtle reminder that expired certificates, even if outdated, still leave behind a breadcrumb trail for OSINT purposes.

Step 10: ViewDNS & WHOIS Historical Reconnaissance

Objective: Investigate the historical DNS and registration details of tryhackme.com and pdx.edu using WHOIS and ViewDNS tools.

1. When was tryhackme.com created?
Using the WHOIS query for tryhackme.com, I confirmed that the domain was first registered on July 5th, 2018, and is currently managed by Namecheap Inc. The expiration is set to July 5th, 2034, which implies long-term domain planning and stability from the site's owners.

2. When did tryhackme.com migrate from DigitalOcean to Cloudflare?

The IP history revealed that tryhackme.com previously used DigitalOcean infrastructure, with entries like:

- 68.183.254.76 (Slough, UK) seen on 2020-05-18
- 142.93.194.248 (North Bergen, US) on 2020-03-05

Starting on 2020-06-10, the IP ownership transitions to Cloudflare, Inc (e.g., 104.31.79.147), and remains consistent through 2025. This indicates the domain officially migrated to Cloudflare in June 2020, likely to leverage their CDN and security features.

3. When was pdx.edu first registered?
Looking at the WHOIS output from EDUCAUSE, the pdx.edu domain has been active since April 15th, 1987. This makes sense historically, as Portland State has maintained a web presence since the early internet days. DNS operations are currently managed by Google Cloud DNS servers (e.g., ns-cloud-e2.googledomains.com).

4. When did Portland State migrate to AWS, and where is the AWS data center located?
The IP history for pdx.edu shows:

- Prior entries like 131.252.115.150 and 131.252.115.23 were owned by PDXNET, the university’s own infrastructure.
- A new entry appears on September 11th, 2023: 54.214.67.95, geolocated in Boardman, Oregon, and attributed to AMAZON-02 (AWS).

Thus, Portland State began migrating to AWS in Fall 2023, and the service is hosted in Boardman, one of the primary data center hubs for the AWS us-west-2 region.

Step 11 & 12: User and Social Media Intelligence

To emulate a real-world attacker’s passive reconnaissance process, I walked through the publicly exposed footprint of MegaCorp One using a combination of social engineering targets, source code leaks, and Shodan intelligence.

1. VAP – Human Resources
While exploring the company's Contact page, I located the HR department's email: hr@megacorpone.com

This address is a prime target for spear-phishing attacks. An adversary could easily send a malware-laced resume via .LNK file or macro-enabled document, preying on the assumption that HR regularly opens attachments from unknown sources.

2. VAP – Sales & Accounts
On the same contact page, the sales department is reachable at:
sales@megacorpone.com

This is a common vector for Business Email Compromise (BEC) attacks, where adversaries impersonate vendors or request fraudulent payments by hijacking the communication channel.

3. VAP – Infrastructure Access
The person responsible for IT and security at MegaCorp One is Alan Grofield, and his listed email is:
agrofield@megacorpone.com
Because of his presumed elevated privileges, Alan is a high-value target. Gaining access to his credentials could lead to full infrastructure compromise.

Step 12: Social Media & Source Code Mining

4. Intern's Username and Password
From a casual inspection of the intern's desk (visible in the provided image), I found a sticky note reading:
walter
TwitterStarz
This strongly suggests the intern's Twitter handle is @walter, and the password is TwitterStarz. Using publicly posted credentials (even accidentally leaked ones) reinforces how physical and digital OPSEC failures can intersect.

5. Password Hash for trivera
After cloning MegaCorp One's GitHub repository, I located the following user entry in the xampp.users file:
trivera:$apr1$A0vSKwao$GV3sgGAj53j.c3GkS4oUC0
This is a classic example of poor password hygiene—credentials hardcoded in a public repository.

6. Hash Algorithm
I used the hashid tool on my Kali VM to analyze the string. The algorithm was correctly identified as:
Apache MD5 (apr1)
which is used in basic .htpasswd configurations and can be cracked using tools like john or hashcat.

Search Engine Recon
7. CVE Count via Shodan
Using dig to resolve the IP of megacorpone.com, I searched the address on Shodan and located the Vulnerabilities section. Shodan reports a total of:
21 CVEs
associated with the exposed server.

8. theHarvester – Email Discovery
Running theHarvester with Bing revealed:
1 email address: sales@megacorpone.com
9. theHarvester – Host Discovery

From the same output:
1 host: ai.megacorpone.com

This validates that the domain has at least one active subdomain publicly indexed.

This step reinforces the importance of proactively monitoring what your organization exposes online—from employee habits to GitHub leaks to DNS and IP mapping. These recon techniques form the foundation for both offensive and defensive OSINT strategies.

Step 13: crosslinked Enumeration

To assess employee exposure for Portland State University, I attempted to use crosslinked, a Python-based OSINT tool that scrapes public search engines for LinkedIn profiles associated with a given organization, then formats candidate email addresses based on a template. I followed the installation instructions successfully:

```
clone https://github.com/m8r0wn/crosslinked
cd crosslinked
virtualenv -p python3 env
source env/bin/activate
pip install -r requirements.txt
```

Then I ran the enumeration script using the email format {first}.{last}@pdx.edu:

```
python3 crosslinked.py -f '{first}.{last}@pdx.edu' 'Portland State'
```

Unfortunately, due to rate-limiting by both Google and Bing, no addresses were returned in names.txt. As a fallback, I manually replicated the query using Google Dorking:
site:linkedin.com/in "Portland State University"

This revealed several PSU-affiliated personnel with publicly viewable LinkedIn profiles, such as:
Wim Wiewel (President)
Julie Smith (Recruitment and Promotion)
Kurt Boedell (Director of Digital Marketing)
Sarah Schwarz (CEO)

This confirms that a motivated adversary could easily gather names from public social networks and use them to construct probable spear-phishing targets.

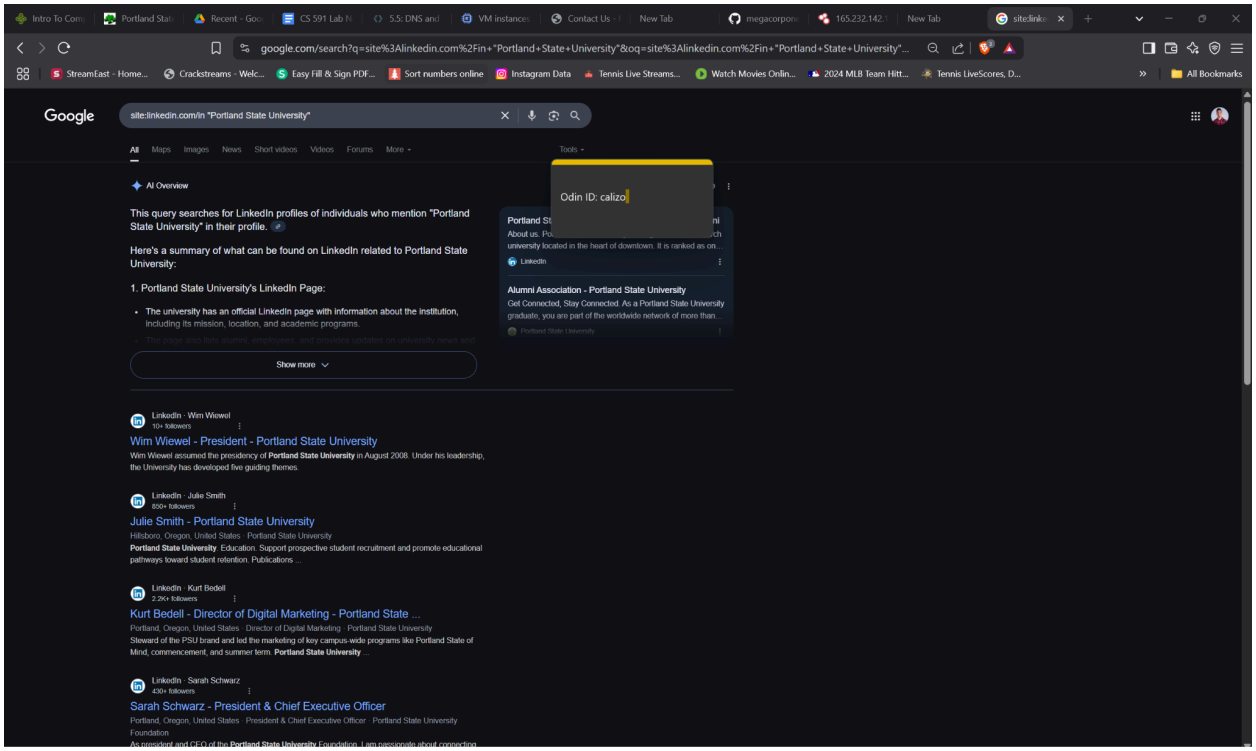
Answers:

How many people did the command return?
→ 0 (due to rate-limiting)

Screenshot of first 10 addresses in names.txt
→ No data to show (names.txt was empty)

Screenshot of people returned from manual search

→



Step 14: recon-ng Whois PoC Mining

Next, I used the recon-ng framework on my Kali VM to enumerate WHOIS Points of Contact (PoCs) for the pdx.edu domain. These contacts are especially valuable for attackers due to their elevated network privileges, making them high-priority for proactive defense.

I launched recon-ng and installed the module:
marketplace search whois
marketplace install recon/domains-contacts/whois_pocs
modules load recon/domains-contacts/whois_pocs

Then I set the target domain and ran the module:

options set SOURCE pdx.edu
run
show contacts

The query successfully identified three PoCs with public email addresses:
abuse@pdx.edu
bass+arin@pdx.edu (Ryan Bass)

noc@pdx.edu (Network Operations Center)

These addresses are now known publicly and could be abused if not properly protected with MFA, strong passwords, and routine security awareness.

Answer:
4. Screenshot of the contact email addresses for PSU

